

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành nhất tới TS. Đỗ Bá Lâm. Trong suốt quá trình thực hiện đề tài, thầy đã luôn dành thời gian định hướng khoa học, tận tình hướng dẫn và hỗ trợ em vượt qua những khó khăn từ giai đoạn bắt đầu cho đến khi hoàn thành đồ án tốt nghiệp này.

Đồng thời, em xin trân trọng cảm ơn tập thể giảng viên Đại học Bách khoa Hà Nội nói chung và các thầy cô giáo tại Trường Công nghệ Thông tin và Truyền thông nói riêng. Những kiến thức chuyên môn sâu sắc cùng phương pháp luận nghiên cứu khoa học mà các thầy cô truyền đạt trong suốt quá trình học tập là hành trang vô cùng quý giá cho em.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình và bạn bè. Đây là nguồn động viên tinh thần to lớn, luôn chia sẻ, đồng hành và tạo mọi điều kiện thuận lợi nhất để em yên tâm tập trung học tập và hoàn thành chặng đường nghiên cứu này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh rủi ro an ninh mạng ngày càng phức tạp, việc bảo vệ các hệ thống liên lạc nội bộ đang trở thành ưu tiên hàng đầu, thúc đẩy sự phổ biến của tiêu chuẩn mã hóa đầu cuối (E2EE) như một giải pháp bảo mật tất yếu. Hiện nay, hầu hết các ứng dụng nhắn tin E2EE hàng đầu vận hành dựa trên giao thức Double Ratchet, một tiêu chuẩn bộc lộ nhiều hạn chế khi mở rộng quy mô nhóm. Nhằm giải quyết bài toán này, Messaging Layer Security (MLS) ra đời như một giao thức tiên phong, tái định nghĩa toàn diện kiến trúc mã hóa nhóm. Tuy nhiên, đặc tả MLS hiện tại tồn tại một khoảng trống học thuật lớn: nó được thiết kế độc quyền cho kiến trúc tập trung và bắt buộc phụ thuộc vào một máy chủ điều phối để đảm bảo thứ tự. Việc triển khai MLS trên môi trường mạng ngang hàng (P2P) phi tập trung vẫn là một thách thức chưa được giải quyết. Từ thực tiễn đó, em quyết định theo đuổi hướng nghiên cứu tích hợp trực tiếp giao thức MLS lên kiến trúc mạng ngang hàng, tước bỏ hoàn toàn máy chủ trung tâm. Hướng đi này được lựa chọn nhằm chứng minh khả năng dung hợp giữa tiêu chuẩn mã hóa nhóm tiên tiến nhất hiện nay và triết lý mạng phi tập trung, từ đó vạch ra mô hình lý thuyết cho một hệ thống truyền thông không tồn tại điểm lỗi duy nhất. Tổng quan giải pháp của nghiên cứu là đề xuất một lớp điều phối phi tập trung. Kiến trúc này thay thế hoàn toàn vai trò của máy chủ trong việc quản lý cập nhật trạng thái nhóm, đồng thời cung cấp khả năng tự động duy trì đồng thuận, nhận diện và hàn gắn phân nhánh khi mạng lưới xảy ra đứt gãy. Đóng góp cốt lõi của đồ án là thiết lập thành công giải pháp hoàn chỉnh cho mô hình ứng dụng MLS trong mạng ngang hàng, sau cùng là một ứng dụng thử nghiệm vận hành ổn định, trực tiếp xác thực tính đúng đắn của nghiên cứu và mở ra nền tảng tham chiếu vững chắc cho các hệ thống bảo mật tương lai.

Sinh viên thực hiện

Lê Tiến Dũng

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	2
1.2.1 Double Ratchet, Sender Keys và Megolm.....	3
1.2.2 MLS và bài toán Delivery Service trong triển khai phân tán.....	4
1.3 Mục tiêu và định hướng giải pháp	5
1.3.1 Mục tiêu của đề án.....	5
1.3.2 Định hướng giải pháp	7
1.4 Đóng góp của đề án	9
1.4.1 Đóng góp về mặt nghiên cứu lý thuyết	9
1.4.2 Đóng góp về mặt hiện thực kỹ thuật.....	10
1.5 Bố cục đề án	10
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	12
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán	12
2.1.1 Từ mô hình tập trung sang mô hình ngang hàng	13
2.1.2 Mô hình hệ thống và giả định nghiên cứu	14
2.1.3 Mâu thuẫn cốt lõi của MLS trên mạng ngang hàng.....	15
2.2 Các kết quả nghiên cứu tương tự	15
2.2.1 Các cơ chế mã hóa nhóm trước MLS.....	15
2.2.2 MLS với Delivery Service	16
2.2.3 Các hướng phi tập trung hóa MLS	17
2.3 Nền tảng MLS và TreeKEM	18
2.3.1 Các khái niệm cốt lõi: Client, Group, Member và Epoch.....	18
2.3.2 Ratchet Tree và TreeKEM.....	18

2.3.3 Proposal, Commit và tiến trình chuyển Epoch.....	19
2.3.4 Tree hash, transcript hash và epoch authenticator	19
2.3.5 Delivery Service và bài toán tuần tự hóa Commit.....	20
2.3.6 Application messages, delayed messages và giới hạn Forward Secrecy	20
2.3.7 External Commit và cơ chế gia nhập lại nhóm	21
2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán	21
2.4.1 Đặc tính của mạng ngang hàng.....	21
2.4.2 go-libp2p và các thành phần truyền thông.....	22
2.4.3 GossipSub và giới hạn về thứ tự thông điệp	22
2.4.4 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng.....	23
2.4.5 Khoảng trống cần giải quyết.....	23
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	25
3.1 Tổng quan giải pháp.....	25
3.1.1 Bài toán cần giải quyết	25
3.1.2 Kiến trúc tổng thể	26
3.1.3 Các thành phần chính của giao thức	27
3.1.4 Luồng hoạt động tổng thể	27
3.2 Giao thức người ghi duy nhất theo Epoch.....	28
3.2.1 Mục tiêu thiết kế	28
3.2.2 Tập ứng viên hợp lệ	28
3.2.3 Hàm bầu chọn Token Holder.....	29
3.2.4 Tạo Proposal và Commit.....	30
3.2.5 Chuyển quyền và failover	31
3.3 Kiểm tra nhất quán Epoch	32
3.3.1 Mục tiêu thiết kế	32

3.3.2 Coordination Envelope	32
3.3.3 Phân loại thông điệp theo Epoch.....	33
3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai	35
3.3.5 Xử lý Proposal bị bỏ sót.....	35
3.4 Cơ chế phục hồi phân nhánh trạng thái	36
3.4.1 Bối cảnh network partition.....	36
3.4.2 Phát hiện fork bằng heartbeat trạng thái	36
3.4.3 Hàm trọng số nhánh	37
3.4.4 Quy trình Fork Healing.....	39
3.4.5 Lưu trạng thái bền vững trong quá trình hàn gắn	40
3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock	41
3.5.1 Lý do cần HLC	41
3.5.2 Tạo timestamp khi gửi tin.....	41
3.5.3 Cập nhật HLC khi nhận tin	42
3.5.4 Thứ tự hiển thị tất định	43
3.6 Quản lý trạng thái Go-Rust và lưu trữ cục bộ.....	44
3.6.1 Lý do sử dụng Rust Sidecar phi trạng thái.....	44
3.6.2 Phân chia trách nhiệm giữa SQLite và Rust Sidecar.....	44
3.6.3 Luồng xử lý khi gửi tin nhắn ứng dụng.....	44
3.6.4 Luồng xử lý khi thay đổi thành viên.....	45
3.7 Tính đúng đắn trực giác của phương pháp	45
3.8 Kết chương.....	46
CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT.....	47
4.1 Mục tiêu và phạm vi phân tích	47
4.2 Mô hình phân tích.....	48

4.3 Bất biến 1: Epoch cục bộ không được thoái lui	48
4.3.1 Phát biểu bất biến.....	48
4.3.2 Cơ sở từ MLS	49
4.3.3 Cơ chế bảo toàn bất biến.....	49
4.4 Bất biến 2: Với cùng một view, chỉ có một Token Holder.....	50
4.4.1 Phát biểu bất biến.....	50
4.4.2 Ý nghĩa đối với concurrent commits	51
4.5 Bất biến 3: Fork trạng thái phải phát hiện được.....	51
4.5.1 Hiện tượng fork trạng thái mật mã	51
4.5.2 Fingerprint trạng thái.....	51
4.6 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh	52
4.6.1 Vấn đề của việc gộp trạng thái.....	52
4.6.2 Quy tắc chọn nhánh.....	53
4.6.3 Giới hạn của lập luận.....	54
4.7 Xử lý thông điệp phát sinh trong nhánh thua.....	54
4.8 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit.....	55
4.8.1 Phân biệt crypto ordering và application ordering	55
4.8.2 Cơ sở của HLC	55
4.8.3 Giới hạn của HLC trong đồ án.....	56
4.9 Phân tích chi phí lý thuyết	56
4.9.1 Chi phí gửi application message	56
4.9.2 Chi phí cập nhật thành viên và rekey.....	56
4.9.3 Chi phí của Single-Writer	57
4.9.4 Chi phí của Epoch Checks và Fork Detection.....	57
4.9.5 Chi phí của Fork Healing	57

4.9.6 Ảnh hưởng của kiến trúc sidecar phi trạng thái	58
4.10 Tổng hợp các bất biến và cơ chế bảo toàn	58
4.11 Giới hạn lý thuyết của phương pháp	59
4.12 Kết chương	59
CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM.....	61
5.1 Mục tiêu và phạm vi đánh giá	61
5.2 Môi trường và phương pháp thí nghiệm	62
5.3 Đánh giá hội tụ trong điều kiện mạng hỗn loạn.....	63
5.4 Đánh giá khả năng phục hồi sau phân vùng mạng	63
5.5 Đánh giá cơ chế Single-Writer và gom Proposal	65
5.6 Đánh giá khả năng mở rộng của thao tác thành viên.....	66
5.7 Đánh giá chi phí mật mã MLS	66
5.8 Chi phí phụ trợ của lớp điều phối	67
5.9 Kiểm chứng Forward Secrecy bằng Rust sidecar thật	68
5.10 Nhận xét và giới hạn đánh giá	68
CHƯƠNG 6. KẾT LUẬN	70
6.1 Kết luận	70
6.2 Hướng phát triển trong tương lai	71
TÀI LIỆU THAM KHẢO.....	75
PHỤ LỤC.....	77
A. HƯỚNG DẪN VIẾT ĐATN	77
A.1 Quy định chung	77
A.2 Ngành học	78
A.3 Đánh dấu (bullet) và đánh số (numering)	78
A.4 Cách thêm bảng	79
A.5 Chèn hình ảnh	79

A.6 Tài liệu tham khảo	80
A.7 Cách viết phương trình và công thức toán học.....	80
A.8 Qui cách đóng quyển.....	80
B. ĐẶC TẢ USE CASE.....	82
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	82
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	82

DANH MỤC HÌNH VẼ

Hình 1.1	Kiến trúc hai lớp của hệ thống đề xuất	7
Hình 2.1	Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS .	13
Hình 2.2	Cấu trúc cây nhị phân Ratchet Tree và tiến trình cập nhật đường dẫn (Path Update) của thành viên N_1	19
Hình 3.1	Kiến trúc triển khai chi tiết của Go Host và Rust Sidecar . . .	26
Hình 5.1	Hội tụ epoch của các nút trong chaos test	63
Hình 5.2	Thời gian phục hồi khi độ dài phân vùng thay đổi	64
Hình 5.3	Hiệu quả của cơ chế gom Proposal khi mức đồng thời tăng . .	65
Hình 5.4	Phân rã chi phí phụ trợ của lớp điều phối	67
Hình A.1	Internet vạn vật	79
Hình A.2	Quy cách đóng quyển đồ án	81
Hình A.3	Quy cách đóng quyển đồ án	81

DANH MỤC BẢNG BIỂU

Bảng 4.1	Quy tắc xử lý thông điệp theo epoch	49
Bảng 4.2	Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn	58
Bảng 5.1	Các tiêu chí đánh giá thực nghiệm	62
Bảng 5.2	Thời gian phục hồi sau phân vùng mạng	64
Bảng 5.3	So sánh baseline và cơ chế gom Proposal	65
Bảng 5.4	Chi phí thao tác thành viên theo quy mô nhóm	66
Bảng 5.5	So sánh độ trễ mật mã giữa các mô hình	67
Bảng A.1	Table to test captions and labels.	79

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
BFT	Chịu lỗi Byzantine (Byzantine Fault Tolerance)
DHT	Bảng băm phân tán (Distributed Hash Table)
DMLS	Bảo mật tầng thông điệp phi tập trung (Decentralized Messaging Layer Security)
DS	Dịch vụ phân phối (Delivery Service)
E2EE	Mã hóa đầu cuối (End-to-End Encryption)
FREEK	Thỏa thuận khóa nhóm liên tục chịu rẽ nhánh (Fork-Resilient Continuous Group Key Agreement)
FS	Bảo mật xuôi (Forward Secrecy)
HLC	Đồng hồ logic hỗn hợp (Hybrid Logical Clock)
IP	Giao thức Internet (Internet Protocol)
LAN	Mạng cục bộ (Local Area Network)
mDNS	Hệ thống phân giải tên miền multicast (multicast Domain Name System)
MLS	Bảo mật tầng thông điệp (Messaging Layer Security)
NAT	Biến đổi địa chỉ mạng (Network Address Translation)
P2P	Mạng ngang hàng (Peer-to-Peer)
PCS	Bảo mật sau thỏa hiệp (Post-Compromise Security)
QUIC	Kết nối Internet UDP nhanh (Quick UDP Internet Connections)
RFC	Tài liệu đặc tả chuẩn / Yêu cầu bình luận (Request for Comments)

Thuật ngữ	Ý nghĩa
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
WAN	Mạng diện rộng (Wide Area Network)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh các hoạt động trao đổi thông tin nội bộ ngày càng phụ thuộc vào hạ tầng số, bảo vệ nội dung liên lạc trở thành một yêu cầu quan trọng đối với tổ chức, doanh nghiệp và cơ quan quản lý nhà nước. Rò rỉ thông tin nhạy cảm, tài liệu nghiệp vụ hoặc nội dung trao đổi nội bộ có thể gây thiệt hại trực tiếp về kinh tế, uy tín và an toàn vận hành. Một trong những hướng tiếp cận quan trọng để giảm rủi ro này là mã hóa đầu cuối (End-to-End Encryption – E2EE), trong đó nội dung thông điệp chỉ được giải mã tại các thiết bị đầu cuối hợp lệ, còn các thành phần trung gian trong quá trình truyền tải không được quyền truy cập bản rõ của thông điệp [1], [2].

Trong nhiều hệ thống nhắn tin bảo mật hiện nay, máy chủ vẫn giữ vai trò quan trọng trong việc định tuyến tin nhắn, lưu trữ khóa công khai ban đầu, hỗ trợ thiết bị ngoại tuyến và đồng bộ trạng thái giữa các thành viên. Cách tổ chức này không đồng nghĩa với việc nội dung tin nhắn luôn bị lộ cho máy chủ, nhưng nó vẫn tạo ra sự phụ thuộc đáng kể vào hạ tầng vận hành trung tâm. Nếu hạ tầng này gặp sự cố, bị tấn công từ chối dịch vụ, bị cấu hình sai hoặc bị kiểm soát bởi một tác nhân không mong muốn, khả năng liên lạc của toàn hệ thống có thể bị ảnh hưởng. Ngoài ra, ngay cả khi nội dung được mã hóa, các thành phần trung gian vẫn có thể quan sát một số loại siêu dữ liệu như thời điểm gửi, tần suất liên lạc, kích thước thông điệp, thông tin định tuyến hoặc quan hệ giữa các thiết bị, tùy theo thiết kế của từng hệ thống [2].

Với các môi trường đặc thù như mạng nội bộ biệt lập, mạng tạm thời không có Internet hoặc tổ chức cần kiểm soát chặt chẽ dữ liệu, sự phụ thuộc vào một máy chủ Internet trung tâm trở thành hạn chế thực tế. Các hệ thống này cần một mô hình trong đó dữ liệu ưu tiên được lưu trữ cục bộ, thiết bị có thể tiếp tục hoạt động khi mất kết nối với máy chủ bên ngoài, và các nút mạng có khả năng trao đổi trực tiếp khi cùng xuất hiện trong mạng LAN, VPN hoặc một mạng ngang hàng. Định hướng này gần với mô hình Local-First và Peer-to-Peer (P2P): dữ liệu và trạng thái ứng dụng được ưu tiên giữ tại thiết bị của người dùng, còn đồng bộ chỉ diễn ra khi các thiết bị có điều kiện kết nối.

Tuy nhiên, xây dựng một hệ thống liên lạc nhóm vừa mã hóa đầu cuối, vừa hoạt động trên mạng ngang hàng bất đồng bộ là một bài toán khó. Với liên lạc hai bên, Double Ratchet đã trở thành một cơ chế phổ biến để tạo khóa mới liên tục cho từng thông điệp, cung cấp tính bảo mật xuôi và khả năng phục hồi sau thỏa hiệp ở mức

phiên hai bên [3]. Khi chuyển sang nhóm có nhiều thành viên, bài toán không chỉ là mã hóa từng tin nhắn, mà còn là duy trì một trạng thái mật mã nhóm nhất quán giữa nhiều thiết bị có thể online, offline, gửi tin không đồng thời và nhận thông điệp theo các thứ tự khác nhau.

Messaging Layer Security (MLS), được chuẩn hóa trong RFC 9420, được thiết kế để giải quyết bài toán thiết lập khóa nhóm bất đồng bộ với các bảo đảm quan trọng như Forward Secrecy (FS) và Post-Compromise Security (PCS). MLS sử dụng cấu trúc cây mật mã để làm cho chi phí cập nhật khóa tăng theo logarit kích thước nhóm, thay vì phải phân phối khóa lại theo cách tuyến tính hoặc bình phương trong nhiều kịch bản nhóm lớn [1]. Tuy nhiên, MLS không phải là một giao thức nhắn tin hoàn chỉnh. Nó cần được tích hợp vào một hệ thống nhắn tin cụ thể, trong đó phần ứng dụng phải quyết định cách định tuyến thông điệp, cách xác thực danh tính, cách lưu trữ vật liệu khóa ban đầu và đặc biệt là cách xử lý thứ tự các thông điệp thay đổi trạng thái nhóm [2].

Chính tại đây xuất hiện vấn đề trung tâm của đề tài. Trong các triển khai có Delivery Service nhất quán mạnh, dịch vụ phân phối có thể đóng vai trò sắp xếp thứ tự Commit và giải quyết xung đột khi nhiều thành viên cùng cập nhật nhóm tại một epoch. Trong môi trường P2P không có một máy chủ trung tâm điều phối, các Commit có thể được phát tán qua mạng theo các thứ tự khác nhau, khiến các thiết bị khác nhau áp dụng những thứ tự thay đổi trạng thái nhóm khác nhau, dẫn đến các trạng thái khác nhau. Nếu không có cơ chế điều phối nhất quán cho các thiết bị, nhóm MLS có thể rơi vào trạng thái phân nhánh mật mã, làm cho các thành viên không còn chia sẻ cùng một trạng thái khóa nhóm, dẫn đến không thể mã hóa và giải mã tin nhắn của nhau trong nhóm.

Từ bối cảnh đó, đề tài “Nghiên cứu và ứng dụng giao thức điều phối phi tập trung cho MLS trên mạng ngang hàng” tập trung vào bài toán thiết kế một lớp điều phối ở tầng ứng dụng để hỗ trợ vận hành MLS trong môi trường P2P. Mục tiêu không phải thay đổi lõi mật mã của MLS, mà là bổ sung cơ chế điều phối, kiểm tra epoch, xử lý xung đột và phục hồi sau phân mảnh mạng để MLS có thể được sử dụng an toàn hơn trong một hệ thống liên lạc nhóm phi tập trung.

1.2 Các giải pháp hiện tại và hạn chế

Các hướng tiếp cận hiện tại cho mã hóa nhóm có thể được chia thành hai nhóm chính. Nhóm thứ nhất dựa trên các kênh bảo mật hai bên, sau đó mở rộng sang nhóm bằng cách phân phối khóa gửi tin cho từng thành viên. Nhóm thứ hai dựa trên giao thức thiết lập khóa nhóm chuyên biệt, tiêu biểu là MLS và TreeKEM. Mỗi hướng đều có ưu điểm riêng, nhưng cũng để lại các giới hạn quan trọng khi

đặt trong yêu cầu vận hành P2P hoàn toàn phi tập trung.

1.2.1 Double Ratchet, Sender Keys và Megolm

Double Ratchet là thuật toán trao đổi thông điệp mã hóa giữa hai bên dựa trên một bí mật chung đã được thiết lập từ trước. Sau giai đoạn khởi tạo, hai bên liên tục dẫn xuất khóa mới cho từng thông điệp thông qua sự kết hợp giữa ratchet đối xứng và ratchet Diffie–Hellman. Cơ chế ratchet đối xứng giúp mỗi thông điệp sử dụng một khóa riêng biệt và có thể xóa sau khi dùng, trong khi ratchet Diffie–Hellman đưa thêm entropy mới vào phiên liên lạc khi hai bên trao đổi khóa công khai mới. Nhờ đó, Double Ratchet cung cấp khả năng bảo vệ thông điệp quá khứ và khôi phục bảo mật cho thông điệp tương lai trong một số kịch bản thỏa hiệp khóa [3]. Đây là nền tảng quan trọng của nhiều giao thức nhắn tin bảo mật hiện đại.

Khi mở rộng từ liên lạc hai bên sang liên lạc nhóm, một hướng tiếp cận phổ biến là sử dụng cơ chế Sender Keys. Trong mô hình này, mỗi thành viên tự sinh một khóa gửi tin riêng và phân phối khóa đó đến các thành viên còn lại thông qua các kênh bảo mật hai bên. Sau khi khóa gửi tin đã được phân phối, các thông điệp tiếp theo của cùng người gửi có thể được mã hóa thành một bản mã duy nhất, sau đó được máy chủ hoặc lớp mạng phân phối tới các thành viên trong nhóm. Vì vậy, Sender Keys cải thiện đáng kể chi phí gửi tin so với mô hình mã hóa riêng biệt cho từng người nhận trong mỗi lần gửi thông điệp [1], [4].

Tuy nhiên, giới hạn chính của Sender Keys nằm ở khả năng đạt bảo mật sau thỏa hiệp và chi phí quản lý thay đổi thành viên. Khi khóa gửi tin của một thành viên bị lộ, kẻ tấn công có thể tiếp tục giải mã thụ động các thông điệp do thành viên đó gửi cho đến khi khóa mới được sinh và phân phối lại. Việc tạo và phân phối lại khóa gửi tin cho một thành viên có chi phí tăng tuyến tính theo kích thước nhóm; nếu nhiều thành viên cùng cần cập nhật khóa, tổng chi phí điều phối và truyền thông có thể tăng đáng kể [1]. Do đó, Sender Keys phù hợp với nhiều hệ thống nhắn tin thực tế, nhưng chưa phải lời giải tối ưu cho các nhóm lớn có biến động thành viên thường xuyên hoặc yêu cầu Post-Compromise Security mạnh.

Megolm là cơ chế mã hóa nhóm được sử dụng trong hệ sinh thái Matrix, với mục tiêu chính là tối ưu chi phí gửi tin trong các phòng trò chuyện nhiều thành viên. Thay vì thiết lập một kênh mã hóa riêng cho từng cặp thiết bị, Megolm sử dụng mô hình *group ratchet*: một sender tạo ra một outbound session và dùng session này để mã hóa chuỗi thông điệp gửi vào phòng [5]. Cách thiết kế này giúp việc gửi tin trong nhóm trở nên nhẹ hơn đáng kể, vì mỗi thông điệp không cần được mã hóa lại riêng biệt cho từng người nhận.

Đánh đổi của Megolm nằm ở chỗ session state có vòng đời dài hơn so với các

cơ chế ratchet chặt chẽ, vốn được gắn theo từng tin nhắn. Khi ratchet state của một Megolm session bị lộ tại một thời điểm, kẻ tấn công có thể suy ra các tin nhắn từ thời điểm đó trở về sau trong cùng session. Vì vậy, Megolm không cung cấp PCS mạnh như mong muốn đối với các nhóm có yêu cầu bảo mật cao; đồng thời, đặc tả của Matrix cũng ghi nhận các giới hạn liên quan đến transcript consistency và forward secrecy [5]. Trong thực tế, các hệ thống dùng Megolm cần định kỳ tạo session mới và phân phối lại session key để thu hẹp khoảng thời gian bị ảnh hưởng nếu thiết bị hoặc session state bị compromise.

Như vậy, Double Ratchet, Sender Keys và Megolm đều là các thiết kế có giá trị thực tiễn lớn và đã chứng minh hiệu quả trong nhiều hệ thống nhắn tin bảo mật. Tuy nhiên, khi bài toán chuyển sang nhóm lớn, membership thay đổi thường xuyên và yêu cầu PCS mạnh hơn, các cơ chế này bắt đầu bộc lộ giới hạn ở khâu rekey, quản lý session và phục hồi sau compromise. Đây là khoảng trống mà Messaging Layer Security (MLS) hướng tới giải quyết bằng cách đưa việc quản lý khóa nhóm về một cấu trúc có hệ thống hơn, trong đó TreeKEM cho phép cập nhật trạng thái nhóm với chi phí hiệu quả hơn và phù hợp hơn cho truyền thông nhóm quy mô lớn.

1.2.2 MLS và bài toán Delivery Service trong triển khai phân tán

MLS được chuẩn hóa nhằm cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ, hiệu quả và có bảo đảm FS/PCS cho các nhóm từ quy mô nhỏ đến lớn. Thay vì dựa vào việc mỗi thành viên phân phối khóa gửi tin riêng lẻ cho toàn bộ nhóm, MLS biểu diễn trạng thái nhóm bằng một cây mật mã. Khi thành viên thay đổi khóa hoặc khi nhóm thay đổi thành viên, thao tác cập nhật chỉ cần tác động đến một đường đi trong cây, làm cho chi phí cập nhật tăng theo logarit kích thước nhóm trong mô hình lý thuyết của TreeKEM [1].

Dù vậy, RFC 9750 nhấn mạnh rằng MLS không phải là một giao thức nhắn tin hoàn chỉnh, mà là một giao thức mật mã cần được nhúng vào hệ thống ứng dụng cụ thể [2]. Một hệ thống sử dụng MLS thường cần hai chức năng trừu tượng: Authentication Service để ràng buộc danh tính ứng dụng với khóa xác thực, và Delivery Service để lưu trữ vật liệu khóa ban đầu, nhận và phân phối thông điệp giữa các thành viên [2]. Quan trọng hơn, RFC 9750 không bắt buộc Delivery Service phải là một máy chủ trung tâm duy nhất. Delivery Service có thể được triển khai trong mô hình một nhà cung cấp, liên miền, hoặc thậm chí được hiện thực hóa phi tập trung bằng cách truyền thông điệp MLS qua mạng ngang hàng [2].

Điểm khó nhất của việc triển khai MLS trong mạng ngang hàng nằm ở tính nhất quán của các thông điệp thay đổi trạng thái nhóm. Trong MLS, Commit là thông

điệp khởi tạo một epoch mới bằng cách yêu cầu các thành viên áp dụng một tập thay đổi như thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa [2]. Nếu hai thành viên cùng tạo Commit dựa trên cùng một epoch, hệ thống phải có cách quyết định Commit nào được chấp nhận trước, hoặc có cách hòa giải khi các thành viên nhìn thấy các Commit theo thứ tự khác nhau. Trong hệ thống sử dụng Delivery Service nhất quán mạnh, nó có thể phá thống nhất thứ tự bằng cách áp đặt một thứ tự tuyến tính cho các Commit. Trong các hệ thống Delivery Service theo triết lý nhất quán cuối cùng hoặc trong mạng P2P, thông điệp có thể đến các thiết bị theo thứ tự khác nhau, và khi đó client phải tham gia vào quá trình hòa giải [2].

Một hướng nghiên cứu gần với bài toán này là Decentralized MLS (DMLS) của Kohbrok. Thay vì giữ nguyên MLS và giải quyết xung đột ở tầng hệ thống, DMLS chọn cách mở rộng trực tiếp bản thân giao thức MLS để hỗ trợ xử lý các Commit đến ngoài thứ tự trong môi trường không có máy chủ trung tâm áp đặt thứ tự. Hướng tiếp cận này cho phép client giữ lại một phần tài nguyên key mã hóa để xử lý các nhánh Commit khác nhau với mức ảnh hưởng tới Forward Secrecy thấp hơn so với việc giữ khóa cũ một cách thô sơ [6], [7]. Tuy nhiên, đặc trưng này cũng là điểm yếu chính của DMLS: để xử lý out-of-order commits, client vẫn phải duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, làm tăng độ phức tạp triển khai. Quan trọng hơn, DMLS hiện vẫn chỉ là một Internet-Draft đã hết hạn, chưa phải là một chuẩn IETF hoàn chỉnh [7]. Vì vậy, đề án này lựa chọn hướng thực dụng hơn: giữ nguyên lõi MLS và giải quyết bài toán ordering, fork detection và fork healing ở tầng điều phối phía ngoài tầng MLS.

1.3 Mục tiêu và định hướng giải pháp

1.3.1 Mục tiêu của đề án

Mục tiêu của đề án là nghiên cứu, thiết kế và hiện thực một lớp điều phối phi tập trung ở tầng ứng dụng nhằm hỗ trợ vận hành Messaging Layer Security (MLS) trên mạng ngang hàng. Lớp điều phối này không thay đổi lõi mật mã của MLS, mà đóng vai trò như một lớp bao quanh, chịu trách nhiệm xử lý các vấn đề phát sinh khi hệ thống không còn phụ thuộc vào một Delivery Service trung tâm có khả năng áp đặt thứ tự Commit một cách nhất quán.

Về mặt lý thuyết, đề án tập trung vào ba mục tiêu chính. Thứ nhất, đề án mô hình hóa bài toán xung đột Commit trong MLS khi các thông điệp handshake được truyền qua môi trường P2P bất đồng bộ, nơi thông điệp có thể đến trễ, đến sai thứ tự hoặc bị chia cắt bởi network partition. Thứ hai, đề án đề xuất cơ chế điều phối dựa trên nguyên tắc *Single-Writer per Epoch*, trong đó tại mỗi epoch chỉ một nút được quyền tạo Commit, qua đó giảm đáng kể khả năng xuất hiện concurrent commits

trong điều kiện mạng liên thông. Thứ ba, đồ án xây dựng cơ chế phát hiện và phục hồi khi phân nhánh trạng thái nhóm vẫn xảy ra, đặc biệt trong các tình huống mạng bị phân mảnh vật lý rồi kết nối trở lại.

Về mặt ứng dụng, đồ án hướng tới xây dựng một ứng dụng cộng tác nội bộ bảo mật, hỗ trợ các chức năng cốt lõi như chat nhóm, lưu trữ dữ liệu cục bộ, đồng bộ trạng thái giữa các thiết bị, chia sẻ tệp tin và phát hiện peer trong mạng ngang hàng. Ứng dụng được thiết kế theo định hướng *Local-First* và giảm phụ thuộc vào hạ tầng máy chủ trung tâm, nhờ đó có thể vận hành trong các kịch bản mạng nội bộ, mạng tạm thời mất Internet hoặc môi trường cần kiểm soát dữ liệu chặt chẽ. Nguyên mẫu này đóng vai trò kiểm chứng tính khả thi của giao thức điều phối đề xuất, đồng thời cung cấp cơ sở thực nghiệm để đánh giá độ trễ, chi phí truyền thông, khả năng phục hồi sau mất kết nối và giới hạn mở rộng của hệ thống.

Trong môi trường phân tán bất đồng bộ, đặc biệt khi xảy ra *network partition*, việc đồng thời bảo đảm tính nhất quán mạnh, tính sẵn sàng và khả năng chịu phân mảnh là không thể. Theo định lý CAP, khi các nút trong hệ phân tán bị chia cắt bởi lỗi mạng, hệ thống buộc phải đánh đổi giữa *Consistency* và *Availability* nếu vẫn muốn duy trì *Partition Tolerance* [8]. Đối với bài toán của đồ án, *Partition Tolerance* là yêu cầu gần như bắt buộc vì hệ thống được thiết kế cho môi trường P2P, nơi thiết bị có thể offline, độ trễ mạng không ổn định và các phân vùng mạng có thể xuất hiện bất cứ lúc nào.

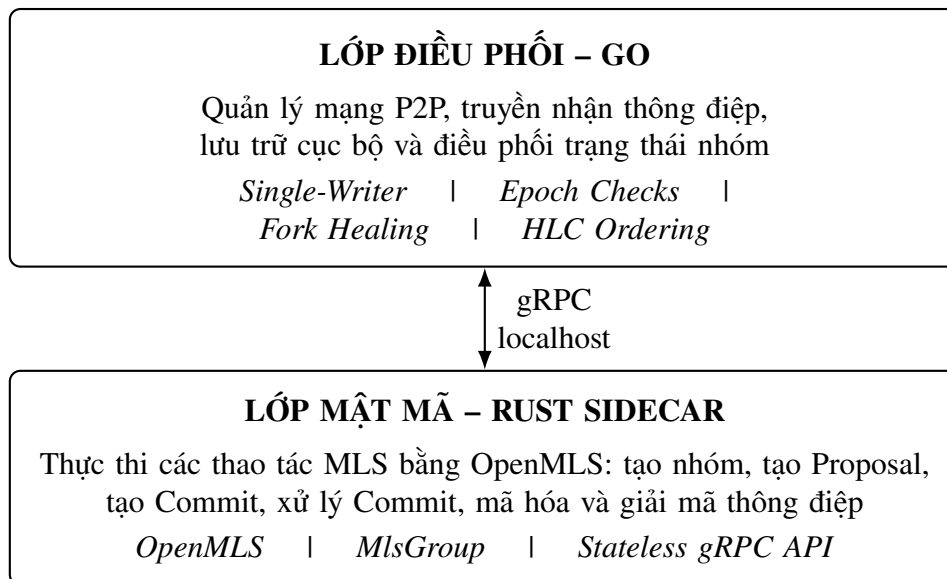
Từ góc nhìn này, đồ án lựa chọn hướng thiết kế thiên về *AP-oriented*: ưu tiên để các phân vùng mạng vẫn có khả năng hoạt động cục bộ trong thời gian bị chia cắt, thay vì dừng toàn bộ hệ thống để chờ một cơ chế đồng thuận toàn cục. Các cơ chế đồng thuận mạnh như BFT, blockchain hoặc quorum-based consensus có thể duy trì một thứ tự Commit duy nhất, nhưng thường đánh đổi bằng độ trễ lớn, chi phí truyền thông cao và mất tính sẵn sàng ở các phân vùng không đạt quorum. Điều này không phù hợp với mục tiêu *Local-First* và P2P của đồ án.

Tuy nhiên, do trạng thái MLS là trạng thái mật mã chứ không phải dữ liệu ứng dụng có thể hợp nhất tùy ý, hệ thống không thể chấp nhận phân kỳ vô hạn. Vì vậy, giải pháp đề xuất không theo hướng AP thuần túy, mà kết hợp giữa tính sẵn sàng cục bộ và cơ chế hội tụ có kiểm soát. Cụ thể, giao thức hạn chế concurrent commits trong điều kiện mạng liên thông bằng Single-Writer, phát hiện phân nhánh trạng thái khi partition xảy ra, và sử dụng Fork Healing để đưa các phân vùng hội tụ trở lại một trạng thái MLS hợp lệ duy nhất sau khi mạng được kết nối lại.

1.3.2 Định hướng giải pháp

Từ các mục tiêu đã nêu, đề án lựa chọn hướng tiếp cận xây dựng một lớp điều phối phi tập trung đặt phía trên lõi mật mã MLS. Lớp này không thay đổi các thuật toán mật mã của MLS, mà bổ sung các cơ chế cần thiết để MLS có thể vận hành trong môi trường mạng ngang hàng, nơi không còn một Delivery Service trung tâm đảm nhiệm vai trò phân phối và sắp xếp thứ tự Commit.

Kiến trúc hệ thống được chia thành hai lớp chính, như minh họa trong Hình 1.1. Lớp thứ nhất là *Coordination Layer*, được hiện thực bằng Go. Lớp này chịu trách nhiệm quản lý mạng P2P, truyền nhận thông điệp qua libp2p/GossipSub, duy trì trạng thái ứng dụng cục bộ, điều phối quá trình chuyển epoch, phát hiện xung đột và xử lý các tình huống mất đồng bộ. Lớp thứ hai là *Crypto Layer*, được hiện thực bằng Rust trên cơ sở thư viện OpenMLS. Lớp này đảm nhiệm toàn bộ các thao tác mật mã của MLS như tạo nhóm, tạo Proposal, tạo Commit, xử lý Commit, mã hóa và giải mã thông điệp. Hai lớp giao tiếp với nhau thông qua gRPC trên localhost theo mô hình sidecar.



Hình 1.1: Kiến trúc hai lớp của hệ thống đề xuất

Trong kiến trúc này, Go đóng vai trò điều phối hệ thống, còn Rust đóng vai trò lõi mật mã tin cậy. Việc tách hai lớp giúp hệ thống tận dụng được ưu điểm của từng ngôn ngữ: Go phù hợp với xử lý đồng thời, lập trình mạng và tích hợp libp2p; Rust phù hợp với các thao tác mật mã yêu cầu an toàn bộ nhớ và hiệu năng cao. Đồng thời, cách tổ chức này giúp đề án tránh việc tự lập trình lại MLS, thay vào đó sử dụng thư viện OpenMLS như một thư viện chuẩn RFC 9420.

Lớp điều phối được xây dựng xung quanh bốn cơ chế chính. Cơ chế thứ nhất là *Single-Writer per Epoch*. Tại mỗi epoch, hệ thống xác định một thành viên duy

nhất có quyền tạo Commit, gọi là *Token Holder*. Token Holder được chọn bằng một hàm tất định dựa trên định danh nhóm, số epoch và tập thành viên hợp lệ. Nhờ vậy, trong điều kiện các nút có cùng góc nhìn về trạng thái nhóm, toàn bộ hệ thống có thể tự xác định cùng một nút chịu trách nhiệm Commit mà không cần thêm một vòng bỏ phiếu hay đồng thuận mạng.

Cơ chế thứ hai là *Epoch Consistency Checks*. Mỗi thông điệp MLS nhận được từ mạng đều được kiểm tra theo `group_id`, epoch, loại thông điệp và trạng thái cục bộ trước khi được chuyển xuống lớp mật mã. Các thông điệp thuộc epoch cũ, epoch tương lai hoặc có dấu hiệu không tương thích với trạng thái hiện tại sẽ được xử lý theo chính sách riêng, chẳng hạn loại bỏ, đưa vào bộ đệm tạm thời hoặc kích hoạt đồng bộ trạng thái. Cơ chế này đóng vai trò như lớp bảo vệ giữa môi trường P2P bất đồng bộ và máy trạng thái mật mã của MLS.

Cơ chế thứ ba là *Fork Detection and Healing*. Trong trường hợp mạng bị phân mảnh, các phân vùng khác nhau có thể tiến hóa độc lập và tạo ra các nhánh trạng thái MLS khác nhau. Khi kết nối được khôi phục, các nút trao đổi thông tin trạng thái như epoch, tree hash, epoch authenticator và commit hash để phát hiện phân mảnh. Sau đó, hệ thống sử dụng một chính sách tất định để chọn nhánh tiếp tục, còn các nút ở nhánh không được chọn sẽ dừng sử dụng trạng thái cũ và gia nhập lại nhánh hợp lệ thông qua cơ chế phù hợp của MLS như Welcome hoặc External Commit [1], [2]. Các thông điệp ứng dụng phát sinh trong thời gian phân mảnh, nếu cần phục hồi, sẽ được mã hóa lại và gửi lại dưới trạng thái MLS mới thay vì phát lại nguyên trạng bản mã cũ.

Cơ chế thứ tư là *HLC Message Ordering*. Commit trong MLS chỉ sắp xếp các thay đổi trạng thái mật mã, trong khi nhiều thông điệp ứng dụng vẫn có thể được gửi song song trong cùng một epoch. Do các thiết bị trong mạng P2P không thể giả định có đồng hồ vật lý đồng bộ tuyệt đối, hệ thống sử dụng Hybrid Logical Clock để gắn nhãn thời gian cho thông điệp ứng dụng, từ đó tạo thứ tự hiển thị ổn định hơn giữa các thiết bị [9]. Cơ chế này chỉ phục vụ lớp ứng dụng và không thay thế các kiểm tra mật mã, kiểm tra epoch hay quá trình xử lý Commit của MLS.

Về mặt triển khai, Rust sidecar hoạt động như một dịch vụ mật mã phi trạng thái từ góc nhìn của Go Host. Go Host lưu trạng thái nhóm bền vững trong SQLite, truyền snapshot `GroupState` hiện tại sang Rust khi cần thực hiện thao tác MLS, và nhận lại trạng thái mới sau khi Rust xử lý. Thiết kế này giữ ranh giới trách nhiệm rõ ràng: Go/SQLite là nguồn sự thật của hệ thống, còn Rust tập trung vào việc thực thi OpenMLS một cách an toàn.

Tóm lại, định hướng giải pháp của đề án là giữ nguyên lõi mật mã MLS, đồng

thời bổ sung một lớp điều phối phi tập trung ở tầng hệ thống để thay thế vai trò tuần tự hóa thường do Delivery Service đảm nhiệm. Cách tiếp cận này cho phép hệ thống tận dụng các bảo đảm mật mã của MLS, đồng thời thích nghi với môi trường mạng ngang hàng bất đồng bộ và có khả năng phân mảnh.

1.4 Đóng góp của đề án

Từ mục tiêu và định hướng giải pháp đã trình bày, đề án mang lại hai nhóm đóng góp chính: đóng góp về mặt mô hình nghiên cứu và đóng góp về mặt hiện thực kỹ thuật.

1.4.1 Đóng góp về mặt nghiên cứu lý thuyết

Đóng góp đầu tiên của đề án là xây dựng cách tiếp cận mới cho bài toán vận hành MLS trong môi trường mạng ngang hàng. Thay vì xem Delivery Service như một thành phần máy chủ cố định, đề án phân tích vai trò của Delivery Service dưới dạng một tập các chức năng mà hệ thống triển khai MLS cần cung cấp, bao gồm phân phối thông điệp, hỗ trợ vật liệu khóa ban đầu, điều phối thứ tự Commit và xử lý khi trạng thái nhóm không còn đồng nhất. Cách tiếp cận này phù hợp với kiến trúc MLS hiện đại, trong đó MLS không bị ràng buộc vào duy nhất một mô hình triển khai tập trung, mà có thể được tích hợp vào nhiều kiến trúc hệ thống khác nhau, bao gồm cả môi trường P2P [2]. Từ đó, đề án đặt lại bài toán nghiên cứu theo hướng: thay thế chức năng điều phối thường do Delivery Service đảm nhiệm bằng một lớp điều phối phi tập trung ở tầng ứng dụng.

Đóng góp thứ hai là đề xuất cơ chế *Single-Writer per Epoch* cho nhóm MLS trên mạng P2P. Cơ chế này xác định một *Token Holder* duy nhất tại mỗi epoch bằng một quy tắc tất định dựa trên trạng thái nhóm, số epoch và tập thành viên hợp lệ. Nhờ vậy, trong điều kiện các nút có cùng góc nhìn về nhóm, toàn bộ hệ thống có thể tự tính ra cùng một nút chịu trách nhiệm tạo Commit mà không cần một vòng bỏ phiếu hoặc đồng thuận mạng bổ sung. Cơ chế này trực tiếp nhắm vào nguyên nhân cốt lõi gây phân nhánh trạng thái MLS trong môi trường P2P: nhiều thành viên cùng tạo Commit cạnh tranh tại cùng một epoch.

Đóng góp thứ ba là mô hình hóa vòng đời phân nhánh và phục hồi của nhóm MLS khi xảy ra *network partition*. Đề án phân biệt rõ hai lớp vấn đề: xung đột Commit trong điều kiện mạng liên thông và phân kỳ trạng thái khi mạng bị chia cắt vật lý hoặc logic. Với trường hợp thứ nhất, Single-Writer giúp giảm khả năng sinh concurrent commits. Với trường hợp thứ hai, đề án đề xuất quy trình phát hiện fork, lựa chọn nhánh tiếp tục và đưa các nút ở nhánh không được chọn gia nhập lại trạng thái MLS hợp lệ. Cách tiếp cận này thể hiện rõ định hướng thiết kế của đề án: ưu tiên tính sẵn sàng cục bộ trong môi trường P2P, nhưng vẫn bảo đảm hệ thống

có cơ chế hội tụ trở lại sau khi kết nối được khôi phục.

1.4.2 Đóng góp về mặt hiện thực kỹ thuật

Về mặt kỹ thuật, đề án hiện thực một nguyên mẫu ứng dụng liên lạc nhóm bảo mật trên mạng ngang hàng, kết hợp Go cho lớp mạng và điều phối, Rust/OpenMLS cho lớp mật mã, SQLite cho lưu trữ cục bộ và gRPC cho giao tiếp liên tiến trình. Thiết kế này giúp hệ thống tận dụng được thư viện MLS chuẩn thay vì tự xây dựng lại các thành phần mật mã phức tạp, đồng thời vẫn giữ được khả năng tùy biến ở tầng điều phối để xử lý các đặc thù của môi trường P2P như mất kết nối, thông điệp đến sai thứ tự, đồng bộ ngoại tuyến và phân mảnh mạng.

Một đóng góp kỹ thuật quan trọng khác là kiến trúc sidecar Go–Rust có ranh giới trách nhiệm rõ ràng. Go Host chịu trách nhiệm điều phối, mạng P2P và lưu trữ bền vững; Rust sidecar chịu trách nhiệm xử lý các thao tác OpenMLS thông qua gRPC. Cách tổ chức này giúp nguyên mẫu tận dụng thư viện mật mã chuẩn trong Rust, đồng thời vẫn giữ trạng thái ứng dụng và trạng thái nhóm trong lớp Go/SQLite để thuận lợi cho khôi phục sau sự cố.

Bên cạnh lõi điều phối, đề án cũng tích hợp các chức năng ứng dụng cần thiết để kiểm chứng giải pháp trong kịch bản thực tế, bao gồm phát hiện thiết bị trong mạng nội bộ, truyền thông điệp qua mạng ngang hàng, đồng bộ khi thiết bị quay trở lại sau thời gian offline, lưu trữ cục bộ lịch sử hội thoại và truyền tệp đã mã hóa. Các thành phần này cho phép đánh giá giao thức không chỉ ở mức mô hình lý thuyết, mà còn trong một nguyên mẫu hệ thống hoàn chỉnh, nơi các vấn đề của mạng P2P và trạng thái MLS xuất hiện đồng thời.

Tổng hợp lại, đóng góp chính của đề án nằm ở việc kết hợp một lõi mật mã nhóm đã được chuẩn hóa với một lớp điều phối phi tập trung được thiết kế riêng cho môi trường P2P. Cách tiếp cận này giữ nguyên các bảo đảm mật mã của MLS, đồng thời bổ sung các cơ chế cần thiết để hệ thống có thể vận hành trong điều kiện không có Delivery Service trung tâm.

1.5 Bố cục đề án

Phần còn lại của báo cáo được tổ chức như sau.

Chương 2 trình bày nền tảng lý thuyết của đề tài, bao gồm mô hình mã hóa đầu cuối, MLS, TreeKEM, Delivery Service, các vấn đề về Commit trong MLS, mạng ngang hàng và các cơ chế liên quan đến đồng bộ trong hệ phân tán.

Chương 3 mô tả chi tiết phương pháp đề xuất, bao gồm kiến trúc Go – Rust sidecar, mô hình trạng thái nhóm, cơ chế Single-Writer theo epoch, kiểm tra epoch, phát hiện phân nhánh và phục hồi sau network partition.

Chương 4 phân tích lý thuyết về tính đúng đắn và các giới hạn của giải pháp, tập trung vào các thuộc tính như nhất quán trạng thái, tính sẵn sàng khi phân mảnh mạng, phạm vi bảo toàn FS/PCS và độ phức tạp tính toán, bảng thông của các thao tác cập nhật nhóm [1].

Chương 5 trình bày đánh giá thực nghiệm của nguyên mẫu hệ thống. Các thí nghiệm tập trung vào độ trễ tạo và xử lý Commit, chi phí cập nhật thành viên, thời gian đồng bộ sau khi thiết bị offline quay trở lại, ảnh hưởng của số lượng thành viên đến hiệu năng và so sánh với các baseline phù hợp.

Cuối cùng, Chương 6 tổng kết kết quả đạt được, chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo, bao gồm mở rộng mô hình đe dọa, tăng cường chống thành viên độc hại, đánh giá trên mạng diện rộng và cải thiện cơ chế phục hồi phân nhánh.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Chương 1 đã trình bày bối cảnh, động lực và mục tiêu của đề tài: xây dựng một lớp điều phối phi tập trung nhằm hỗ trợ vận hành Messaging Layer Security (MLS) trên môi trường mạng ngang hàng. Để làm rõ cơ sở khoa học cho hướng tiếp cận này, Chương 2 tập trung trình bày các nền tảng lý thuyết cốt lõi của đề tài, bao gồm bài toán nhất quán trạng thái mật mã trong hệ phân tán, các hướng nghiên cứu liên quan, cơ chế hoạt động của MLS và TreeKEM, cùng các đặc tính của mạng ngang hàng phi tập trung.

Nội dung chương được tổ chức thành bốn phần chính. Mục 2.1 phân tích ngữ cảnh bài toán và chỉ ra vì sao việc đưa MLS lên mạng ngang hàng không chỉ là một bài toán truyền thông, mà là một bài toán nhất quán trạng thái mật mã. Mục 2.2 tổng hợp các kết quả nghiên cứu và giải pháp liên quan, từ các mô hình Sender Keys, Megolm đến MLS tập trung, DMLS và các hướng dựa trên đồng thuận phân tán. Mục 2.3 trình bày nền tảng MLS và TreeKEM, đặc biệt là khái niệm Epoch, Proposal, Commit, Delivery Service và bài toán tuần tự hóa thay đổi trạng thái. Mục 2.4 trình bày nền tảng mạng ngang hàng, go-libp2p, GossipSub và vai trò của đồng hồ logic trong việc sắp xếp thông điệp ứng dụng.

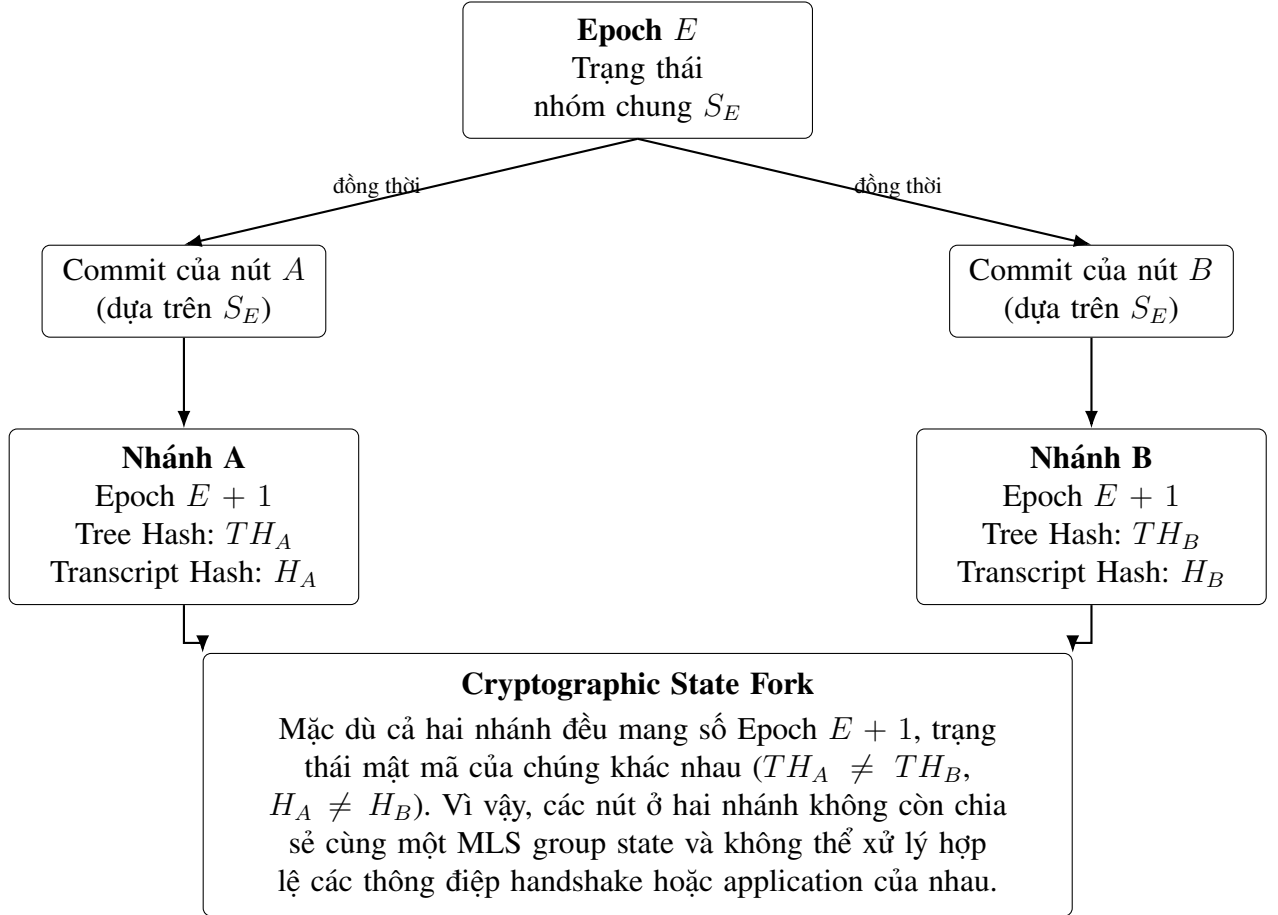
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán

Trong các hệ thống phân tán thông thường, nhất quán trạng thái là bài toán đảm bảo nhiều nút mạng cùng quan sát và cập nhật dữ liệu theo một quy tắc thống nhất. Đối với hệ thống nhắn tin mã hóa đầu-cuối nhóm, bài toán này trở nên phức tạp hơn vì trạng thái cần nhất quán không chỉ là dữ liệu ứng dụng, mà còn là trạng thái mật mã dùng để sinh khóa mã hóa, xác thực thành viên và bảo vệ lịch sử hội thoại.

MLS biểu diễn một nhóm bằng một trạng thái mật mã tiến hóa qua các kỷ nguyên, gọi là Epoch. Mỗi Epoch tương ứng với một tập thành viên hợp lệ và một tập bí mật nhóm được chia sẻ giữa các thành viên đó. Khi nhóm thay đổi thành viên hoặc khi thành viên cập nhật khóa, trạng thái nhóm phải chuyển từ Epoch hiện tại sang Epoch mới. Nếu tất cả thành viên cùng áp dụng một thay đổi theo cùng một thứ tự, nhóm tiếp tục duy trì khả năng giao tiếp. Ngược lại, nếu các thành viên áp dụng các thay đổi khác nhau hoặc áp dụng cùng các thay đổi theo thứ tự không tương thích, nhóm có thể rơi vào trạng thái phân nhánh mật mã.

Hiện tượng này được gọi là rẽ nhánh trạng thái mật mã (Cryptographic State Fork). Đây là tình huống trong đó hai tập thành viên cùng tin rằng mình đang ở Epoch hợp lệ tiếp theo của cùng một nhóm, nhưng thực tế lại nắm giữ các bí mật

(secrets), hàm băm cây (tree hash) và lịch sử giao tiếp (transcript hash) khác nhau. Khi đó, thông điệp được mã hóa trên một nhánh sẽ không thể được xử lý hợp lệ trên nhánh còn lại. Khác với xung đột dữ liệu thông thường trong hệ phân tán, xung đột trạng thái mật mã không thể được giải quyết đơn giản bằng cách “merge” hai phiên bản dữ liệu, vì mỗi nhánh đã phát sinh một chuỗi khóa và xác thực riêng. Hiện tượng rẽ nhánh trạng thái mật mã này được minh họa chi tiết trong Hình 2.1.



Hình 2.1: Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS

2.1.1 Từ mô hình tập trung sang mô hình ngang hàng

Trong các hệ thống nhắn tin hiện đại, kể cả khi nội dung tin nhắn được mã hóa đầu-cuối, máy chủ trung tâm vẫn thường giữ vai trò quan trọng trong việc định tuyến, lưu trữ khóa công khai ban đầu, phân phối thông điệp và hỗ trợ thiết bị ngoại tuyến. Với MLS, vai trò này thường được mô hình hóa thông qua Delivery Service (DS) [2]. Trong các triển khai thông thường, DS là nơi nhận thông điệp từ client, phân phối thông điệp tới các thành viên còn lại và có thể tham gia áp đặt thứ tự đối với các thông điệp thay đổi trạng thái nhóm.

Mô hình tập trung giúp đơn giản hóa bài toán tuần tự hóa. Khi nhiều thành viên cùng gửi Commit tại cùng một Epoch, DS có thể đóng vai trò như một bộ sắp xếp thứ tự, quyết định Commit nào được chấp nhận trước và Commit nào trở thành dư

thừa. Chính điểm này khiến MLS vận hành thuận lợi trong mô hình Client-Server: hệ thống có một vị trí tự nhiên để áp đặt thứ tự tuyến tính cho các thay đổi trạng thái.

Khi chuyển sang mạng ngang hàng, điểm tựa này biến mất. Các nút mạng có thể gửi thông điệp trực tiếp hoặc gián tiếp qua nhiều đường truyền khác nhau. Độ trễ truyền tin không đồng đều, thông điệp có thể đến khác thứ tự, nút mạng có thể offline đột ngột và mạng có thể bị chia cắt thành nhiều phân vùng. Trong môi trường như vậy, không còn một máy chủ trung tâm hiển nhiên để quyết định đâu là Commit “đến trước”. Đây là điểm gây kiến trúc khi đưa MLS từ mô hình triển khai tập trung thông thường sang mô hình P2P phi tập trung.

2.1.2 Mô hình hệ thống và giả định nghiên cứu

Trong phạm vi đề án, hệ thống được mô hình hóa như một mạng gồm nhiều nút ngang hàng, trong đó mỗi nút tương ứng với một thiết bị của người dùng. Các nút này có thể tham gia vào một hoặc nhiều nhóm liên lạc khác nhau. Mỗi nhóm được xác định bởi một định danh riêng, một tập thành viên hợp lệ và một trạng thái MLS được lưu cục bộ trên từng thiết bị. Thay vì phụ thuộc vào một máy chủ trung tâm luôn sẵn sàng để lưu trữ và phân phối toàn bộ thông điệp, các thiết bị trao đổi dữ liệu trực tiếp hoặc gián tiếp với nhau thông qua mạng ngang hàng.

Đặc điểm quan trọng của môi trường này là các nút không luôn quan sát được cùng một trạng thái tại cùng một thời điểm. Một thông điệp có thể đến trễ, đến sai thứ tự, bị lặp lại hoặc tạm thời không đến được do điều kiện mạng. Một thiết bị cũng có thể rời mạng, quay trở lại sau một khoảng thời gian ngoại tuyến, hoặc chỉ nhìn thấy một phần các thành viên đang hoạt động. Những hiện tượng này là bình thường trong mạng P2P, nhưng lại trở thành vấn đề nghiêm trọng khi áp dụng cho MLS, vì mỗi thay đổi trạng thái nhóm đều gắn với một epoch và một trạng thái mật mã cụ thể.

Do đó, đề án tập trung vào bài toán điều phối trạng thái MLS trong môi trường P2P không ổn định. Các tình huống chính được xem xét bao gồm độ trễ mạng không đồng đều, thiết bị ngoại tuyến, peer churn và network partition. Mục tiêu của mô hình là phản ánh các điều kiện vận hành thực tế mà một ứng dụng liên lạc ngang hàng có thể gặp phải, từ đó làm cơ sở cho việc thiết kế các cơ chế như Single-Writer, Epoch Checks và Fork Healing ở các chương sau.

Một giả định xuyên suốt của đề án là MLS vẫn giữ vai trò lõi mật mã chuẩn của hệ thống. Đề án không thay đổi thuật toán mã hóa, xác thực hay key schedule của MLS. Thay vào đó, phần đóng góp được đặt ở lớp điều phối phía trên MLS: kiểm soát thời điểm tạo Commit, kiểm tra epoch của thông điệp nhận được, phát hiện

khi trạng thái nhóm bị phân nhánh và đưa các nút hội tụ trở lại sau khi mạng được kết nối. Cách tiếp cận này giúp hệ thống tận dụng được các bảo đảm mật mã của MLS/OpenMLS, đồng thời bổ sung những cơ chế cần thiết để vận hành trong môi trường mạng ngang hàng.

2.1.3 Mâu thuẫn cốt lõi của MLS trên mạng ngang hàng

Mâu thuẫn cốt lõi có thể được phát biểu như sau: MLS yêu cầu trạng thái nhóm tiến hóa theo chuỗi Epoch tuyến tính, trong khi mạng ngang hàng bất đồng bộ không tự cung cấp một thứ tự tuyến tính toàn cục cho các thông điệp thay đổi trạng thái. Khi hai thành viên cùng tạo Commit dựa trên Epoch E , mỗi Commit đều có thể là một ứng viên hợp lệ để đưa nhóm sang Epoch $E + 1$ nếu xét riêng tại thời điểm nó được tạo. Tuy nhiên, hai Commit này thường tạo ra hai trạng thái cây khác nhau. Nếu một phần nhóm áp dụng Commit thứ nhất, trong khi phần còn lại áp dụng Commit thứ hai, toàn nhóm sẽ phân kỳ.

Trong mô hình tập trung, DS có thể xử lý xung đột bằng cách chỉ phát tán Commit thắng cuộc hoặc yêu cầu client chỉ chấp nhận Commit đầu tiên được DS xác nhận. Nhưng trong mô hình P2P, không có máy chủ trung tâm nào có quyền hòa giải việc đó. Do đó, nếu không có lớp điều phối bổ sung, MLS trên P2P có nguy cơ rơi vào trạng thái mà các nhánh khác nhau đều có vẻ hợp lệ cục bộ nhưng không thể giao tiếp liên nhánh.

Đây chính là khoảng trống mà đề án hướng tới: xây dựng một cơ chế điều phối phi tập trung để ảo hóa vai trò tuần tự hóa của DS, nhưng không đưa trở lại một máy chủ trung tâm và không sử dụng các thuật toán đồng thuận nặng nề cho mọi Commit.

2.2 Các kết quả nghiên cứu tương tự

Các nghiên cứu và hệ thống hiện có về mã hóa đầu-cuối nhóm có thể được nhìn từ góc độ phù hợp với bài toán của đề án: cách hệ thống quản lý khóa nhóm, cách xử lý thay đổi thành viên, và đặc biệt là cách duy trì thứ tự các thay đổi trạng thái khi nhóm vận hành trong môi trường phân tán. Theo hướng tiếp cận này, các giải pháp liên quan có thể chia thành ba nhóm chính: các cơ chế mã hóa nhóm trước MLS như Sender Keys và Megolm, MLS trong mô hình có Delivery Service, và các nỗ lực đưa MLS sang môi trường phi tập trung.

2.2.1 Các cơ chế mã hóa nhóm trước MLS

Double Ratchet là nền tảng quan trọng của nhiều hệ thống nhắn tin bảo mật hiện đại, đặc biệt trong liên lạc hai bên. Khi mở rộng sang nhóm, một hướng phổ biến là sử dụng Sender Keys, trong đó mỗi thành viên tạo một khóa gửi tin riêng

và phân phối khóa này cho các thành viên còn lại thông qua các kênh bảo mật hai bên [3], [4]. Sau khi khóa gửi tin đã được phân phối, người gửi có thể mã hóa một thông điệp một lần, còn hạ tầng bên dưới chịu trách nhiệm phân phối bản mã tới các thành viên trong nhóm. Cách tiếp cận này có hiệu quả thực tiễn cao, nhưng chi phí cập nhật khóa và phục hồi sau thỏa hiệp vẫn tăng đáng kể khi nhóm lớn hoặc khi membership thay đổi thường xuyên.

Megolm, được sử dụng trong hệ sinh thái Matrix, cũng đi theo hướng tối ưu hiệu năng gửi tin nhóm. Cơ chế này sử dụng *group ratchet*, cho phép một outbound session phục vụ nhiều thông điệp liên tiếp trong cùng một phòng trò chuyện [5]. Thiết kế này giúp giảm chi phí mã hóa và truyền thông, nhưng đánh đổi ở khả năng phục hồi sau khi session state bị compromise. Khi ratchet state của một session bị lộ, phạm vi thông điệp bị ảnh hưởng có thể kéo dài trong chính session đó cho đến khi hệ thống thiết lập session mới và phân phối lại khóa.

Nhìn chung, Sender Keys và Megolm là các thiết kế có giá trị thực tiễn lớn, nhưng chúng không trực tiếp giải quyết bài toán mà đề án tập trung vào: duy trì một trạng thái khóa nhóm nhất quán khi nhiều thành viên có thể cùng thực hiện thay đổi nhóm trong môi trường P2P bất đồng bộ. Đây là lý do MLS trở thành nền tảng phù hợp hơn cho đề tài, vì MLS cung cấp một mô hình quản lý khóa nhóm có cấu trúc thông qua TreeKEM, Proposal, Commit và Epoch.

2.2.2 MLS với Delivery Service

MLS được chuẩn hóa nhằm cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ, hiệu quả và có các thuộc tính bảo mật phù hợp cho truyền thông nhóm quy mô lớn [1]. Thông qua cấu trúc TreeKEM, MLS cho phép cập nhật trạng thái khóa nhóm theo đường đi trong cây thay vì phân phối lại khóa theo cách tuyến tính tới toàn bộ thành viên. Đây là điểm khác biệt quan trọng so với các mô hình dựa trên việc mỗi sender tự quản lý khóa gửi tin riêng.

Tuy nhiên, MLS không tự định nghĩa toàn bộ hệ thống nhắn tin. Trong kiến trúc MLS, Delivery Service thường đảm nhiệm việc lưu trữ KeyPackage, phân phối MLS messages và hỗ trợ quá trình truyền thông giữa các client [2]. Với mô hình Delivery Service nhất quán mạnh, hệ thống còn có thể dựa vào DS để áp đặt một thứ tự tuyến tính cho các Commit. Khi hai thành viên cùng tạo Commit tại cùng một epoch, DS có thể đóng vai trò phá hòa, giúp toàn nhóm chỉ đi theo một chuỗi trạng thái thống nhất.

Chính vai trò này làm cho MLS vận hành thuận lợi trong mô hình Client-Server, nhưng cũng tạo ra giới hạn khi chuyển sang P2P. Nếu loại bỏ DS trung tâm, hệ thống không chỉ mất một thành phần truyền tin, mà còn mất điểm điều phối thứ tự

cho các Commit. Trong khi đó, thứ tự Commit lại là yếu tố sống còn đối với MLS, vì mỗi Commit tạo ra một epoch mới và một trạng thái mật mã mới. Do đó, đưa MLS lên mạng ngang hàng đòi hỏi một cơ chế thay thế vai trò ordering của DS ở tầng ứng dụng.

2.2.3 Các hướng phi tập trung hóa MLS

Một hướng tiếp cận trực tiếp là bổ sung một lớp đồng thuận phân tán để thống nhất thứ tự Commit. Các cơ chế như BFT, blockchain hoặc quorum-based consensus có thể tạo ra một log tuyến tính cho các thay đổi trạng thái nhóm. Nếu mọi Commit đều được ghi vào cùng một log và được áp dụng theo cùng một thứ tự, hệ thống có thể tránh phân nhánh trạng thái. Tuy nhiên, cách tiếp cận này thường yêu cầu nhiều vòng trao đổi mạng, phụ thuộc vào quorum và tạo ra chi phí triển khai lớn. Với mục tiêu xây dựng một hệ thống P2P nhẹ, Local-First và có thể vận hành trong mạng nội bộ hoặc mạng không ổn định, việc duy trì đồng thuận nặng nề cho mọi Commit không phải lựa chọn phù hợp.

Một hướng khác là Decentralized MLS (DMLS) của Kohbrok. Khác với hướng dùng đồng thuận bên ngoài, DMLS mở rộng trực tiếp bản thân MLS để hỗ trợ xử lý Commit đến ngoài thứ tự trong môi trường không có thực thể trung tâm áp đặt ordering [7]. Hướng này cho phép client giữ lại một phần key material để xử lý các nhánh Commit khác nhau với mức ảnh hưởng tới Forward Secrecy thấp hơn so với việc giữ khóa cũ một cách thô sơ [6]. Tuy nhiên, đặc trưng này cũng là điểm đánh đổi chính của DMLS: để xử lý out-of-order commits, client vẫn phải duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, làm tăng độ phức tạp triển khai và làm giảm phần nào mức bảo vệ FS so với mô hình xóa khóa nghiêm ngặt của MLS gốc. Ngoài ra, DMLS hiện vẫn là một Internet-Draft đã hết hạn, chưa trở thành một chuẩn IETF hoàn chỉnh [7].

Từ các phân tích trên, đề án lựa chọn một hướng tiếp cận khác: giữ nguyên lõi MLS và giải quyết bài toán ordering ở tầng điều phối phía trên. Cụ thể, đề án đề xuất lớp điều phối gồm Single-Writer để hạn chế concurrent commits trong điều kiện mạng liên thông, Epoch Checks để ngăn client áp dụng thông điệp sai trạng thái, Fork Healing để phục hồi khi network partition tạo ra nhiều nhánh, và HLC Message Ordering để sắp xếp thông điệp ứng dụng trong cùng một epoch. Cách tiếp cận này không biến hệ thống thành một blockchain hay một hệ thống BFT đầy đủ, đồng thời cũng không yêu cầu sửa đổi key schedule của MLS. Đây là khoảng trống chính mà đề án khai thác: một lớp điều phối đủ nhẹ cho P2P, nhưng vẫn đủ chặt để bảo vệ chuỗi trạng thái MLS.

2.3 Nền tảng MLS và TreeKEM

2.3.1 Các khái niệm cốt lõi: Client, Group, Member và Epoch

Trong MLS [1], client là thực thể nắm giữ khóa mật mã và tham gia vào quá trình thiết lập trạng thái nhóm. Group là một tập logic gồm các client cùng chia sẻ một trạng thái mật mã tại một thời điểm. Member là client đang nằm trong trạng thái chia sẻ của group và có quyền truy cập các bí mật (secrets) của group. Epoch là một phiên bản cụ thể của trạng thái nhóm, trong đó tập thành viên hợp lệ cùng nắm giữ trạng thái mật mã tương ứng.

Điểm quan trọng là trạng thái group trong MLS không phải một tập dữ liệu rời rạc có thể hợp nhất tùy ý. Nó là một chuỗi Epoch tuyến tính, trong đó mỗi Epoch phụ thuộc vào Epoch trước đó. Mỗi thay đổi thành viên hoặc cập nhật khóa làm phát sinh một Epoch mới. Vì vậy, tính đúng đắn của MLS phụ thuộc mạnh vào việc tất cả thành viên cùng nhìn nhận một chuỗi Epoch tương thích.

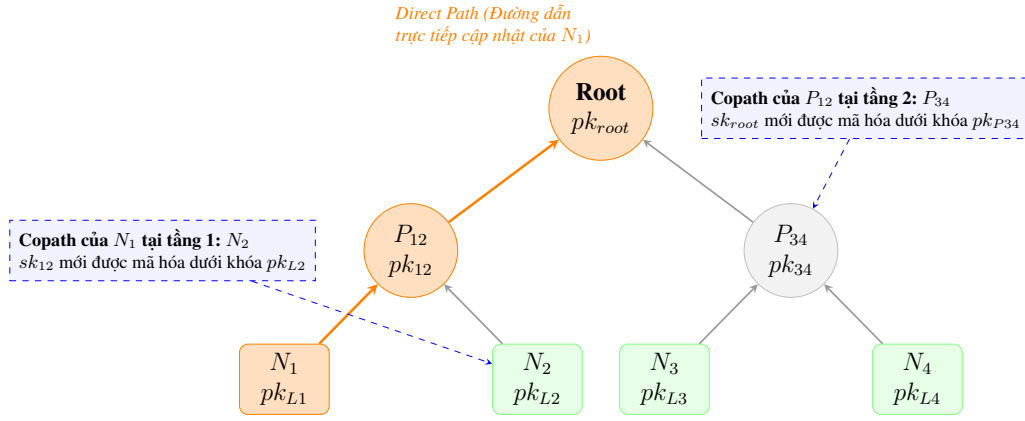
2.3.2 Ratchet Tree và TreeKEM

TreeKEM là cơ chế trung tâm giúp MLS đạt hiệu quả khi cập nhật khóa nhóm. Thay vì mọi thành viên phải trao đổi khóa với mọi thành viên còn lại, MLS tổ chức các thành viên thành lá của một cây nhị phân gọi là Ratchet Tree. Các nút trong cây chứa vật liệu khóa công khai, còn các thành viên biết một số bí mật tương ứng với đường đi từ lá của mình lên gốc.

Khi một thành viên cập nhật khóa, thành viên đó tạo một đường dẫn cập nhật (update path) mới từ lá của mình lên gốc cây. Các bí mật mới trên đường đi này được mã hóa cho các nhánh đồng cấp tương ứng (copath), sao cho các thành viên còn lại có thể giải mã và cập nhật trạng thái. Nhờ cấu trúc cây, số lượng thao tác mã hóa và kích thước thông tin cập nhật tăng theo chiều cao cây, tức xấp xỉ logarit theo số thành viên trong nhóm ($O(\log N)$).

Sơ đồ cấu trúc cây Ratchet Tree và quá trình cập nhật đường dẫn (Path Update) được mô tả chi tiết trong Hình 2.2.

Cần lưu ý rằng bí mật tại gốc cây (sk_{root}) không được sử dụng trực tiếp như khóa mã hóa tin nhắn ứng dụng. Sau mỗi Commit, MLS đưa các bí mật mới qua lịch trình sinh khóa (key schedule) để tạo ra các bí mật của Epoch mới, bao gồm các khóa dùng cho xác thực, mã hóa thông điệp, xuất dữ liệu khóa (exporter) và các cơ chế liên quan. Vì vậy, TreeKEM nên được hiểu là cơ chế cập nhật trạng thái khóa nhóm, còn việc sinh khóa cụ thể cho thông điệp ứng dụng (application messages) được thực hiện bởi key schedule của MLS.



Hình 2.2: Cấu trúc cây nhị phân Ratchet Tree và tiến trình cập nhật đường dẫn (Path Update) của thành viên N_1

2.3.3 Proposal, Commit và tiến trình chuyển Epoch

MLS phân biệt rõ Proposal và Commit. Proposal là thông điệp đề xuất một thay đổi đối với nhóm, ví dụ thêm thành viên (Add), loại bỏ thành viên (Remove), cập nhật khóa (Update) hoặc thay đổi phần mở rộng của bối cảnh nhóm (group context). Proposal tự nó chưa làm thay đổi Epoch. Commit là thông điệp thực sự áp dụng một tập Proposal và đưa nhóm từ Epoch hiện tại sang Epoch mới.

Khi tạo Commit, thành viên thực hiện Commit lựa chọn một tập Proposal hợp lệ, áp dụng chúng lên Ratchet Tree và group context, tạo đường dẫn cập nhật (update path) nếu cần, sau đó sinh ra trạng thái mật mã mới. Các thành viên nhận Commit sẽ kiểm tra chữ ký, kiểm tra điều kiện hợp lệ của danh sách Proposal, áp dụng thay đổi theo đúng quy tắc, tính toán trạng thái mới và xác minh thẻ xác nhận (confirmation tag). Confirmation tag đóng vai trò xác nhận rằng các thành viên đã đi tới cùng một trạng thái nhóm sau khi xử lý Commit.

Từ góc nhìn hệ phân tán, Commit là thao tác ghi vào trạng thái mật mã nhóm. Vì mỗi Commit tạo ra một Epoch mới, việc có nhiều Commit cạnh tranh tại cùng một Epoch chính là nguồn gốc của phân nhánh trạng thái. Do đó, lớp điều phối của đồ án tập trung kiểm soát quyền tạo Commit, trong khi vẫn cho phép nhiều thành viên gửi Proposal.

2.3.4 Tree hash, transcript hash và epoch authenticator

Để phát hiện sự đồng bộ hoặc phân kỳ giữa các thành viên, MLS sử dụng nhiều giá trị ràng buộc trạng thái. Tree hash phản ánh cấu trúc Ratchet Tree hiện tại. Transcript hash ràng buộc lịch sử các thông điệp bắt tay (handshake) đã được xác nhận. Epoch authenticator là giá trị có thể được dùng để xác nhận rằng các thành viên đang chia sẻ cùng trạng thái Epoch.

Các giá trị này mang ý nghĩa thiết yếu đối với đồ án. Trong môi trường P2P, hai nút có thể cùng báo rằng chúng thuộc cùng một định danh nhóm (group_id) và cùng một số Epoch, nhưng thực chất tree hash hoặc transcript hash khác nhau. Khi đó, hai nút không thật sự ở cùng trạng thái mật mã. Vì vậy, cơ chế Fork Healing ở Chương 3 cần dựa trên các định danh trạng thái như epoch, tree hash, commit hash hoặc epoch authenticator để phát hiện phân nhánh.

2.3.5 Delivery Service và bài toán tuần tự hóa Commit

Delivery Service trong kiến trúc MLS [2] có thể được hiểu là lớp dịch vụ cung cấp hai chức năng chính: lưu trữ và cung cấp vật liệu khóa ban đầu, đồng thời phân phối các thông điệp MLS giữa các client. Trong các triển khai Client-Server thông thường, DS còn thường được dùng như một điểm tuần tự hóa tự nhiên cho các thông điệp thay đổi trạng thái nhóm.

Về mặt tiêu chuẩn, MLS không bắt buộc DS phải là một máy chủ trung tâm duy nhất. DS có thể được hiện thực bởi một hạ tầng tập trung, một hệ thống liên miền hoặc thậm chí bởi chính mạng ngang hàng nếu các client truyền các thông điệp MLS qua P2P. Tuy nhiên, khi không có DS tập trung, các client phải tự cung cấp những thuộc tính mà DS thường đảm nhiệm, đặc biệt là phân phối thông điệp, xử lý Commit đến trễ, phát hiện Commit cạnh tranh và hòa giải trạng thái.

Có thể nói, trong triển khai MLS thông thường, DS là “người gác cổng thời gian” của trạng thái nhóm: nó giúp tất cả thành viên nhìn thấy các thay đổi theo một thứ tự đủ nhất quán. Khi loại bỏ DS khỏi kiến trúc, hệ thống không chỉ mất một máy chủ truyền tin, mà còn mất điểm sắp xếp thứ tự trung tâm. Đây là lý do lớp điều phối tập trung trở thành thành phần bắt buộc trong thiết kế của đồ án.

2.3.6 Application messages, delayed messages và giới hạn Forward Secrecy

MLS phân biệt handshake messages và application messages. Handshake messages như Proposal và Commit làm thay đổi hoặc chuẩn bị thay đổi trạng thái nhóm. Application messages là thông điệp ứng dụng được mã hóa bằng khóa sinh ra từ trạng thái Epoch hiện tại.

Trong mạng thực tế, application messages có thể đến muộn hoặc đến sai thứ tự. MLS có cơ chế xử lý ở mức bộ khóa của người gửi (sender ratchet) và bộ đếm thế hệ (generation counter) để hỗ trợ một mức độ nhất định đối với thông điệp bị trễ hoặc bị sắp xếp lại. Tuy nhiên, để bảo vệ Forward Secrecy, client không nên giữ khóa cũ vô thời hạn. Điều này tạo ra một đánh đổi quan trọng: nếu xóa khóa cũ quá sớm, các thông điệp đến muộn có thể không giải mã được; nếu giữ khóa cũ quá lâu, phạm vi thiệt hại khi thiết bị bị thỏa hiệp sẽ tăng.

Đánh đổi này là nền tảng cho cơ chế Epoch Checks và bộ đệm thông điệp trong đồ án. Khi một nút nhận thông điệp thuộc Epoch cũ, Epoch hiện tại hoặc Epoch tương lai, lớp điều phối phải quyết định loại bỏ, xử lý, đệm tạm thời hay kích hoạt đồng bộ trạng thái. Chính sách này cần cân bằng giữa tính sẵn sàng của hệ thống, khả năng đồng bộ ngoại tuyến và yêu cầu bảo mật xuôi.

2.3.7 External Commit và cơ chế gia nhập lại nhóm

MLS hỗ trợ cơ chế gia nhập từ bên ngoài (external join) thông qua External Commit, cho phép một client bên ngoài nhóm tạo Commit để tự thêm mình vào nhóm nếu có thông tin nhóm (GroupInfo) hợp lệ và thỏa mãn các chính sách của ứng dụng. Đây là cơ chế quan trọng đối với đồ án vì trong quá trình Fork Healing, các nút ở nhánh thua có thể cần hủy trạng thái cũ và gia nhập lại nhánh thắng.

Trong ngữ cảnh của đề tài, External Join không được dùng như một thao tác tùy tiện. Nó là bước phục hồi có kiểm soát sau khi hệ thống xác định rằng trạng thái cục bộ của một nút không còn thuộc nhánh được chọn. Nút đó cần dừng sử dụng khóa cũ cho thông điệp mới, lấy trạng thái công khai cần thiết của nhánh thắng, thực hiện quy trình gia nhập lại và chỉ tiếp tục gửi thông điệp sau khi có trạng thái MLS hợp lệ mới.

2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán

2.4.1 Đặc tính của mạng ngang hàng

Mạng ngang hàng khác với mô hình Client-Server ở chỗ không có một máy chủ trung tâm duy nhất điều phối toàn bộ luồng truyền thông. Các nút có thể tự phát hiện nhau, thiết lập kết nối trực tiếp hoặc gián tiếp, và trao đổi dữ liệu qua nhiều đường truyền. Đặc tính này giúp hệ thống giảm phụ thuộc vào hạ tầng trung tâm, nhưng đồng thời làm tăng độ phức tạp của điều phối.

Trong mạng P2P, các vấn đề phổ biến gồm địa chỉ IP thay đổi, NAT, tường lửa, thiết bị ngoại tuyến (offline), sự biến động nút mạng (peer churn), độ trễ không đồng đều, trùng lặp thông điệp (message duplication) và phân mảnh mạng (network partition). Với ứng dụng thông thường, các vấn đề này có thể chỉ gây trễ hoặc mất dữ liệu tạm thời. Với MLS, chúng có thể làm phát sinh lỗi nghiêm trọng hơn: một số nút chuyển Epoch trước, một số nút vẫn ở Epoch cũ, hoặc hai phân vùng mạng cùng tạo Commit khác nhau.

Do đó, lớp mạng P2P chỉ giải quyết bài toán truyền thông. Nó không tự giải quyết bài toán nhất quán trạng thái mật mã. Bất kỳ hệ thống MLS-over-P2P nào cũng cần một lớp điều phối phía trên để quyết định khi nào được tạo Commit, Commit nào được chấp nhận, thông điệp nào cần đệm và trạng thái nào là trạng

thái hợp lệ của nhóm.

2.4.2 go-libp2p và các thành phần truyền thông

Thư viện go-libp2p [10] là một thư viện mạng ngang hàng mô-đun hóa, cung cấp các thành phần quan trọng để xây dựng ứng dụng P2P. Mỗi nút trong libp2p có một mã định danh peer (PeerID) được dẫn xuất từ khóa công khai, nhờ đó danh tính mạng không phụ thuộc hoàn toàn vào địa chỉ IP. Các nút có thể được định vị thông qua multiaddr, cho phép mô tả địa chỉ kết nối qua nhiều giao thức và tầng mạng khác nhau.

Để thiết lập kết nối, libp2p hỗ trợ nhiều giao thức truyền vận (transport) như TCP, QUIC hoặc WebRTC tùy môi trường triển khai. Sau khi kết nối, các peer có thể nâng cấp kênh truyền lên kênh bảo mật bằng các giao thức như Noise hoặc TLS. Libp2p cũng hỗ trợ cơ chế đa luồng (stream multiplexing) để nhiều luồng ứng dụng chạy trên cùng một kết nối, giúp hệ thống có thể vừa gửi thông điệp chat, vừa đồng bộ trạng thái hoặc truyền tệp.

Về phát hiện peer, libp2p cung cấp nhiều cơ chế như mDNS cho mạng cục bộ LAN, bootstrap peer, rendezvous hoặc Kademlia DHT cho môi trường rộng hơn. Với ứng dụng của đề án, mDNS phù hợp cho kịch bản mạng nội bộ, còn DHT hoặc bootstrap có thể hỗ trợ các kịch bản WAN/VPN.

2.4.3 GossipSub và giới hạn về thứ tự thông điệp

GossipSub [10] là cơ chế publish/subscribe phổ biến trong libp2p, cho phép các nút tham gia vào một chủ đề (topic) và phát tán thông điệp qua mạng lưới liên kết (mesh). Khi một peer gửi thông điệp vào topic, thông điệp được lan truyền qua các peer lân cận và tiếp tục được truyền phát (gossip) tới các peer khác. Cơ chế này phù hợp để phát tán Proposal, Commit, nhịp đập mạng (heartbeat) hoặc thông báo trạng thái nhóm trong môi trường P2P.

Tuy nhiên, GossipSub không cung cấp các bảo đảm mà MLS cần cho việc tuân tự hóa Commit. Nó không bảo đảm mọi peer nhận thông điệp theo cùng một thứ tự, không bảo đảm phân phối đúng một lần (exactly-once delivery) và không tự quyết định thông điệp nào là Commit thắng cuộc khi có xung đột. GossipSub là lớp truyền bá thông điệp, không phải lớp đồng thuận trạng thái.

Giới hạn này là lý do đề án không thể chỉ “đưa MLS lên GossipSub” là đủ. Nếu chỉ phát tán Commit qua GossipSub mà không có cơ chế điều phối, hệ thống vẫn có nguy cơ concurrent commits và cryptographic state fork. Vì vậy, các cơ chế Single-Writer, Epoch Checks và Fork Healing được đặt phía trên GossipSub như một lớp kiểm soát trạng thái.

2.4.4 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng

Trong MLS, Epoch chỉ thay đổi khi có thao tác thay đổi trạng thái nhóm, chẳng hạn Add, Remove, Update hoặc Commit. Trong cùng một Epoch, nhiều thành viên có thể gửi application messages song song. Các thông điệp này không nhất thiết cần một thứ tự mật mã toàn cục giống Commit, nhưng ứng dụng chat vẫn cần một thứ tự hiển thị ổn định và hợp lý giữa các thiết bị.

Đồng hồ vật lý không đủ tin cậy trong mạng P2P vì các thiết bị có thể lệch giờ, không có NTP hoặc bị cấu hình sai. Vì vậy, hệ phân tán thường sử dụng các loại đồng hồ logic để biểu diễn quan hệ thứ tự giữa sự kiện. Lamport Clock cung cấp thứ tự logic đơn giản; Vector Clock biểu diễn quan hệ nhân quả chi tiết hơn nhưng có chi phí tăng theo số nút; Hybrid Logical Clock (HLC) [9] kết hợp thời gian vật lý và bộ đếm logic để tạo timestamp vừa gần với thời gian thực, vừa ổn định hơn khi có lệch giờ.

Trong đồ án, HLC được sử dụng cho thứ tự hiển thị application messages, không dùng để quyết định tính hợp lệ của Commit. Nói cách khác, HLC giúp người dùng trên các thiết bị khác nhau nhìn thấy dòng chat theo thứ tự nhất quán hơn, nhưng không thay thế cơ chế MLS Epoch, không thay thế chữ ký, không thay thế confirmation tag và không giải quyết concurrent commits. Sự phân tách này đóng vai trò then chốt: Commit cần cơ chế điều phối trạng thái; application messages cần cơ chế sắp xếp hiển thị.

2.4.5 Khoảng trống cần giải quyết

Từ các nền tảng trên, có thể thấy bài toán của đề tài nằm ở giao điểm giữa mật mã học nhóm và hệ phân tán. MLS cung cấp lõi mật mã mạnh nhưng giả định hệ thống triển khai phải giải quyết phân phối và thứ tự thông điệp. go-libp2p cung cấp hạ tầng truyền thông ngang hàng nhưng không cung cấp thứ tự tuyến tính cho Commit. GossipSub giúp phát tán thông điệp nhưng không phải cơ chế đồng thuận. HLC giúp sắp xếp application messages nhưng không bảo vệ trạng thái mật mã.

Khoảng trống nghiên cứu của đồ án chính là thiết kế một lớp điều phối phi tập trung nằm giữa hai thế giới này. Lớp điều phối phải đủ nhẹ để phù hợp với P2P, đủ chặt để bảo vệ chuỗi Epoch tuyến tính của MLS, và đủ thực dụng để phục hồi khi mạng bị chia cắt. Bốn cơ chế được đề xuất ở Chương 3 — Single-Writer, Epoch Checks, Fork Healing và HLC Message Ordering — được xây dựng trực tiếp từ khoảng trống này.

Chương 2 đã trình bày nền tảng lý thuyết cho bài toán vận hành MLS trên mạng ngang hàng. Trước hết, chương làm rõ rằng thách thức chính của đề tài không chỉ là

truyền thông điệp qua P2P, mà là duy trì tính nhất quán trạng thái mật mã của một nhóm MLS khi không còn máy chủ trung tâm đóng vai trò tuần tự hóa. Tiếp theo, chương phân tích các hướng tiếp cận hiện có như Sender Keys, Megolm, MLS tập trung, đồng thuận phân tán và DMLS, qua đó chỉ ra khoảng trống giữa hiệu năng, bảo mật và khả năng vận hành phi tập trung.

Chương cũng trình bày các khái niệm cốt lõi của MLS và TreeKEM, bao gồm Group, Epoch, Proposal, Commit, Delivery Service, tree hash, transcript hash, application messages và External Commit. Những khái niệm này là cơ sở để hiểu vì sao concurrent commits có thể gây rẽ nhánh trạng thái mật mã và vì sao việc phục hồi sau phân mảnh mạng cần được xử lý cẩn trọng. Cuối cùng, chương phân tích nền tảng go-libp2p, GossipSub và HLC, đồng thời nhấn mạnh rằng lớp mạng ngang hàng chỉ cung cấp khả năng truyền thông, không tự bảo đảm thứ tự tuyến tính hay đồng thuận trạng thái.

Các phân tích trên tạo tiền đề trực tiếp cho Chương 3, nơi đề án trình bày chi tiết giao thức điều phối phi tập trung gồm bốn cơ chế: Single-Writer để giảm concurrent commits, Epoch Checks để bảo vệ tính đơn điệu của trạng thái cục bộ, Fork Healing để phục hồi khi mạng bị phân mảnh, và HLC Message Ordering để sắp xếp thông điệp ứng dụng trong cùng một Epoch.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Chương 2 đã trình bày nền tảng lý thuyết của MLS, TreeKEM, mạng ngang hàng và bài toán rẽ nhánh trạng thái mật mã khi nhiều Commit cạnh tranh xuất hiện trong cùng một Epoch. Từ các phân tích đó, Chương 3 đề xuất một giao thức điều phối phi tập trung đặt bên ngoài lõi mật mã MLS. Mục tiêu của giao thức là cho phép MLS vận hành trên mạng ngang hàng mà không cần một Delivery Service trung tâm đóng vai trò bộ tuần tự hóa, đồng thời vẫn giữ nguyên logic mật mã chuẩn của MLS.

Nội dung chương được tổ chức thành sáu phần chính. Mục 3.1 mô tả tổng quan giải pháp, kiến trúc hai lớp Go-Rust và luồng hoạt động tổng thể của hệ thống. Mục 3.2 chi tiết hóa giao thức người ghi duy nhất (Single-Writer Protocol) theo Epoch nhằm giảm thiểu xung đột Commit. Mục 3.3 trình bày cơ chế kiểm tra nhất quán Epoch (Epoch Consistency Checks) để bảo vệ trạng thái cục bộ của các nút. Mục 3.4 đề xuất quy trình phát hiện và hàn gắn phân nhánh mật mã (Fork Healing) khi mạng bị chia cắt và kết nối lại. Mục 3.5 mô tả cơ chế sắp xếp hiển thị tin nhắn ứng dụng bằng đồng hồ logic lai HLC. Mục 3.6 trình bày mô hình quản lý trạng thái Go-Rust sidecar và cấu trúc lưu trữ dữ liệu cục bộ bằng SQLite.

3.1 Tổng quan giải pháp

3.1.1 Bài toán cần giải quyết

Trong mô hình MLS thông thường, Delivery Service đóng vai trò quan trọng trong việc phân phối thông điệp và hỗ trợ sắp xếp thứ tự các Commit. Khi hai thành viên cùng tạo Commit tại cùng một Epoch, một DS nhất quán mạnh có thể quyết định Commit nào được chấp nhận trước, từ đó giúp toàn nhóm duy trì cùng một chuỗi Epoch tuyến tính. Khi loại bỏ DS trong môi trường P2P, hệ thống mất đi điểm tuần tự hóa trung tâm này.

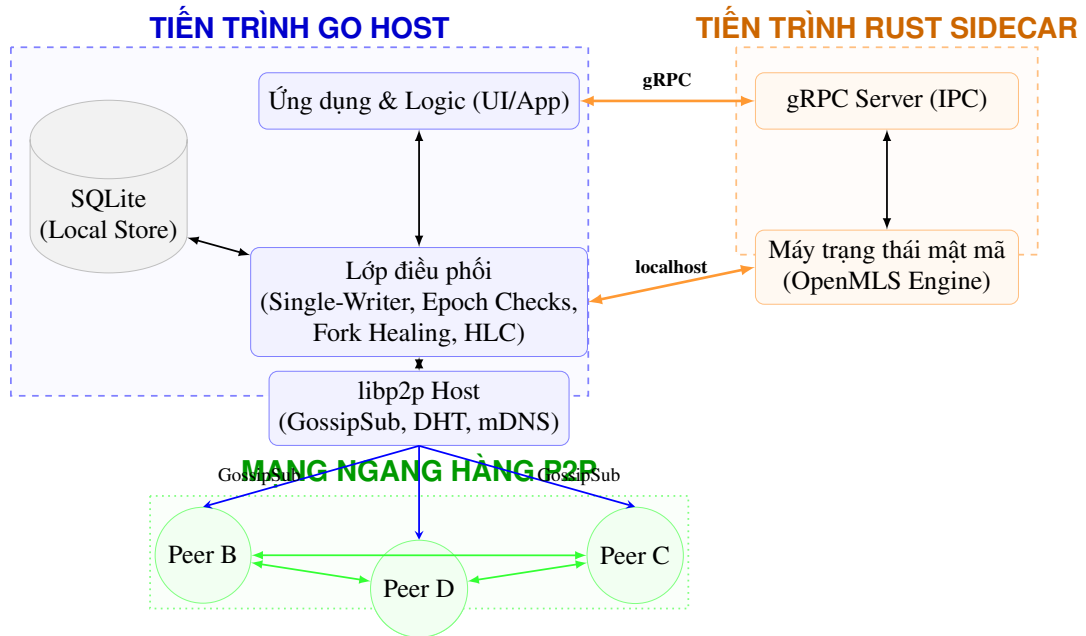
Giả sử nhóm đang ở Epoch E . Thành viên A tạo Commit C_A để chuyển nhóm sang trạng thái $S_A(E+1)$, trong khi thành viên B đồng thời tạo Commit C_B để chuyển nhóm sang trạng thái $S_B(E+1)$. Nếu một phần các nút nhận và áp dụng C_A trước, còn phần còn lại nhận và áp dụng C_B trước, nhóm sẽ bị tách thành hai nhánh trạng thái khác nhau. Hai nhánh này có thể cùng mang số Epoch $E+1$, nhưng khác tree hash, transcript hash, epoch secret và khóa ứng dụng. Khi đó, các thành viên ở hai nhánh không còn chia sẻ cùng trạng thái mật mã và không thể xử lý thông điệp của nhau một cách hợp lệ.

Do đó, phương pháp đề xuất cần giải quyết bốn yêu cầu chính: (i) giảm tối đa

khả năng xuất hiện concurrent commits trong điều kiện mạng liên thông; (ii) mỗi nút phải có khả năng kiểm tra thông điệp đến để tránh áp dụng sai Epoch hoặc áp dụng Commit dựa trên trạng thái không tương thích; (iii) khi phân mảnh mạng (network partition) xảy ra khiến nhiều nhánh xuất hiện, hệ thống phải tự động phát hiện và có quy trình hội tụ trở lại một nhánh hợp lệ duy nhất; (iv) các thông điệp ứng dụng thông thường trong cùng một Epoch vẫn cần được sắp xếp hiển thị ổn định trên các thiết bị mà không phụ thuộc vào đồng hồ vật lý đồng bộ tuyệt đối.

3.1.2 Kiến trúc tổng thể

Giải pháp đề xuất được tổ chức theo kiến trúc hai lớp, gồm Go Host và Rust Sidecar. Go Host là tiến trình chính của ứng dụng, chịu trách nhiệm giao diện người dùng, lưu trữ cục bộ, mạng ngang hàng và logic điều phối. Rust Sidecar là tiến trình mật mã chuyên biệt, chịu trách nhiệm thực thi các thao tác MLS bằng thư viện OpenMLS. Sơ đồ kiến trúc triển khai chi tiết được thể hiện trong Hình 3.1.



Hình 3.1: Kiến trúc triển khai chi tiết của Go Host và Rust Sidecar

Trong kiến trúc này, Go Host là nguồn quyết định về thứ tự điều phối ở tầng ứng dụng. Rust Sidecar không tự quyết định Commit nào là hợp lệ về mặt phân tán; nó chỉ kiểm tra và xử lý thông điệp theo logic mật mã của MLS. Ngược lại, Go Host không tự triển khai mật mã MLS; nó chỉ gọi Rust Sidecar khi cần tạo Proposal, Commit, Welcome, External Commit, hoặc mã hóa/giải mã application message.

Một điểm quan trọng trong thiết kế là Rust Sidecar được giữ phi trạng thái từ góc nhìn của Go Host. Go Host lưu trạng thái nhóm bền vững trong SQLite, lấy snapshot `GroupState` hiện tại khi cần xử lý, gửi snapshot đó sang Rust qua

gRPC, sau đó lưu lại `GroupState` mới do Rust trả về. Cách tổ chức này làm rõ nguồn sự thật của hệ thống và giúp quá trình khôi phục sau khi ứng dụng hoặc sidecar khởi động lại trở nên trực tiếp hơn.

3.1.3 Các thành phần chính của giao thức

Giao thức điều phối phi tập trung gồm bốn cơ chế chính bao gồm: (i) cơ chế người ghi duy nhất (Single-Writer Protocol) để bầu chọn Token Holder động nhằm kiểm soát quyền tạo Commit; (ii) kiểm tra nhất quán Epoch (Epoch Consistency Checks) nhằm đảm bảo tính đơn điệu của trạng thái cục bộ tại mỗi nút; (iii) cơ chế hàn gắn phân nhánh nhóm (Fork Healing) để phát hiện phân nhánh trạng thái mật mã khi xảy ra phân mảnh mạng và tự động phục hồi khi các phân vùng kết nối lại; và (iv) sắp xếp tin nhắn ứng dụng (HLC Message Ordering) bằng đồng hồ logic lai HLC để đảm bảo thứ tự hiển thị ổn định trên giao diện người dùng. Các thành phần này phối hợp chặt chẽ với nhau để tạo thành một lớp điều phối phi tập trung vững chắc bên ngoài lõi mật mã MLS.

3.1.4 Luồng hoạt động tổng thể

Khi người dùng gửi một tin nhắn ứng dụng thông thường, Go Host lấy trạng thái nhóm hiện tại, tạo timestamp HLC, gọi Rust Sidecar để mã hóa nội dung theo MLS, sau đó đóng gói bản mã vào envelope và phát tán qua GossipSub. Vì tin nhắn ứng dụng không thay đổi trạng thái thành viên hay cây MLS, nó không cần cơ chế Single-Writer.

Khi người dùng thực hiện thao tác thay đổi nhóm (thêm, xóa thành viên hoặc cập nhật khóa), Go Host tạo Proposal và phát tán qua mạng. Tất cả thành viên đều có thể gửi Proposal. Tuy nhiên, chỉ Token Holder của Epoch hiện tại được phép thu thập các Proposal hợp lệ để tạo Commit. Sau khi Commit được phát tán và áp dụng thành công, nhóm chuyển sang Epoch mới, đồng thời Token Holder của Epoch tiếp theo được tính lại. Tiến trình Commit trong hệ thống được thực hiện qua các bước tuần tự logic bao gồm: (i) một hoặc nhiều thành viên tạo Proposal tại Epoch E ; (ii) các bản tin Proposal được phát tán qua mạng ngang hàng P2P; (iii) Token Holder của Epoch E thu thập các Proposal hợp lệ từ hàng đợi cục bộ; (iv) Token Holder gọi Rust Sidecar để tạo Commit; (v) Commit được đóng gói trong envelope của lớp điều phối và phát tán qua GossipSub; (vi) các nút nhận Commit tiến hành kiểm tra Epoch, chữ ký, group state và confirmation tag; (vii) nếu Commit hợp lệ, các nút cập nhật cây mật mã cục bộ và chuyển sang Epoch $E + 1$; và (viii) Token Holder của Epoch mới được tính toán lại một cách tự động bằng hàm tất định cục bộ trên mỗi thiết bị.

Luồng trên làm rõ vai trò của lớp điều phối. MLS vẫn chịu trách nhiệm kiểm

tra hợp lệ mật mã của Commit, nhưng lớp điều phối quyết định nút nào được tạo Commit và thông điệp nào được đưa vào Rust Sidecar ở thời điểm nào.

3.2 Giao thức người ghi duy nhất theo Epoch

3.2.1 Mục tiêu thiết kế

Concurrent commits là nguyên nhân trực tiếp dẫn đến phân nhánh trạng thái mật mã trong MLS trên P2P. Một hướng tiếp cận là cho phép nhiều Commit xuất hiện rồi xử lý xung đột sau đó. Tuy nhiên, cách này đẩy độ phức tạp vào lõi mật mã và có thể yêu cầu giữ nhiều trạng thái tạm thời, làm yếu thuộc tính bảo mật xuôi. Đề án lựa chọn hướng ngược lại: giảm khả năng xung đột ngay từ nguồn bằng cách giới hạn quyền tạo Commit tại mỗi Epoch.

Single-Writer Protocol đặt ra quy tắc: tại một Epoch E , chỉ một thành viên trong tập ứng viên hợp lệ được quyền tạo Commit. Thành viên này được gọi là Token Holder, ký hiệu là $TH(E)$. Các thành viên khác vẫn được quyền tạo Proposal, nhưng không được tự ý tạo Commit nếu không phải Token Holder. Cơ chế này biến bài toán từ “nhiều nút cùng ghi vào trạng thái nhóm” thành “nhiều nút gửi yêu cầu, một nút được quyền ghi”. Trong điều kiện mạng liên thông và tập active view hội tụ, tất cả nút sẽ tính ra cùng một Token Holder, từ đó tránh concurrent commits mà không cần một máy chủ trung tâm.

3.2.2 Tập ứng viên hợp lệ

Token Holder không được chọn từ toàn bộ định danh nút (PeerID) đang nhìn thấy trên mạng, mà phải được chọn từ tập ứng viên hợp lệ của nhóm. Tập này được định nghĩa dựa trên trạng thái MLS đã được Commit, trạng thái hoạt động của peer và chính sách quyền của ứng dụng.

$$Eligible(E) = \mathcal{M}(E) \cap \mathcal{A}(E) \cap \mathcal{C}(E) \setminus \mathcal{S}(E) \quad (3.1)$$

Trong đó, $\mathcal{M}(E) = GroupMembers(E)$ đại diện cho tập hợp các thành viên hợp lệ của nhóm theo trạng thái mật mã MLS tại Epoch E . Tập hợp $\mathcal{A}(E) = ActiveView(group_id, E)$ là tập các thành viên được quan sát là đang hoạt động trong nhóm thông qua bản tin heartbeat hoặc kết nối P2P gần đây. Đồng thời, $\mathcal{C}(E) = AuthorizedCommitters(E)$ thể hiện tập các thành viên được chính sách ứng dụng cho phép tạo Commit, và $\mathcal{S}(E) = SuspendedOrExcluded(group_id, E)$ là tập hợp các nút bị tạm loại khỏi quá trình điều phối do ngoại tuyến quá lâu, bị phát hiện lỗi hoặc bị chính sách nhóm loại trừ tạm thời.

Việc tách *ActiveView* ra khỏi membership của MLS là cần thiết. Một thành viên có thể vẫn nằm trong ratchet tree nhưng đang offline, do đó không thể đóng vai

trò Token Holder trong thực tế. Ngược lại, một peer đang online nhưng không phải thành viên MLS của nhóm thì không được tham gia tạo Commit. Chỉ giao của các tập này mới có ý nghĩa.

3.2.3 Hàm bầu chọn Token Holder

Token Holder được chọn bằng cách tính hàm băm trên định danh nhóm, số Epoch và định danh peer của từng ứng viên. Ứng viên có giá trị băm nhỏ nhất theo thứ tự từ điển được chọn làm Token Holder:

$$TH(E) = \operatorname{argmin}_{n \in Eligible(E)} H(\text{group_id} \parallel E \parallel n) \quad (3.2)$$

Trong đó H là hàm băm mật mã SHA-256. Việc đưa *group_id* vào đầu vào của hàm băm giúp cùng một peer không nhất thiết là Token Holder ở tất cả các nhóm. Việc đưa Epoch vào hàm băm giúp quyền Token Holder thay đổi ngẫu nhiên sau mỗi lần Commit, tránh tình trạng một nút luôn nắm quyền ghi. Việc đưa PeerID vào hàm băm giúp kết quả có tính phân tán và không phụ thuộc vào thứ tự danh sách cục bộ. Thuật toán bầu chọn Token Holder được mô tả chi tiết trong Thuật toán 1.

Algorithm 1: Thuật toán bầu chọn Token Holder (*ComputeTokenHolder*)

Đầu vào: *groupID*: định danh nhóm; *epoch*: Epoch hiện tại; *members*: tập thành viên MLS; *activeView*: tập thành viên hoạt động; *authorized*: tập thành viên có quyền tạo Commit; *suspended*: tập thành viên bị đình chỉ.

Đầu ra : *holderID*: định danh Token Holder được chọn.

eligible $\leftarrow$ *members* $\cap$ *activeView* $\cap$ *authorized* $\setminus$ *suspended*;

if *eligible* là tập rỗng **then**

return Lỗi *NoEligibleCommitter*;

end

bestID $\leftarrow$ null;

bestHash $\leftarrow$ MAX_HASH;

for mỗi *peerID* trong *eligible* **do**

data $\leftarrow$ Encode(*groupID*, *epoch*, *peerID*);

candidateHash $\leftarrow$ SHA256(*data*);

if *candidateHash* < *bestHash* **then**

bestHash $\leftarrow$ *candidateHash*;

bestID $\leftarrow$ *peerID*;

end

end

return *bestID*;

Thuật toán này không yêu cầu các nút trao đổi phiếu bầu. Nếu các nút có cùng trạng thái nhóm và cùng tập ứng viên hoạt động, chúng chắc chắn tính ra cùng một Token Holder. Vì vậy, chi phí mạng của bước bầu chọn bằng không ($O(0)$ tin nhắn). Chi phí tính toán cục bộ tăng tuyến tính theo số lượng ứng viên $O(k)$ với $k = |Eligible(E)|$.

3.2.4 Tạo Proposal và Commit

Giao thức phân biệt rõ Proposal và Commit. Proposal là yêu cầu thay đổi nhóm, còn Commit là thao tác thực sự chuyển nhóm sang Epoch mới. Mọi thành viên hợp lệ đều có thể tạo Proposal, nhưng chỉ Token Holder được tạo Commit.

Khi một nút muốn thêm thành viên, xóa thành viên hoặc cập nhật khóa, nó gọi Rust Sidecar để tạo Proposal hợp lệ theo MLS, sau đó phát tán Proposal qua GossipSub. Các nút nhận Proposal sẽ kiểm tra chữ ký, kiểm tra Epoch và đưa Proposal vào hàng đợi cục bộ nếu hợp lệ. Token Holder của Epoch hiện tại định kỳ gom các Proposal hợp lệ để tạo Commit. Khoảng thời gian gom Proposal được

cấu hình bằng tham số trì hoãn *batch_delay*. Việc gom Proposal giúp giảm số lần tăng Epoch không cần thiết khi nhiều thay đổi xảy ra gần nhau và giảm khả năng Proposal bị bỏ sót. Quy trình tạo Commit của Token Holder được mô tả trong Thuật toán 2.

Algorithm 2: Quy trình tạo Commit của Token Holder
(*CommitByTokenHolder*)

Đầu vào: *localState*: trạng thái MLS cục bộ tại Epoch E ; *proposalQueue*: hàng đợi Proposal đã nhận; *holderID*: Token Holder tại Epoch E ; *selfID*: định danh nút cục bộ.

Đầu ra : Bản tin Commit được phát tán nếu nút cục bộ là Token Holder.

if *selfID* $\neq$ *holderID* **then**

 Kết thúc (Không có quyền tạo Commit);

end

Chọn tập Proposal hợp lệ P từ *proposalQueue*;

if P rỗng và không có yêu cầu cập nhật khóa cục bộ **then**

 Kết thúc;

end

Gửi yêu cầu *CreateCommit(localState, P)* tới Rust Sidecar;

if Rust Sidecar trả về Commit hợp lệ **then**

 Đóng gói Commit vào *CoordinationEnvelope*;

 Lưu trạng thái pending commit cục bộ;

 Phát tán bản tin Commit qua GossipSub;

end

else

 Ghi nhận lỗi và loại bỏ các Proposal không hợp lệ khỏi hàng đợi;

end

3.2.5 Chuyển quyền và failover

Sau khi Commit hợp lệ được áp dụng, nhóm chuyển sang Epoch $E + 1$. Vì hàm bầu chọn phụ thuộc vào Epoch, Token Holder mới sẽ được tính lại tự động. Cơ chế này giúp quyền ghi được luân chuyển giữa các thành viên, tránh phụ thuộc lâu dài vào một nút.

Trong mạng P2P, Token Holder có thể ngoại tuyến hoặc mất kết nối trước khi tạo Commit. Để tránh bế tắc, mỗi nút duy trì heartbeat và ActiveView. Nếu Token Holder hiện tại không gửi heartbeat hoặc không tạo Commit trong khoảng thời gian vượt quá *holder_timeout*, các nút có thể đánh dấu Token Holder đó là tạm thời không hoạt động và tính lại tập *Eligible(E)*. Khi tập ứng viên thay đổi, hàm

băm sẽ chọn ra Token Holder mới.

Cơ chế failover này không phải đồng thuận Byzantine đầy đủ. Trong trường hợp mạng bị phân mảnh, các phân vùng khác nhau có thể có ActiveView khác nhau và tính ra Token Holder khác nhau. Đồ án chấp nhận đây là hệ quả tự nhiên của network partition. Mục tiêu của Single-Writer không phải ngăn mọi fork trong mọi điều kiện vật lý, mà là loại bỏ concurrent commits trong điều kiện mạng liên thông và biến fork thành một tình huống ngoại lệ có thể phát hiện, sau đó xử lý bằng Fork Healing.

3.3 Kiểm tra nhất quán Epoch

3.3.1 Mục tiêu thiết kế

MLS yêu cầu mỗi Commit được áp dụng trên đúng trạng thái bắt đầu mà nó được tạo ra. Nếu một Commit được tạo từ Epoch E nhưng lại được áp dụng lên trạng thái khác, kết quả sẽ không đúng. Vì vậy, mỗi nút cần một lớp kiểm tra trước khi đưa thông điệp vào Rust Sidecar. Epoch Consistency Checks có nhiệm vụ bảo vệ biên giữa mạng P2P bất đồng bộ và lõi mật mã MLS. Mạng P2P có thể đưa thông điệp đến muộn, đến sớm, lặp lại hoặc đến sai thứ tự. Lớp điều phối phải phân loại các thông điệp này thành thông điệp hiện tại, thông điệp cũ, thông điệp tương lai hoặc thông điệp nghi vấn fork.

3.3.2 Coordination Envelope

Mọi thông điệp MLS khi truyền qua P2P được bọc trong một cấu trúc envelope của lớp điều phối. Envelope không thay thế MLS message, mà bổ sung metadata phục vụ định tuyến, kiểm tra Epoch và phục hồi trạng thái. Cấu trúc của Coordination Envelope được thể hiện trong Mã nguồn 3.1.

Listing 3.1: Cấu trúc Coordination Envelope ở lớp điều phối Go

```

1  type CoordinationEnvelope struct {
2      GroupID      []byte    // Định danh nhóm MLS
3      SenderID     PeerID    // Định danh nút gửi o tang P2P
4      MessageID    []byte    // Chong xu ly lap lai tin nhan
5      ContentType  Type      // Proposal | Commit | Application
           ↳ | Heartbeat | StateSync
6      Epoch        uint64    // Epoch tai thoi diem tao payload
7      HLCTimestamp HLC       // Dong ho logic lai HLC cho tin
           ↳ nhan ung dung
8      TreeHashHint []byte    // Tree hash cuc bo cua nguoi gui
           ↳ (tuy chon)
9      CommitHash   []byte    // Commit hash tuong ung (tuy chon
           ↳ )
10     Payload       []byte    // Du lieu ciphertext/plaintext
    
```

```

11     ↪ MLS
12     Signature    []byte    // Chu ký xác thực lớp điều phối
13 }

```

Trường *group_id* xác định nhóm MLS. Trường *sender_id* xác định nút gửi ở tầng P2P. Trường *message_id* dùng để chống xử lý lặp lại cùng một envelope. Trường *content_type* giúp lớp điều phối phân biệt handshake message và application message. Trường *epoch* là Epoch mà payload được tạo ra. Trường *hlc_timestamp* chỉ có ý nghĩa sắp xếp application message. Các trường như *tree_hash_hint* hoặc *commit_hash* hỗ trợ phát hiện fork và đồng bộ trạng thái. Trường *payload* chứa MLS message thật sự. Trường *signature* của envelope được dùng để xác thực metadata điều phối, trong khi payload MLS vẫn có chữ ký và xác thực riêng theo chuẩn mật mã bên dưới.

3.3.3 Phân loại thông điệp theo Epoch

Khi nhận envelope, Go Host lấy Epoch cục bộ của nhóm tương ứng và so sánh với *env.epoch*. * Nếu *env.epoch == localEpoch*, thông điệp thuộc Epoch hiện tại. Nếu phát hiện *tree_hash_hint* hoặc *epoch_authenticator* không tương thích trạng thái cục bộ, Go Host kích hoạt cơ chế phát hiện Fork. Ngược lại, nếu là Proposal hoặc Commit, payload được chuyển cho Rust Sidecar để xử lý handshake. Nếu là tin nhắn ứng dụng, payload được chuyển cho Rust Sidecar để giải mã và đưa vào hàng đợi hiển thị theo HLC. * Nếu *env.epoch < localEpoch*, thông điệp là thông điệp cũ. Với Commit hoặc Proposal cũ, nút cục bộ không áp dụng lại. Với tin nhắn ứng dụng cũ, hệ thống có thể thử giải mã nếu chính sách giữ khóa cũ (stale window) cho phép. * Nếu *env.epoch > localEpoch*, thông điệp thuộc về tương lai, biểu thị nút cục bộ đã bỏ lỡ các Commit trung gian. Go Host đưa envelope vào *future_buffer* và kích hoạt State Sync để đồng bộ trạng thái còn thiếu.

Quy trình xử lý phong tỏa này được mô tả trong Thuật toán 3.

Algorithm 3: Quy trình phân loại và xử lý Envelope đến
(*ProcessIncomingEnvelope*)

Đầu vào: *env*: bản tin CoordinationEnvelope nhận được; *localState*: trạng thái nhóm cục bộ.

Đầu ra : Hành động xử lý tương ứng cục bộ.

if *env.message_id* đã được xử lý **then**

 Bỏ qua envelope;

 Kết thúc;

end

if *env.groupID* không tồn tại cục bộ **then**

 Bỏ qua hoặc đưa vào hàng đợi xử lý gia nhập;

 Kết thúc;

end

localEpoch $\leftarrow$ *localState.epoch*;

if *env.epoch* == *localEpoch* **then**

if *env.tree_hash_hint* $\neq$ *localState.tree_hash* **then**

 Kích hoạt *ForkDetection*(*env*);

 Kết thúc;

end

if *env.contentType* là *Proposal* hoặc *Commit* **then**

 Chuyển *env.payload* sang Rust Sidecar để xử lý handshake;

end

if *env.contentType* là *Application* **then**

 Chuyển *env.payload* sang Rust Sidecar để giải mã;

 Đưa tin nhắn giải mã vào hàng đợi hiển thị theo HLC;

end

end

else if *env.epoch* < *localEpoch* **then**

if *env.contentType* là *Application* và thuộc *stale window* **then**

 Thử giải mã bằng khóa cũ được lưu giữ;

end

else if *env.contentType* là *Proposal* **then**

 Gửi yêu cầu *RePropose* cho người gửi hoặc bỏ qua;

end

else

 Bỏ qua envelope;

end

end

else

3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai

Khi một nút nhận thông điệp tương lai, nó không tự ý tăng Epoch cục bộ. Epoch chỉ được tăng khi Rust Sidecar xử lý thành công Commit hợp lệ. Do đó, quy trình State Sync cần lấy chuỗi Commit còn thiếu.

State Sync hoạt động theo hai chế độ. Ở chế độ thứ nhất, nút yêu cầu các bản tin Commit còn thiếu từ Epoch hiện tại đến Epoch đích. Nếu các Commit còn được lưu trong lịch sử và có thể kiểm tra tuần tự, nút áp dụng từng Commit để bắt kịp Epoch. Ở chế độ thứ hai, nếu lịch sử bị khuyết thiếu đáng kể hoặc trạng thái đã rẽ nhánh phức tạp, nút yêu cầu thông tin nhóm đầy đủ (*GroupInfo* hoặc *Welcome*) hoặc thực hiện quy trình External Join. Việc không tự động áp dụng thông điệp tương lai giúp bảo vệ tính toàn vẹn của máy trạng thái mật mã.

3.3.5 Xử lý Proposal bị bỏ sót

Một dị thường quan trọng trong môi trường P2P là Proposal bị bỏ sót do Commit đã xảy ra trước khi Proposal đến được Token Holder. Ví dụ, tại Epoch E , nút A gửi Proposal P_A , nút B gửi Proposal P_B . Token Holder nhận P_A trước, tạo Commit đưa nhóm lên $E + 1$. Sau đó P_B mới đến và trở thành Proposal cũ.

Để tránh mất mát thao tác của người dùng, giao thức sử dụng cơ chế Re-Propose. Khi một nút phát hiện Proposal của mình không được đưa vào Commit của Epoch tương ứng, nó kiểm tra xem thao tác đó còn hợp lệ trong Epoch mới hay không. Nếu còn hợp lệ, nút tạo lại Proposal trên Epoch mới và phát tán lại. Nếu không còn hợp lệ, ứng dụng hiển thị thông báo lỗi rõ ràng cho người dùng. Quy trình xử lý này được mô tả trong Thuật toán 4.

Algorithm 4: Quy trình Re-Propose cho Proposal bị bỏ sót (*ReProposeIfDropped*)

Đầu vào: *localProposal*: Proposal do nút cục bộ tạo tại Epoch E ;
appliedCommit: Commit đã được áp dụng để chuyển sang Epoch $E + 1$; *newState*: trạng thái nhóm sau Commit.

if *localProposal* nam trong *appliedCommit* **then**

 Đánh dấu Proposal đã hoàn thành và áp dụng;
 Kết thúc;

end

if *localProposal* không con hợp lệ trong *newState* **then**

 Đánh dấu Proposal thất bại và thông báo cho người dùng;
 Kết thúc;

end

Tạo Proposal mới tương đương tại Epoch $E + 1$;

Phát tán Proposal mới qua GossipSub;

3.4 Cơ chế phục hồi phân nhánh trạng thái

3.4.1 Bối cảnh network partition

Single-Writer hoạt động tốt khi các nút có cùng góc nhìn về ActiveView. Tuy nhiên, trong mạng P2P, network partition có thể chia nhóm thành nhiều phân vùng không thể liên lạc với nhau. Mỗi phân vùng có thể loại các nút ngoài phân vùng khỏi ActiveView và bầu ra Token Holder riêng. Khi đó, nhiều phân vùng có thể tạo các Commit khác nhau dựa trên cùng một Epoch gốc. Đây là giới hạn nền tảng của hệ phân tán bất đồng bộ. Vấn đề quan trọng là khi mạng kết nối lại, hệ thống phải tự động phát hiện được phân nhánh và đưa nhóm hội tụ về một trạng thái duy nhất.

3.4.2 Phát hiện fork bằng heartbeat trạng thái

Mỗi nút định kỳ phát thông điệp trạng thái ‘GroupStateAnnouncement’ cho các nhóm mà nó tham gia qua kênh GossipSub. Thông điệp này không chứa bí mật mật mã, mà chỉ chứa các định danh trạng thái công khai. Cấu trúc của thông điệp trạng thái được thể hiện trong Mã nguồn 3.2.

Listing 3.2: Cấu trúc thông điệp State Announcement để phát hiện Fork

```
1 type GroupStateAnnouncement struct {
2     GroupID          []byte // Định danh nhóm
3     SenderID         PeerID  // Định danh nút gửi o tang
4     Epoch            uint64  // Epoch cục bộ hiện tại của
5     // P2P
6     // nút
```

```

5      TreeHash          [][]byte      // Tree hash tương ứng của cây
      ↪ TreeKEM
6      EpochAuthenticator [][]byte      // Giá trị xác thực Epoch hiện
      ↪ tại
7      LastCommitHash    [][]byte      // Mã băm Commit cuối cùng
      ↪ được áp dụng
8      ActiveMemberCount uint32        // Số thành viên đang hoạt
      ↪ động trong ActiveView
9      HLCTimestamp      HLC           // Thời gian logic lai HLC
10     Signature          [][]byte      // Chu ký xác thực của người
      ↪ gửi
11 }

```

Khi nhận, nút cục bộ so sánh với trạng thái của mình. Nếu định danh nhóm giống nhau nhưng *tree_hash* hoặc *epoch_authenticator* khác nhau tại cùng Epoch, hệ thống xác định hai nút không ở cùng trạng thái mật mã và kích hoạt cơ chế Fork Healing. Việc so sánh mã băm cây *tree_hash* đảm bảo phát hiện chính xác mọi rẽ nhánh trạng thái.

3.4.3 Hàm trọng số nhánh

Sau khi phát hiện fork, các nút cần chọn một nhánh làm nhánh thắng cuộc để hội tụ. Đồ án đề xuất sử dụng hàm trọng số nhánh có thứ tự ưu tiên từ cao xuống thấp:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (3.3)$$

Trong đó: * $C_{active}(B)$ là số thành viên đang hoạt động trong ActiveView và xác nhận nhánh B . * $E(B)$ là số Epoch hiện tại của nhánh. * $H_{commit}(B)$ là mã băm Commit cuối cùng của nhánh, dùng làm tie-breaker tất định.

Hai nhánh được so sánh theo thứ tự từ trái sang phải. Nhánh có nhiều thành viên hoạt động hơn được ưu tiên nhằm giảm thiểu số lượng thiết bị phải thực hiện gia nhập lại. Nếu bằng nhau, nhánh có Epoch cao hơn được chọn. Nếu vẫn bằng nhau, mã băm Commit được so sánh từ điển để phá hòa một cách tất định cục bộ. Quy trình so sánh được mô tả trong Thuật toán 5.

Algorithm 5: Thuật toán so sánh trọng số nhánh để giải quyết Fork
(*CompareBranchWeight*)

Đầu vào: *localBranch*: thông tin nhánh cục bộ; *remoteBranch*: thông tin nhánh nhận từ mạng.

Đầu ra : Kết quả so sánh: *BranchLocal* (Nhánh cục bộ thắng),
BranchRemote (Nhánh từ mạng thắng) hoặc *BranchEqual*
(Bằng nhau).

```

if localBranch.tree_hash == remoteBranch.tree_hash and
    localBranch.epoch_authenticator ==
    remoteBranch.epoch_authenticator then
    |   return BranchEqual;
end
if localBranch.active_count ≠ remoteBranch.active_count then
    |   if localBranch.active_count > remoteBranch.active_count then
    |   |   return BranchLocal;
    |   end
    |   else
    |   |   return BranchRemote;
    |   end
end
if localBranch.epoch ≠ remoteBranch.epoch then
    |   if localBranch.epoch > remoteBranch.epoch then
    |   |   return BranchLocal;
    |   end
    |   else
    |   |   return BranchRemote;
    |   end
end
if localBranch.last_commit_hash < remoteBranch.last_commit_hash
    then
    |   return BranchLocal;
end
else
    |   return BranchRemote;
end

```

Hàm trọng số này là chính sách hội tụ tắt định cho môi trường mà các nút trung

thực nhưng bị phân mảnh mạng. Đối với các kịch bản nút độc hại cố tình giả mạo thông tin, hệ thống cần bổ sung cơ chế xác thực quyền hành và các chính sách bảo mật khác.

3.4.4 Quy trình Fork Healing

Sau khi xác định nhánh thắng, các nút thuộc nhánh thua thực hiện quy trình hàn gắn gồm các bước logic sau: 1. ****Đóng băng nhánh thua:**** Nút dừng tạo Commit hoặc gửi tin nhắn ứng dụng mới trên trạng thái nhánh thua. Các tin nhắn cục bộ chưa gửi được đưa vào hàng đợi phục hồi. 2. ****Hủy vật liệu khóa nhánh thua:**** Các bí mật mật mã không còn thuộc nhánh được chọn phải bị xóa sạch khỏi bộ nhớ nhằm bảo toàn tính bảo mật xuôi (Forward Secrecy). 3. ****Gia nhập lại nhánh thắng cuộc:**** Nút lấy thông tin nhóm (*GroupInfo* hoặc *Welcome*) từ nhánh thắng cuộc, xác thực thông tin và thực hiện quy trình gia nhập lại thông qua cơ chế *External Join* của MLS. 4. ****Phục hồi tin nhắn (Autonomous Replay):**** Các tin nhắn ứng dụng do chính nút đó tạo trong thời gian phân mảnh mạng nhưng chưa xuất hiện trên nhánh thắng sẽ được mã hóa lại và phát lại dưới Epoch mới của nhánh thắng cuộc. Nút tuyệt đối không phát lại tin nhắn của nút khác để bảo toàn tính xác thực nguồn gửi.

Quy trình Fork Healing tổng quát được mô tả trong Thuật toán 6.

Algorithm 6: Quy trình hàn gắn phân nhánh mật mã (*ForkHealing*)

Đầu vào: *localBranch*: thông tin nhánh cục bộ; *winningBranch*: thông tin nhánh thắng cuộc; *outbox*: hàng đợi tin nhắn cục bộ chưa gửi trong thời gian phân mảnh.

if *localBranch* == *winningBranch* **then**

 Tiếp tục hoạt động bình thường;
 Kết thúc;

end

Phong tỏa các thao tác ghi và gửi tin nhắn trên *localBranch*;

Lưu danh sách tin nhắn do chính mình tạo trong *outbox*;

Tiêu hủy tức thời các bí mật mật mã liên quan đến *localBranch*;

Lấy thông tin nhóm (*GroupInfo*) mới nhất từ *winningBranch*;

Xác thực danh tính và tính hợp lệ của *winningBranch*;

Gửi yêu cầu *ExternalJoin* tới Rust Sidecar để gia nhập nhánh thắng cuộc;

if Gia nhập thành công **then**

 Cập nhật trạng thái MLS cục bộ sang nhánh mới;

for mỗi *msg* trong *outbox* do chính mình tạo **do**

 Mã hóa lại *msg* dưới Epoch mới của nhóm;
 Phát tán tin nhắn qua GossipSub;

end

end

else

 Đánh dấu trạng thái lỗi và yêu cầu can thiệp đồng bộ thủ công;

end

3.4.5 Lưu trạng thái bền vững trong quá trình hàn gắn

Fork Healing là quy trình có nhiều bước và có thể bị gián đoạn do sự cố thiết bị hoặc mất điện. Để đảm bảo tính bền vững, Go Host lưu trạng thái phục hồi vào SQLite theo cấu trúc thực thể được định nghĩa trong Mã nguồn 3.3.

Listing 3.3: Cấu trúc HealingState lưu trữ trong cơ sở dữ liệu SQLite

```
1 type HealingState struct {
2     GroupID          []byte    // Định danh nhóm MLS
3     OldBranchID      []byte    // Nhánh cũ bị có lỗi
4     WinningBranchID  []byte    // Nhánh thắng cuộc được chọn
5     ↪ de hoi tu
6     Phase            string    // Detecting | Freezing |
7     ↪ Joining | Replaying | Completed | Failed
```

```

6      PendingMessages  [][]byte // Hàng đợi tin nhắn cục bộ cần
      ↪ replay
7      LastError         string   // Ghi nhận lỗi hệ thống nếu có
8      UpdatedAt         time.Time // Thời gian cập nhật trạng
      ↪ thái gần nhất
9  }

```

Khi ứng dụng khởi động lại, Go Host kiểm tra bảng trạng thái phục hồi này. Nếu ghi nhận nhóm đang ở pha *Joining* hoặc *Replaying*, hệ thống sẽ tự động tiếp tục quy trình từ bước tương ứng thay vì bắt đầu lại từ đầu, đảm bảo tính sẵn sàng và an toàn cho dữ liệu người dùng.

3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock

3.5.1 Lý do cần HLC

Epoch trong MLS chỉ phản ánh thứ tự thay đổi trạng thái mật mã (handshake). Nó không cung cấp thứ tự toàn phần cho các tin nhắn ứng dụng được gửi song song trong cùng một Epoch. Alice và Bob có thể gửi tin nhắn đồng thời và các nút khác nhau nhận được chúng theo thứ tự ngược nhau do độ trễ truyền dẫn mạng. Nếu ứng dụng hiển thị theo thời điểm nhận cục bộ, người dùng trên các thiết bị khác nhau sẽ quan sát thấy lịch sử hội thoại không nhất quán.

Sử dụng đồng hồ vật lý cục bộ không khả thi trong mạng P2P vì các thiết bị thường bị lệch giờ hoặc không đồng bộ NTP. Do đó, đề án sử dụng đồng hồ logic lai HLC để gán nhãn thời gian cho các thông điệp ứng dụng. HLC kết hợp thời gian vật lý và bộ đếm logic, biểu diễn dưới dạng bộ ba:

$$HLC = (L, C, N) \quad (3.4)$$

Trong đó L là thời gian vật lý lớn nhất từng được ghi nhận, C là bộ đếm logic cho các sự kiện đồng thời, và N là định danh nút (PeerID) để phá hòa một cách tất định.

3.5.2 Tạo timestamp khi gửi tin

Khi người dùng gửi tin nhắn ứng dụng, Go Host gọi hàm ‘HLC_Now’. Nếu thời gian vật lý hiện tại của hệ thống lớn hơn giá trị L đang lưu, HLC đặt L bằng thời gian vật lý hiện tại và khởi tạo bộ đếm $C = 0$. Ngược lại, nếu thời gian vật lý không tăng, hệ thống tăng bộ đếm C lên 1 để đảm bảo tính đơn điệu tăng của thời gian. Quy trình được mô tả trong Thuật toán 7.

Algorithm 7: Thuật toán sinh thời gian logic lai (HLC_Now)

Đầu vào: hlc : trạng thái HLC cục bộ; $nodeID$: định danh nút cục bộ.

Đầu ra : Timestamp HLC mới ($L, C, nodeID$).

$pt \leftarrow$ thời gian vật lý hiện tại (mili-giây);

if $pt > hlc.L$ **then**

$hlc.L \leftarrow pt$;

$hlc.C \leftarrow 0$;

end

else

$hlc.C \leftarrow hlc.C + 1$;

end

return ($hlc.L, hlc.C, nodeID$);

Timestamp này được đóng gói trực tiếp vào CoordinationEnvelope bên ngoài payload tin nhắn.

3.5.3 Cập nhật HLC khi nhận tin

Khi nhận một envelope có chứa timestamp HLC từ mạng, nút cục bộ tiến hành cập nhật HLC của mình dựa trên thời gian vật lý cục bộ và HLC nhận được để đảm bảo đồng hồ logic tiến lên một cách nhất quán. Quy trình cập nhật được mô tả trong Thuật toán 8.

Algorithm 8: Thuật toán cập nhật đồng hồ logic lai (HLC_Update)

Đầu vào: hlc : trạng thái HLC cục bộ; msg : timestamp HLC nhận được
 $(msg.L, msg.C, msg.NodeID)$.

Đầu ra : Trạng thái HLC cục bộ sau khi cập nhật.

$pt \leftarrow$ thời gian vật lý hiện tại (mili-giây);

if $msg.L - pt > MAX_DRIFT_MS$ **then**

 Đánh dấu thông điệp nghi vấn clock drift lớn và xử lý theo chính sách;
 Kết thúc;

end

$newL \leftarrow \max(hlc.L, msg.L, pt)$;

if $newL == hlc.L$ **and** $newL == msg.L$ **then**

$hlc.C \leftarrow \max(hlc.C, msg.C) + 1$;

end

else if $newL == hlc.L$ **then**

$hlc.C \leftarrow hlc.C + 1$;

end

else if $newL == msg.L$ **then**

$hlc.C \leftarrow msg.C + 1$;

end

else

$hlc.C \leftarrow 0$;

end

$hlc.L \leftarrow newL$;

return hlc ;

Hằng số MAX_DRIFT_MS được sử dụng để phát hiện các thông điệp có timestamp sai lệch đáng kể so với đồng hồ cục bộ nhằm ngăn chặn hành vi cố tình đẩy timestamp về tương lai để làm xáo trộn thứ tự dòng chat.

3.5.4 Thứ tự hiển thị tất định

Tin nhắn ứng dụng sau khi giải mã được sắp xếp hiển thị dựa trên khóa thứ tự:

$$OrderKey = (epoch, HLC.L, HLC.C, senderID, messageID) \quad (3.5)$$

Quy tắc sắp xếp tuân theo: trước hết phân loại theo Epoch của nhóm mật mã; trong cùng Epoch, sắp xếp theo thời gian logic lai L và bộ đếm C ; nếu trùng, sử dụng định danh người gửi và định danh tin nhắn để phá hòa một cách tất định. Nhờ đó, tất cả các thiết bị trong nhóm nhận cùng một tập tin nhắn sẽ hiển thị hội thoại

theo một thứ tự hoàn toàn đồng nhất.

3.6 Quản lý trạng thái Go-Rust và lưu trữ cục bộ

3.6.1 Lý do sử dụng Rust Sidecar phi trạng thái

Trong nguyên mẫu hiện tại, Go Host lưu toàn bộ trạng thái MLS bền vững trong SQLite. Mỗi lần cần mã hóa, giải mã hoặc tạo Commit, Go lấy trạng thái nhóm hiện tại, gửi sang Rust Sidecar qua gRPC và nhận lại trạng thái mới sau khi OpenMLS xử lý. Cách này có chi phí tuần tự hóa và truyền dữ liệu qua IPC, nhưng đổi lại giúp tách nhiệm lưu trữ và khôi phục trạng thái thuộc về Go/SQLite một cách rõ ràng.

Do đó, Rust Sidecar được thiết kế như một lớp xử lý mật mã thuần túy: Rust không quyết định ordering, không lưu trạng thái nhóm lâu dài và không trở thành nguồn sự thật của ứng dụng. Mọi quyết định về epoch, Token Holder, lưu lịch sử và phục hồi sau sự cố đều nằm ở Go Host và cơ sở dữ liệu cục bộ.

3.6.2 Phân chia trách nhiệm giữa SQLite và Rust Sidecar

SQLite đóng vai trò là kho lưu trữ dữ liệu bền vững của Go Host, đảm bảo dữ liệu không bị mất khi ứng dụng tắt hoặc thiết bị mất điện. SQLite lưu trữ lịch sử tin nhắn, trạng thái điều phối, hàng đợi tin nhắn chưa gửi và snapshot trạng thái MLS. Rust Sidecar nhận snapshot này như dữ liệu đầu vào, thực hiện thao tác OpenMLS tương ứng, rồi trả lại kết quả để Go Host lưu xuống SQLite.

3.6.3 Luồng xử lý khi gửi tin nhắn ứng dụng

Quy trình gửi tin nhắn ứng dụng được mô tả chi tiết trong Thuật toán 9.

Algorithm 9: Quy trình gửi tin nhắn ứng dụng (*SendApplicationMessage*)

Đầu vào: *groupID*: định danh nhóm; *plaintext*: nội dung tin nhắn dạng rõ.

localState $\leftarrow$ lấy trạng thái nhóm cục bộ từ SQLite;

timestamp $\leftarrow$ *HLC_Now()*;

Gọi Rust Sidecar:

EncryptApplicationMessage(localState.groupState, plaintext);

Nhận về bản mã *mlsCiphertext* và trạng thái nhóm mới từ Sidecar;

Lưu trạng thái nhóm mới vào SQLite;

Tạo *CoordinationEnvelope* mang *groupID*, *epoch* = *localState.epoch*,

hlc_timestamp = *timestamp* và *payload* = *mlsCiphertext*;

Lưu tin nhắn vào SQLite ở trạng thái chờ gửi (*pending*);

Phát tán envelope qua GossipSub mạng lưới P2P;

Tin nhắn ứng dụng không làm thay đổi Epoch mật mã nên không yêu cầu quyền Token Holder, nhưng vẫn phải mang Epoch hiện tại để nút nhận xác định đúng

khóa giải mã.

3.6.4 Luồng xử lý khi thay đổi thành viên

Quy trình xử lý khi người dùng muốn thêm hoặc xóa thành viên được mô tả trong Thuật toán 10.

Algorithm 10: Quy trình yêu cầu thay đổi thành viên
(*RequestMembershipChange*)

Đầu vào: *groupID*: định danh nhóm; *operation*: loại thao tác
(Add/Remove); *targetMember*: thành viên mục tiêu.

Gọi Rust Sidecar để tạo đối tượng Proposal tương ứng với thao tác;

Tạo *CoordinationEnvelope* mang loại *Proposal*, *epoch = localEpoch* và
payload = proposal;

Lưu Proposal vào hàng đợi cục bộ *proposalQueue*;

Phát tán Proposal qua GossipSub mạng P2P;

if Bản thân nút cục bộ là *Token Holder* của *Epoch* hiện tại **then**

Thiết lập bộ đếm thời gian gửi Commit sau khoảng trì hoãn
batch_delay;

end

Việc tách biệt Proposal và Commit cho phép nhiều thành viên đề xuất thay đổi cùng lúc mà không phá vỡ nguyên tắc một người ghi duy nhất (Single-Writer) tại mỗi Epoch.

3.7 Tính đúng đắn trực giác của phương pháp

Tính đúng đắn trực giác của giao thức điều phối đề xuất được thể hiện qua các luận điểm khoa học sau: * **Triệt tiêu rẽ nhánh mật mã ở điều kiện mạng liên thông:** Nhờ cơ chế Single-Writer, tại mỗi Epoch chỉ có duy nhất Token Holder được phép sinh bản tin Commit để nâng Epoch của nhóm. Do đó, hiện tượng rẽ nhánh trạng thái mật mã do các Commit đồng thời cạnh tranh sẽ không thể xảy ra trong điều kiện mạng hoạt động bình thường và liên thông. * **Bảo vệ máy trạng thái cục bộ:** Lớp kiểm tra Epoch Checks hoạt động như một bộ lọc biên chặn đúng các thông điệp đến sai thứ tự hoặc không đồng bộ, đưa chúng vào hàng đệm tương lai hoặc loại bỏ. Điều này đảm bảo lỗi mật mã MLS bên dưới luôn được cập nhật theo chuỗi Epoch tuyến tính đơn điệu, tránh lỗi sụp đổ máy trạng thái do nhận Commit ngoài luồng. * **Khả năng hội tụ an toàn sau phân mảnh mạng:** Khi có network partition, các phân vùng mạng rẽ nhánh độc lập. Quy trình Fork Healing sử dụng hàm trọng số nhánh tất định dựa trên số lượng thành viên hoạt động để chọn ra nhánh thắng cuộc một cách thống nhất. Các nút nhánh thua tiêu hủy tức thời khóa nhánh cũ và gia nhập lại nhánh thắng cuộc qua cơ chế External Join hợp

lệ của MLS, bảo toàn tính chất Forward Secrecy và Post-Compromise Security của nhóm nhắn tin. * **Tính nhất quán của giao diện hiển thị:** Mặc dù các tin nhắn ứng dụng thông thường được gửi song song và bất đồng bộ, việc sử dụng đồng hồ logic lai HLC để sắp xếp hiển thị đảm bảo tất cả các thành viên trong nhóm luôn quan sát thấy một lịch sử hội thoại đồng nhất, bảo vệ trải nghiệm người dùng.

3.8 Kết chương

Chương 3 đã trình bày chi tiết về giao thức điều phối phi tập trung đề xuất cho MLS trên mạng ngang hàng. Bằng việc xây dựng kiến trúc hai lớp Go-Rust sidecar kết hợp bốn cơ chế cốt lõi — gồm Single-Writer Protocol để kiểm soát quyền ghi Commit, Epoch Consistency Checks để bảo vệ trạng thái Epoch cục bộ, Fork Healing để tự động phục hồi khi mạng bị phân mảnh, và HLC Message Ordering để sắp xếp thứ tự hiển thị tin nhắn — giải pháp đề xuất đã giải quyết thành công bài toán nhất quán trạng thái mật mã của MLS mà không cần dựa vào Delivery Service tập trung hay các thuật toán đồng thuận đắt đỏ.

Những đề xuất thiết kế và thuật toán logic chi tiết trong chương này là cơ sở trực tiếp để Chương tiếp theo — Chương 4 tiến hành phân tích lý thuyết chuyên sâu về độ phức tạp tính toán, băng thông truyền thông, tính nhất quán trạng thái mật mã và phạm vi bảo vệ an toàn thông tin của hệ thống.

CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT

4.1 Mục tiêu và phạm vi phân tích

Chương này phân tích cơ sở lý thuyết của lớp điều phối phi tập trung được đề xuất trong đề án. Các chương trước đã trình bày bối cảnh bài toán, nền tảng MLS và kiến trúc tổng thể của hệ thống. Trọng tâm của chương này là trả lời câu hỏi: vì sao các cơ chế Single-Writer, Epoch Checks, Fork Healing và HLC Message Ordering có thể giúp MLS vận hành trong môi trường mạng ngang hàng mà không cần một Delivery Service trung tâm đảm nhiệm toàn bộ vai trò điều phối Commit.

Điểm cần làm rõ trước hết là đề án không chứng minh lại các thuộc tính mật mã của MLS. Các thuộc tính như xác thực thành viên, bảo mật thông điệp, Forward Secrecy và Post-Compromise Security thuộc về bản thân chuẩn MLS và được kế thừa thông qua việc sử dụng OpenMLS [1]. Đóng góp của đề án nằm ở lớp điều phối phía trên MLS: lớp này không thay đổi thuật toán mã hóa, không thay đổi key schedule, và không tự tạo ra trạng thái mật mã ngoài mô hình của MLS.

Trong MLS, trạng thái nhóm tiến hóa theo chuỗi các epoch. Mỗi Commit có nhiệm vụ áp dụng một tập Proposal và đưa nhóm sang epoch mới [1]. Khi có Delivery Service nhất quán mạnh, hệ thống có thể dựa vào DS để áp đặt một thứ tự tuyến tính cho các Commit, đồng thời phá hòa khi nhiều thành viên gửi Commit tại cùng một epoch [2]. Tuy nhiên, trong môi trường P2P hoặc eventually consistent delivery, các client có thể nhận thông điệp theo các thứ tự khác nhau và phải tự thực hiện reconciliation [2]. Đây chính là khoảng trống mà lớp điều phối của đề án hướng tới xử lý.

Về mặt lý thuyết, bài toán của đề án có thể phát biểu như sau: trong một hệ thống không có DS trung tâm, làm thế nào để các nút P2P vẫn duy trì được một chuỗi trạng thái MLS hợp lệ, hoặc ít nhất phát hiện và phục hồi khi chuỗi trạng thái đó bị phân nhánh. Do tính chất của hệ phân tán, đặc biệt khi xảy ra network partition, hệ thống không thể đồng thời đảm bảo tuyệt đối tính nhất quán mạnh, tính sẵn sàng và khả năng chịu phân mảnh [8]. Vì vậy, mục tiêu của đề án không phải là loại bỏ mọi khả năng fork trong mọi điều kiện mạng, mà là đạt ba mục tiêu thực tế hơn:

- giảm khả năng sinh concurrent commits trong điều kiện các nút có cùng góc nhìn về nhóm;
- phát hiện được fork khi các phân vùng mạng tiến hóa sang các trạng thái MLS khác nhau;
- đưa các nút hội tụ trở lại một trạng thái MLS hợp lệ sau khi mạng kết nối trở

lại.

Các phân tích trong chương này được trình bày theo dạng bất biến và lập luận tính đúng đắn. Đây không phải là chứng minh hình thức đầy đủ theo mô hình toán học hoặc mô hình đối kháng mạnh, nhưng đủ để làm rõ vì sao các cơ chế đề xuất phù hợp với mục tiêu của đề án và vì sao chúng không phá vỡ mô hình mật mã của MLS.

4.2 Mô hình phân tích

Xét một nhóm MLS có định danh $groupID$ và tập thành viên tại epoch E là M_E . Mỗi nút trong nhóm duy trì một trạng thái cục bộ S_E , bao gồm số epoch hiện tại, trạng thái cây MLS, các giá trị định danh trạng thái như tree hash, transcript hash, epoch authenticator, và một số thông tin điều phối ở tầng ứng dụng. Hai nút được xem là có cùng trạng thái MLS tại epoch E nếu các định danh trạng thái quan trọng của chúng trùng nhau, đặc biệt là epoch, tree hash và epoch authenticator.

Mạng P2P được xem là môi trường bất đồng bộ. Thông điệp có thể đến trễ, đến sai thứ tự, bị lặp hoặc tạm thời không đến được. Trong điều kiện mạng liên thông, các nút cuối cùng có thể nhận được thông tin từ nhau, nhưng không giả định rằng mọi nút nhận thông điệp theo cùng một thứ tự. Trong điều kiện network partition, nhóm có thể bị chia thành nhiều phân vùng, mỗi phân vùng chỉ quan sát được một phần các thành viên đang hoạt động.

Ký hiệu $ActiveView_i(E)$ là tập thành viên mà nút i quan sát là đang hoạt động tại epoch E . Trong điều kiện mạng ổn định, các nút có xu hướng hội tụ về cùng một $ActiveView$. Ngược lại, khi mạng bị phân mảnh, hai phân vùng khác nhau có thể có hai $ActiveView$ khác nhau. Đây là nguyên nhân trực tiếp khiến cùng một công thức bầu Token Holder có thể cho ra kết quả khác nhau giữa các phân vùng.

Trong phạm vi phân tích, đề án giả định các nút tuân thủ giao thức ở mức cơ bản và OpenMLS xử lý đúng các thao tác mật mã. Các hành vi đối kháng phức tạp không phải trọng tâm của chương này. Cách giới hạn phạm vi như vậy giúp phân tích tập trung vào vấn đề chính của đề án: điều phối trạng thái MLS trong môi trường P2P không ổn định.

4.3 Bất biến 1: Epoch cục bộ không được thoái lui

4.3.1 Phát biểu bất biến

Tại mỗi nút, giá trị epoch cục bộ chỉ được giữ nguyên hoặc tăng lên sau khi nút xử lý thành công một Commit hợp lệ. Nút không được áp dụng một thông điệp thuộc epoch cũ lên trạng thái hiện tại, vì điều này có thể dẫn đến rollback trạng thái mật mã hoặc xử lý bản mã trên một trạng thái cây MLS không còn tương thích.

Bất biến này có thể viết ngắn gọn như sau:

$$E_i(t+1) \geq E_i(t)$$

trong đó $E_i(t)$ là epoch cục bộ của nút i tại thời điểm logic t .

4.3.2 Cơ sở từ MLS

MLS tổ chức lịch sử nhóm thành một chuỗi các epoch, trong đó mỗi epoch biểu diễn một trạng thái nhóm với tập thành viên đã xác thực cùng chia sẻ trạng thái mật mã [1]. Mỗi Commit khởi tạo một epoch mới bằng cách yêu cầu các thành viên áp dụng một tập Proposal và cập nhật ratchet tree [1]. Vì vậy, nếu một nút áp dụng thông điệp sai epoch, nút đó có nguy cơ đặt một thông điệp hợp lệ về mặt định dạng lên một trạng thái mật mã không còn tương thích.

4.3.3 Cơ chế bảo toàn bất biến

Cơ chế Epoch Checks của đồ án bọc mỗi thông điệp nhận từ mạng trong một lớp envelope chứa *groupID*, loại thông điệp và epoch của người gửi. Khi nhận thông điệp, lớp điều phối so sánh epoch của thông điệp với epoch cục bộ trước khi chuyển thông điệp xuống OpenMLS.

Quan hệ epoch	Hành động của lớp điều phối
$E_{msg} = E_{local}$	Thông điệp được chuyển tiếp cho lớp MLS xử lý theo luồng tiến hóa bình thường.
$E_{msg} < E_{local}$	Thông điệp bị từ chối áp dụng lên trạng thái MLS hiện tại nhằm ngăn chặn hành vi thoái lui (rollback) trạng thái.
$E_{msg} > E_{local}$	Thông điệp chưa được xử lý ngay; nút đưa vào bộ đệm tạm thời và kích hoạt quá trình đồng bộ trạng thái.

Bảng 4.1: Quy tắc xử lý thông điệp theo epoch

Mệnh đề 1. Nếu một nút chỉ tăng epoch sau khi xử lý thành công Commit hợp lệ và từ chối áp dụng thông điệp thuộc epoch cũ, thì epoch cục bộ của nút là đơn điệu không giảm.

Lập luận. Giả sử một nút chỉ tăng epoch sau khi xử lý thành công một Commit hợp lệ. Theo quy tắc định tuyến tại Bảng 4.1, nút từ chối áp dụng mọi thông điệp có $E_{msg} < E_{local}$, do đó trạng thái MLS cục bộ không bao giờ bị ghi đè hoặc thoái lui về quá khứ. Đối với thông điệp có $E_{msg} > E_{local}$, nút buộc phải đồng bộ hóa chuỗi trạng thái thay vì áp dụng trực tiếp, đảm bảo mọi quá trình chuyển đổi trạng thái đều phải tuần tự trải qua các Commit hợp lệ. Do OpenMLS chỉ sinh trạng thái

mới thông qua quá trình xử lý Commit hợp lệ, kết hợp với cơ chế kiểm soát vòng ngoài của lớp điều phối, chuỗi giá trị epoch cục bộ tại mọi nút luôn thỏa mãn tính chất đơn điệu không giảm.

Bất biến này không đảm bảo rằng mọi nút luôn có cùng epoch tại mọi thời điểm. Trong mạng P2P, một số nút có thể tạm thời đi sau do mất kết nối hoặc nhận thông điệp chậm. Điều bất biến đảm bảo là mỗi nút không tự rollback trạng thái MLS của chính nó.

4.4 Bất biến 2: Với cùng một view, chỉ có một Token Holder

4.4.1 Phát biểu bất biến

Tại epoch E , nếu hai nút có cùng $groupID$, cùng số epoch và cùng tập ứng viên hợp lệ, chúng phải tính ra cùng một Token Holder. Token Holder là nút duy nhất được quyền tạo Commit cho epoch đó trong view tương ứng.

Tập ứng viên hợp lệ được xác định như sau:

$$\begin{aligned} Eligible(E) = & GroupMembers(E) \cap ActiveView(E) \\ & \cap AuthorizedCommitters(E) \setminus RemovedOrSuspended(E) \end{aligned} \quad (4.1)$$

Trong đó, $GroupMembers(E)$ là tập thành viên có trong trạng thái MLS tại epoch E ; $ActiveView(E)$ là tập thành viên đang được quan sát là còn hoạt động; $AuthorizedCommitters(E)$ là tập thành viên được chính sách ứng dụng cho phép tạo Commit; và $RemovedOrSuspended(E)$ là tập thành viên tạm thời không được xét làm Token Holder.

Token Holder được tính bằng quy tắc tất định:

$$TokenHolder(E) = \operatorname{argmin}_{n \in Eligible(E)} H(groupID \parallel E \parallel n) \quad (4.2)$$

Trong đó H là hàm băm mật mã, ví dụ SHA-256.

Mệnh đề 2. Nếu hai nút có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì chúng chọn cùng một Token Holder.

Lập luận. Nếu hai nút i và j có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì đầu vào của Công thức 4.2 trên hai nút là giống nhau. Vì hàm băm là tất định, tập giá trị băm thu được trên hai nút cũng giống nhau. Do cùng áp dụng phép chọn argmin trên cùng một tập giá trị, hai nút sẽ chọn cùng một Token Holder.

Trong điều kiện các nút có cùng view, Single-Writer đảm bảo chỉ có một Token Holder, do đó hệ thống không cần thêm một vòng bỏ phiếu mạng để thống nhất ai là người tạo Commit. Mỗi nút có thể tự tính cục bộ và đi đến cùng kết quả. Các nút không phải Token Holder vẫn có thể tạo Proposal, nhưng không trực tiếp tạo Commit tại epoch đó.

4.4.2 Ý nghĩa đối với concurrent commits

Trong MLS, Commit là thao tác làm thay đổi trạng thái nhóm và đưa nhóm sang epoch mới [1]. Với DS nhất quán mạnh, DS có thể phá hòa khi hai thành viên gửi Commit cùng lúc [2]. Khi không có DS trung tâm, nếu mọi thành viên đều được tự do Commit, hệ thống dễ sinh nhiều Commit cạnh tranh tại cùng một epoch. Single-Writer giải quyết vấn đề này bằng cách giới hạn quyền Commit vào một Token Holder duy nhất trong mỗi view.

Cơ chế này không khẳng định hệ thống không bao giờ fork. Khi network partition xảy ra, các phân vùng có thể có *ActiveView* khác nhau, dẫn tới *Eligible(E)* khác nhau và Token Holder khác nhau. Vì vậy, Single-Writer không phải cơ chế chống fork tuyệt đối trong mọi điều kiện mạng. Vai trò đúng của nó là hạn chế concurrent commits trong điều kiện các nút có cùng góc nhìn, từ đó giảm đáng kể nguy cơ fork khi mạng còn liên thông.

4.5 Bất biến 3: Fork trạng thái phải phát hiện được

4.5.1 Hiện tượng fork trạng thái mật mã

Một fork trạng thái mật mã xảy ra khi hai tập nút có cùng *groupID* nhưng tiến hóa sang hai trạng thái MLS khác nhau. Trường hợp dễ gây nhầm lẫn nhất là hai nhánh đều mang số epoch $E + 1$, nhưng tree hash, transcript hash hoặc epoch authenticator khác nhau. Khi đó, dù số epoch giống nhau, hai nhánh thực chất không còn chia sẻ cùng một MLS group state.

DMLS cũng chỉ ra vấn đề tương tự: trong môi trường phi tập trung, có thể xuất hiện nhiều Commit cho cùng một số epoch, khiến số nguyên epoch không còn đủ để định danh duy nhất một trạng thái nhóm [11]. Điều này củng cố nhận định rằng trong P2P MLS, chỉ kiểm tra số epoch là chưa đủ; hệ thống cần so sánh thêm các fingerprint của trạng thái mật mã.

4.5.2 Fingerprint trạng thái

MLS định nghĩa nhiều giá trị có thể dùng để nhận diện trạng thái nhóm. Tree hash tóm tắt trạng thái ratchet tree và root tree hash được dùng trong GroupContext để xác nhận rằng các thành viên đồng ý về toàn bộ cây [1]. Epoch authenticator có thể được dùng để xác nhận rằng các client đang ở cùng epoch và cùng trạng thái

nhóm [1].

Dựa trên các giá trị này, lớp điều phối trao đổi định kỳ một bản tóm tắt trạng thái:

$$StateSummary = (groupID, E, treeHash, epochAuthenticator, commitHash)$$

Trong đó *commitHash* là định danh của Commit gần nhất theo lớp điều phối. Giá trị này không thay thế các định danh mật mã của MLS, nhưng giúp hệ thống truy vết nhanh nhánh trạng thái ở tầng ứng dụng.

Mệnh đề 3. Khi network partition xảy ra, hệ thống không đảm bảo ngăn fork tuyệt đối, nhưng có thể phát hiện sự khác biệt trạng thái mật mã thông qua việc so sánh các giá trị định danh trạng thái và phục hồi sau khi các phân vùng kết nối trở lại.

Lập luận. Giả sử hai nút i và j có cùng *groupID*. Nếu chúng cùng ở epoch E nhưng có $treeHash_i \neq treeHash_j$ hoặc $epochAuthenticator_i \neq epochAuthenticator_j$, thì hai nút không có cùng trạng thái MLS tại epoch đó. Khi đó, lớp điều phối có thể kết luận rằng đã xuất hiện fork trạng thái.

Lập luận quan trọng ở đây là fork MLS không phải là trạng thái mơ hồ ở tầng ứng dụng. Khi hai nhánh đã khác trạng thái mật mã, sự khác biệt đó để lại dấu vết trong các fingerprint như tree hash và epoch authenticator. Vì vậy, hệ thống không cần lưu toàn bộ lịch sử thông điệp để phát hiện mọi trường hợp; nó có thể phát hiện fork bằng cách so sánh các trạng thái đặc trưng.

4.6 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh

4.6.1 Vấn đề của việc gộp trạng thái

Khi hai nhánh MLS đã phân kỳ, mỗi nhánh có lịch sử Commit, ratchet tree, transcript và key schedule riêng. Việc cố gắng trộn trực tiếp khóa, cây hoặc transcript của hai nhánh để tạo ra một trạng thái chung mới là không phù hợp với mô hình của MLS. Trong MLS, trạng thái nhóm chỉ nên tiến hóa thông qua các thao tác hợp lệ như Proposal và Commit; mỗi Commit đưa nhóm sang một epoch mới và làm cho các giá trị trạng thái như ratchet tree, transcript hash, epoch secret và epoch authenticator được cập nhật theo đúng quy tắc của giao thức [1]. Nếu lớp điều phối tự tạo ra một trạng thái mật mã mới bằng cách gộp thủ công hai nhánh đã fork, trạng thái đó không còn là kết quả của một chuỗi Commit hợp lệ. Khi đó, hệ thống không còn có thể dựa trực tiếp vào các bảo đảm của MLS/OpenMLS.

Do đó, đề án đặt ra bất biến sau: khi fork đã xảy ra, hệ thống không hợp nhất trực tiếp hai trạng thái MLS. Thay vào đó, hệ thống chọn một nhánh tiếp tục và đưa các nút ở nhánh còn lại gia nhập lại nhóm để tạo ra một trạng thái MLS hợp lệ mới.

4.6.2 Quy tắc chọn nhánh

Giả sử sau khi network partition kết thúc, hệ thống quan sát được tập các nhánh:

$$\mathcal{B} = \{B_1, B_2, \dots, B_k\}$$

Mỗi nhánh B được gán với một trọng số:

$$W(B) = (E_B, C_{valid}(B), H_{commit}(B), H_{tree}(B)) \quad (4.3)$$

Trong đó E_B là epoch của nhánh, $C_{valid}(B)$ là số thành viên hợp lệ hoặc số nút quan sát được đang ủng hộ nhánh, $H_{commit}(B)$ là mã băm Commit gần nhất, và $H_{tree}(B)$ là tree hash của nhánh. Các trường hash không biểu diễn nhánh "tốt hơn" về mặt bảo mật, mà chỉ đóng vai trò phá hòa tất định khi các tiêu chí trước bằng nhau.

Nhánh thắng được chọn bằng một hàm tất định:

$$B_{win} = SelectBranch(\mathcal{B}) = \operatorname{argmax}_{B \in \mathcal{B}} W(B) \quad (4.4)$$

Việc so sánh được hiểu là so sánh từ điển theo thứ tự các thành phần trong $W(B)$.

Mệnh đề 4. Mọi phân vùng quan sát cùng một tập nhánh sẽ chọn cùng một nhánh thắng để phục hồi sau fork.

Lập luận. Nếu hai nút quan sát cùng một tập nhánh $\mathcal{B}$ và cùng sử dụng hàm $SelectBranch$, chúng sẽ chọn cùng một nhánh thắng B_{win} . Tính hội tụ ở đây đến từ tính tất định của quy tắc lựa chọn, không đến từ một máy chủ trung tâm.

Sau khi nhánh thắng được xác định, các nút thuộc nhánh thua không tiếp tục gửi thông điệp mới bằng trạng thái cũ. Chúng đóng băng trạng thái cũ ở tầng ứng dụng, sau đó gia nhập lại nhánh thắng bằng cơ chế phù hợp của MLS, chẳng hạn Welcome hoặc External Commit tùy điều kiện triển khai. RFC 9420 mô tả External

Commit như một cơ chế cho phép một thành viên bên ngoài tham gia hoặc resync vào nhóm trong những điều kiện nhất định [1].

Suy ra, sau quá trình healing, hệ thống không duy trì song song nhiều nhánh cho các thông điệp mới. Thay vào đó, các nút hội tụ về một trạng thái MLS hợp lệ được tạo ra thông qua cơ chế của MLS, không phải thông qua việc tự trộn khóa cũ.

4.6.3 Giới hạn của lập luận

Lập luận trên phụ thuộc vào hai điều kiện. Thứ nhất, các phân vùng mạng phải kết nối trở lại để thông tin về các nhánh được trao đổi. Nếu partition kéo dài vô hạn, không có cơ chế P2P nào có thể buộc các phân vùng không liên lạc với nhau phải hội tụ tức thời. Thứ hai, các nút cần dần quan sát được cùng tập nhánh hoặc ít nhất cùng nhánh có trọng số cao nhất. Trước khi thông tin hội tụ, một số nút có thể tạm thời chưa chọn cùng kết quả.

Những giới hạn này không làm mất ý nghĩa của Fork Healing. Chúng chỉ phản ánh bản chất của hệ phân tán: khi mất liên lạc, hệ thống không thể vừa cho phép các phân vùng tiếp tục hoạt động cục bộ, vừa đảm bảo mọi phân vùng luôn có cùng trạng thái tại mọi thời điểm.

4.7 Xử lý thông điệp phát sinh trong nhánh thua

Trong thời gian network partition, người dùng ở mỗi phân vùng có thể tiếp tục gửi application message. Khi partition kết thúc và một nhánh được chọn làm nhánh tiếp tục, các thông điệp phát sinh ở nhánh thua đặt ra một vấn đề riêng: chúng là dữ liệu ứng dụng hợp lệ đối với người gửi, nhưng bản mã của chúng được tạo dưới một trạng thái MLS không còn được tiếp tục sử dụng.

Vì vậy, đồ án không phát lại nguyên trạng các bản mã cũ của nhánh thua. Nếu chính sách ứng dụng cho phép phục hồi nội dung, nút chỉ phát lại các thông điệp do chính nó tạo ra và mã hóa lại nội dung dưới epoch hợp lệ của nhánh thắng. Cách làm này có ba ý nghĩa.

Thứ nhất, hệ thống không kéo dài vòng đời sử dụng của khóa cũ thuộc nhánh thua. Thứ hai, mỗi nút chỉ có thể phát lại nội dung mà nó là tác giả, tránh việc một nút tự ý đại diện cho nút khác để tái tạo thông điệp. Thứ ba, thông điệp sau khi được phát lại sẽ là application message mới, được mã hóa và xác thực trong trạng thái MLS hiện hành của nhánh thắng.

Lập luận này không khẳng định rằng mọi dữ liệu phát sinh trong partition đều có thể được phục hồi trọn vẹn. Một số thông điệp có thể bị mất nếu tác giả không còn lưu plaintext cục bộ hoặc chính sách ứng dụng không cho phép replay. Tuy nhiên, cách xử lý này giữ được ranh giới quan trọng: phục hồi dữ liệu ứng dụng

không được đánh đổi bằng việc hợp nhất hoặc tái sử dụng tùy tiện trạng thái mật mã cũ.

4.8 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit

4.8.1 Phân biệt crypto ordering và application ordering

Trong hệ thống, cần phân biệt hai loại thứ tự khác nhau. Thứ tự thứ nhất là thứ tự Commit, quyết định chuỗi trạng thái mật mã của MLS. Thứ tự này ảnh hưởng trực tiếp đến epoch, ratchet tree và key schedule. Thứ tự thứ hai là thứ tự hiển thị application message trong cùng một epoch. Thứ tự này phục vụ trải nghiệm người dùng và đồng bộ dữ liệu ứng dụng.

MLS đã có cơ chế riêng để xử lý application message bị trễ hoặc đến sai thứ tự thông qua group identifier, epoch và generation counter [1]. Tuy nhiên, trong một ứng dụng P2P, các nút vẫn cần một quy tắc ổn định để hiển thị các thông điệp đã giải mã hợp lệ theo cùng một thứ tự tương đối. Đây là vai trò của HLC trong đồ án.

4.8.2 Cơ sở của HLC

Hybrid Logical Clock kết hợp thông tin thời gian vật lý và đồng hồ logic. Theo Kulkarni và Demirbas, HLC giữ được thuộc tính một chiều của quan hệ nhân quả: nếu sự kiện e xảy ra trước sự kiện f theo quan hệ happened-before, thì timestamp HLC của e nhỏ hơn timestamp HLC của f [9].

Trong đồ án, timestamp của một application message được biểu diễn dưới dạng:

$$HLC = (L, C, NodeID) \quad (4.5)$$

Trong đó L là thành phần thời gian logic gắn với thời gian vật lý, C là bộ đếm logic dùng khi nhiều sự kiện có cùng L , và $NodeID$ là khóa phá hòa tất định.

Mệnh đề 5. HLC tạo thứ tự hiển thị tất định cho cùng một tập thông điệp đã được giải mã hợp lệ, không thay thế cơ chế ordering của Commit.

Lập luận. Khi một nút gửi application message, nó gắn HLC hiện tại vào message. Khi một nút nhận message từ nút khác, nó cập nhật HLC cục bộ dựa trên timestamp nhận được và thời gian cục bộ. Nếu message A là nguyên nhân dẫn đến message B , ví dụ người dùng trả lời sau khi đã nhận A , thì HLC của B sẽ lớn hơn HLC của A theo quan hệ từ điển. Điều này giúp hệ thống giữ được thứ tự nhân quả cơ bản trong quá trình hiển thị.

Đối với các message đồng thời, tức không có quan hệ nhân quả rõ ràng, hệ thống sử dụng so sánh từ điển trên $(L, C, NodeID)$ để tạo thứ tự hiển thị tất định. Điều

này không biến HLC thành cơ chế đồng thuận, và HLC không đảm bảo mọi nút có lịch sử giống nhau tuyệt đối. Nó chỉ đảm bảo rằng khi hai nút đã có cùng một tập message hợp lệ, chúng có thể sắp xếp tập message đó theo cùng một quy tắc.

4.8.3 Giới hạn của HLC trong đồ án

HLC không được dùng để quyết định Commit nào hợp lệ. HLC cũng không được dùng để sửa fork trạng thái MLS. Một Commit chỉ hợp lệ khi được MLS/OpenMLS xác thực và phù hợp với trạng thái epoch hiện tại. Vì vậy, HLC chỉ nằm ở tầng ứng dụng, sau các bước xác thực và giải mã, nhằm phục vụ thứ tự hiển thị và đồng bộ hội thoại.

Sự phân định này đóng vai trò thiết yếu. Nếu nhầm HLC với cơ chế ordering cho Commit, hệ thống có thể vô tình cho phép một timestamp ứng dụng ảnh hưởng đến tiến trình key schedule của MLS. Đồ án tránh điều này bằng cách tách rõ hai tầng: Commit ordering thuộc lớp điều phối MLS, còn application ordering thuộc lớp hiển thị và đồng bộ dữ liệu.

4.9 Phân tích chi phí lý thuyết

Phần này phân tích chi phí của hệ thống theo ba lớp: chi phí mật mã MLS, chi phí điều phối P2P và chi phí triển khai. Việc tách ba lớp là cần thiết vì độ phức tạp thuật toán của MLS không phản ánh đầy đủ độ trễ end-to-end của một hệ thống thực tế có gRPC, lưu trữ cục bộ và truyền thông P2P.

4.9.1 Chi phí gửi application message

Trong mô hình mã hóa pairwise, người gửi thường phải tạo hoặc bọc khóa riêng cho từng người nhận. Với nhóm có N thành viên, chi phí này có xu hướng tăng tuyến tính theo số người nhận.

Với MLS, application message được bảo vệ bằng khóa dẫn xuất từ trạng thái nhóm tại epoch hiện tại. Người gửi không cần tạo một bản mã độc lập cho từng thành viên. Vì vậy, xét riêng thao tác mã hóa nội dung ứng dụng, chi phí không tăng tuyến tính theo số lượng thành viên như mô hình pairwise. Đây là một trong những lý do MLS phù hợp hơn cho truyền thông nhóm quy mô lớn [1].

Tuy nhiên, trong hệ thống P2P, chi phí toàn trình còn phụ thuộc vào lớp mạng. Nếu một thông điệp phải được truyền trực tiếp tới nhiều peer, chi phí mạng có thể tăng theo số lượng peer hoặc theo cơ chế lan truyền của overlay. Do đó, cần phân biệt giữa chi phí mật mã của MLS và chi phí phân phối message của hệ thống P2P.

4.9.2 Chi phí cập nhật thành viên và rekey

Các thao tác thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa đều được biểu diễn thông qua Proposal và Commit. Ở mức TreeKEM, ratchet tree cho phép

cập nhật khóa hiệu quả hơn so với việc mã hóa riêng cho từng thành viên; RFC 9420 mô tả rằng việc mã hóa tới các subtree giúp giảm chi phí từ tuyến tính xuống theo chiều cao cây trong một số thao tác, ví dụ loại bỏ một thành viên khỏi nhóm [1].

Trong triển khai thực tế, chi phí Commit không chỉ gồm phần mật mã lõi. Nó còn bao gồm chi phí tạo Proposal, tạo Commit, xử lý Welcome hoặc External Commit, lưu trạng thái mới, truyền message qua mạng P2P và đồng bộ các nút nhận. Vì vậy, ở Chương 5, việc đánh giá cần tách hai loại số liệu: chi phí MLS core và chi phí end-to-end của toàn hệ thống.

4.9.3 Chi phí của Single-Writer

Cơ chế Single-Writer không yêu cầu một vòng đồng thuận mạng riêng để bầu Token Holder trong điều kiện các nút có cùng view. Mỗi nút tự tính Token Holder bằng cách duyệt qua tập $Eligible(E)$ và áp dụng hàm băm cho từng ứng viên. Nếu số thành viên hợp lệ là N , chi phí tính toán cục bộ của bước chọn Token Holder là $O(N)$ trong cách cài đặt trực tiếp.

Điểm quan trọng là chi phí này là chi phí cục bộ, không phải chi phí nhiều vòng mạng. Trong bối cảnh P2P, tránh thêm các vòng đồng thuận cho mỗi Commit là một lợi thế quan trọng, vì độ trễ mạng và peer churn có thể làm các giao thức đồng thuận trở nên nặng hơn nhu cầu thực tế của ứng dụng.

4.9.4 Chi phí của Epoch Checks và Fork Detection

Epoch Checks có chi phí nhỏ vì mỗi message chỉ cần so sánh một số trường metadata như *groupID*, epoch và loại thông điệp trước khi chuyển xuống MLS. Chi phí này có thể xem là $O(1)$ trên mỗi message nếu không tính phần xử lý MLS bên dưới.

Fork Detection yêu cầu các nút trao đổi StateSummary chứa epoch, tree hash, epoch authenticator và commit hash. Kích thước của mỗi StateSummary là nhỏ so với toàn bộ trạng thái MLS. Do đó, chi phí mỗi lần trao đổi tóm tắt trạng thái có thể xem là $O(1)$ theo kích thước nhóm, mặc dù tổng chi phí mạng phụ thuộc vào tần suất heartbeat và số lượng peer mà mỗi nút duy trì kết nối.

4.9.5 Chi phí của Fork Healing

Fork Healing phát sinh khi hệ thống đã có phân nhánh trạng thái. Chi phí của quá trình này gồm ba phần: phát hiện nhánh, chọn nhánh, và đưa các nút ở nhánh thua gia nhập lại. Phần chọn nhánh có chi phí phụ thuộc vào số nhánh quan sát được k , thường nhỏ hơn nhiều so với số thành viên nhóm. Phần tốn kém hơn là quá trình gia nhập lại, vì nó cần thực hiện các thao tác MLS như Welcome hoặc

External Commit và đồng bộ trạng thái mới.

Về lý thuyết, Fork Healing là chi phí bất thường, không phải chi phí trên mọi message. Nó chỉ xuất hiện khi có partition hoặc fork. Do đó, đề án chấp nhận chi phí này để đổi lấy khả năng duy trì hoạt động cục bộ trong thời gian mạng bị chia cắt và khả năng hội tụ sau khi kết nối được khôi phục.

4.9.6 Ảnh hưởng của kiến trúc sidecar phi trạng thái

Trong kiến trúc triển khai của đề án, Go Host lưu snapshot trạng thái nhóm trong SQLite và truyền GroupState sang Rust sidecar khi cần thực hiện thao tác MLS. Rust xử lý snapshot này bằng OpenMLS rồi trả lại trạng thái mới để Go Host lưu bền vững. Cách thiết kế này không làm thay đổi độ phức tạp lý thuyết của TreeKEM, nhưng làm phát sinh thêm chi phí tuần tự hóa, truyền dữ liệu qua gRPC và ghi lại trạng thái khi quy mô nhóm tăng.

Do đó, khi đánh giá hiệu năng, cần tránh nhầm lẫn giữa hai tầng. Nếu đo MLS core, kết quả phản ánh chi phí mật mã và xử lý trạng thái trong OpenMLS. Nếu đo end-to-end, kết quả phản ánh thêm chi phí IPC, lưu trữ, truyền mạng và điều phối. Trong phạm vi đề án, các kết quả thực nghiệm chủ yếu dùng để kiểm chứng tính đúng đắn, khả năng hội tụ và thuộc tính bảo mật; tối ưu hóa chi phí IPC và persistence được xem là hướng phát triển tiếp theo.

4.10 Tổng hợp các bất biến và cơ chế bảo toàn

Bảng 4.2 tổng hợp các bất biến chính được phân tích trong chương này và cơ chế tương ứng trong thiết kế của đề án.

Bất biến / Thuộc tính	Cơ chế bảo toàn	Ý nghĩa
Epoch cục bộ không thoái lui	Epoch Checks	Ngăn nút áp dụng thông điệp lên trạng thái MLS cũ hoặc không tương thích.
Một Token Holder trong cùng view	Single-Writer per Epoch	Giảm concurrent commits khi các nút có cùng góc nhìn về nhóm.
Fork trạng thái phát hiện được	StateSummary, tree hash, epoch authenticator	Nhận diện trường hợp cùng nhóm nhưng khác trạng thái mật mã.
Không gộp trực tiếp trạng thái MLS đã fork	Fork Healing qua chọn nhánh và gia nhập lại	Tránh tự tạo trạng thái mật mã ngoài mô hình MLS.
Thứ tự hiển thị không quyết định Commit	HLC Message Ordering	Tách application ordering khỏi crypto ordering.
Chi phí điều phối không thay thế chi phí MLS core	Phân tách Go Coordination và Rust OpenMLS	Cho phép đánh giá riêng chi phí mật mã và chi phí hệ thống.

Bảng 4.2: Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn

Qua các bất biến trên, có thể thấy đóng góp cốt lõi của đề án không nằm ở việc thay đổi MLS, mà nằm ở việc xây dựng một lớp điều phối giúp MLS thích nghi với môi trường P2P. Lớp này thay thế một phần vai trò ordering thường do DS đảm nhiệm, đồng thời vẫn giữ ranh giới rõ ràng với phần mật mã chuẩn.

4.11 Giới hạn lý thuyết của phương pháp

Phương pháp đề xuất không phải là một giao thức đồng thuận mạnh. Nó không đảm bảo rằng mọi phân vùng mạng luôn nhìn thấy cùng một trạng thái tại mọi thời điểm. Khi network partition xảy ra, mỗi phân vùng có thể tiếp tục hoạt động cục bộ và tiến hóa trạng thái riêng. Đây là đánh đổi có chủ đích để phù hợp với mục tiêu P2P và Local-First của hệ thống.

Single-Writer chỉ đảm bảo một Token Holder duy nhất khi các nút có cùng view. Nếu các phân vùng có *ActiveView* khác nhau, chúng có thể chọn ra Token Holder khác nhau. Trong trường hợp này, tính đúng đắn của hệ thống không nằm ở việc ngăn fork tuyệt đối, mà nằm ở khả năng phát hiện fork và phục hồi sau đó.

Fork Healing cũng phụ thuộc vào việc các phân vùng cuối cùng kết nối lại. Nếu các phân vùng không bao giờ trao đổi được StateSummary, hệ thống không có đủ thông tin để hội tụ. Đây là giới hạn tự nhiên của mọi cơ chế phục hồi trong hệ phân tán bất đồng bộ.

Cuối cùng, HLC chỉ hỗ trợ thứ tự hiển thị application message. HLC không thay thế kiểm tra epoch, không thay thế xác thực của MLS, và không được dùng để quyết định Commit nào hợp lệ. Nhờ giới hạn rõ vai trò như vậy, hệ thống tránh việc để một cơ chế ordering ở tầng ứng dụng can thiệp sai vào tiến trình mật mã của MLS.

Những giới hạn trên không làm suy yếu đóng góp của đề án. Ngược lại, chúng làm rõ phạm vi đúng của phương pháp: đề án không cố biến P2P MLS thành một hệ thống đồng thuận toàn cục nặng nề, mà xây dựng một lớp điều phối nhẹ hơn, phù hợp với môi trường P2P, có khả năng giảm fork trong điều kiện bình thường và phục hồi khi fork xảy ra.

4.12 Kết chương

Chương này đã phân tích các bất biến lý thuyết quan trọng của lớp điều phối phi tập trung cho MLS trên mạng P2P. Các bất biến chính bao gồm: epoch cục bộ không được thoái lui, trong cùng một view chỉ có một Token Holder, fork trạng thái phải phát hiện được, hai trạng thái MLS đã phân nhánh không được gộp trực tiếp, và HLC chỉ dùng để sắp xếp application message chứ không quyết định Commit.

Các lập luận trong chương cho thấy lớp điều phối đề xuất không thay đổi lõi mật

mã của MLS, mà bổ sung các cơ chế cần thiết để vận hành MLS trong môi trường không có DS trung tâm. Single-Writer giúp giảm concurrent commits khi các nút có cùng góc nhìn; Epoch Checks bảo vệ client khỏi việc xử lý thông điệp sai trạng thái; Fork Healing giúp hệ thống hội tụ lại sau network partition; và HLC tạo thứ tự hiển thị ổn định hơn cho application message.

Từ góc độ lý thuyết, phương pháp của đề án là hợp lý vì nó tách rõ hai trách nhiệm: MLS/OpenMLS chịu trách nhiệm bảo mật mật mã, còn lớp điều phối chịu trách nhiệm ordering, phát hiện fork và phục hồi trạng thái ở tầng hệ thống. Những phân tích này là cơ sở để Chương 5 tiếp tục kiểm chứng bằng thực nghiệm về hiệu năng, độ trễ và khả năng phục hồi của nguyên mẫu triển khai.

CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM

Tổng quan chương

Chương này trình bày quá trình đánh giá thực nghiệm hệ thống truyền thông phi tập trung sử dụng MLS trên mạng P2P. Mục tiêu của chương không chỉ là đo hiệu năng thuần túy, mà quan trọng hơn là kiểm chứng các thuộc tính cốt lõi đã được đề xuất ở các chương trước: tính an toàn của cơ chế Single-Writer, khả năng hội tụ epoch và TreeHash sau phân mảnh mạng, tính đúng đắn của cơ chế Fork Healing, khả năng sắp xếp thông điệp nhất quán bằng Hybrid Logical Clock, và việc bảo toàn các thuộc tính bảo mật của MLS khi tích hợp vào kiến trúc Go-Rust sidecar.

Các kết quả trong chương được tổng hợp từ các bài kiểm thử tự động trong mã nguồn, các tệp dữ liệu đo đạc trong thư mục `evaluation/data`, và các biểu đồ được sinh từ những dữ liệu này. Cần nhấn mạnh rằng hệ thống đánh giá được chia thành nhiều tầng: một số bài kiểm thử sử dụng mạng giả lập và MLS giả lập để cô lập logic điều phối; một số bài kiểm thử khác sử dụng Rust sidecar thật để kiểm chứng thuộc tính mật mã. Cách phân tầng này giúp tách bạch rõ ràng giữa kiểm chứng giao thức điều phối và kiểm chứng chi phí mật mã của OpenMLS.

5.1 Mục tiêu và phạm vi đánh giá

Hệ thống được đánh giá theo ba nhóm mục tiêu chính.

Thứ nhất, nhóm đánh giá tính đúng đắn tập trung vào việc trả lời câu hỏi: trong điều kiện mạng P2P có thể mất kết nối, chia cắt, trễ gói hoặc nhận thông điệp không theo thứ tự, các nút có còn hội tụ về cùng một trạng thái MLS hay không. Với nhóm mục tiêu này, các đại lượng quan trọng là epoch cuối cùng, TreeHash cuối cùng, số lượng Commit hợp lệ trong mỗi epoch, và việc phát hiện các nhánh phân kỳ.

Thứ hai, nhóm đánh giá hiệu quả điều phối tập trung vào chi phí mà lớp Coordination thêm vào phía trên MLS. Vì MLS vốn giả định có Delivery Service để tuần tự hóa Commit, hệ thống đề xuất thay thế vai trò này bằng cơ chế Single-Writer phi tập trung. Do đó, cần đo xem cơ chế gom Proposal, bầu Token Holder và phát tán Commit có giúp giảm xung đột hay không, đặc biệt khi nhiều nút cùng gửi Proposal trong một khoảng thời gian ngắn.

Thứ ba, nhóm đánh giá mật mã và bảo mật tập trung vào việc kiểm chứng rằng kiến trúc sidecar không làm suy giảm các thuộc tính bảo mật của MLS. Bài kiểm thử quan trọng nhất trong nhóm này là kiểm chứng Forward Secrecy bằng Rust sidecar thật: một thành viên đã bị loại khỏi nhóm không được phép giải mã thông điệp ở các epoch sau.

Tiêu chí	Ý nghĩa đánh giá
Hội tụ epoch	Tất cả các nút hợp lệ phải kết thúc ở cùng một epoch sau khi mạng ổn định trở lại.
Hội tụ TreeHash	Tất cả các nút hợp lệ phải có cùng mã băm cây MLS, chứng minh trạng thái mật mã đã hội tụ.
Single-Writer Safety	Không xuất hiện hai Commit hợp lệ cạnh tranh trong cùng một epoch.
Hiệu quả gom Proposal	Nhiều Proposal đồng thời có thể được gom vào một Commit thay vì tạo nhiều epoch liên tiếp.
Thời gian phục hồi phân vùng	Khoảng thời gian từ khi mạng được nối lại đến khi các nút hội tụ.
Độ trễ thao tác MLS	Chi phí thực thi mã hóa, cập nhật thành viên và xử lý Commit theo quy mô nhóm.
Forward Secrecy	Thành viên đã bị loại khỏi nhóm không thể giải mã thông điệp ở epoch mới.

Bảng 5.1: Các tiêu chí đánh giá thực nghiệm

5.2 Môi trường và phương pháp thí nghiệm

Các thí nghiệm được thực hiện trực tiếp trên mã nguồn của hệ thống. Thành phần điều phối được viết bằng Go, thành phần mật mã MLS được triển khai bằng Rust/OpenMLS và được Go khởi chạy dưới dạng sidecar thông qua gRPC trên localhost. Đối với các kiểm thử cần cô lập logic giao thức, hệ thống sử dụng FakeNetwork, FakeClock và MockMLSEngine. Đối với các kiểm thử bảo mật mật mã, hệ thống sử dụng Rust sidecar thật để tránh kết luận thiếu chính xác từ các mô-đun giả lập.

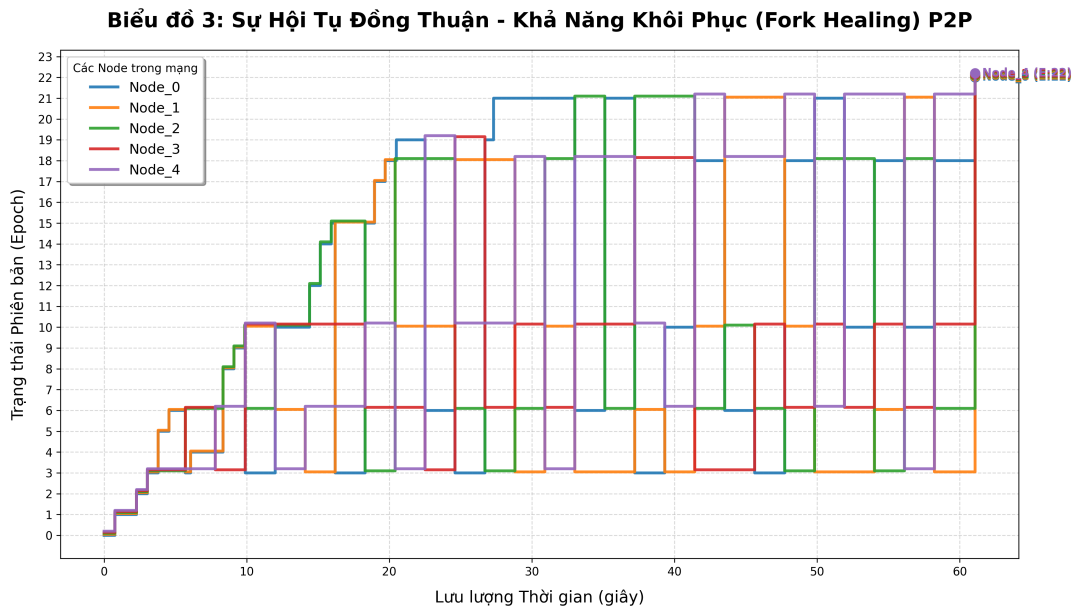
Bộ dữ liệu và biểu đồ đánh giá được tổ chức trong thư mục `evaluation`. Một số tệp quan trọng gồm: `concurrency_metrics.csv`, `epoch_convergence_metrics.csv`, `partition_recovery_metrics.csv`, `mls_optimization_benchmark.csv`, và `chaos_metrics.csv`. Trong chương này, dữ liệu benchmark MLS chỉ sử dụng hai đường đo chính là `pairwise_baseline` và `current_full_blob_mls_encrypt`. Các biểu đồ tương ứng được sinh bằng các script Python trong cùng thư mục.

Phạm vi đánh giá có hai giới hạn cần nêu rõ. Thứ nhất, các bài chaos test dùng mạng giả lập nhằm kiểm chứng tính đúng đắn của thuật toán điều phối, không đại diện hoàn toàn cho độ trễ mạng libp2p thực tế. Thứ hai, các benchmark mật mã phản ánh chi phí xử lý tại sidecar và chi phí tuần tự hóa trạng thái, không phải độ trễ đầu cuối mà người dùng cảm nhận trong toàn bộ ứng dụng.

5.3 Đánh giá hội tụ trong điều kiện mạng hỗn loạn

Bài kiểm thử quan trọng nhất của lớp điều phối là `\texttt{TestIntegration_Chaos_Convergence}`. Thí nghiệm tạo một cụm gồm 5 nút cùng tham gia một nhóm MLS. Trong quá trình chạy, một tiến trình thử nghiệm liên tục chia mạng thành hai phân vùng, duy trì trạng thái phân vùng trong khoảng 600 ms, sau đó nối mạng lại. Đồng thời, các nút vẫn tiếp tục gửi thông điệp và phát sinh thao tác thay đổi thành viên.

Kết quả được ghi vào `chaos_metrics.csv`, gồm thời điểm đo, định danh nút, epoch hiện tại và TreeHash rút gọn. Hình 5.1 minh họa quá trình các nút có thể tạm thời phân kỳ trong lúc mạng bị chia cắt, sau đó hội tụ về cùng epoch và TreeHash khi mạng được nối lại.



Hình 5.1: Hội tụ epoch của các nút trong chaos test

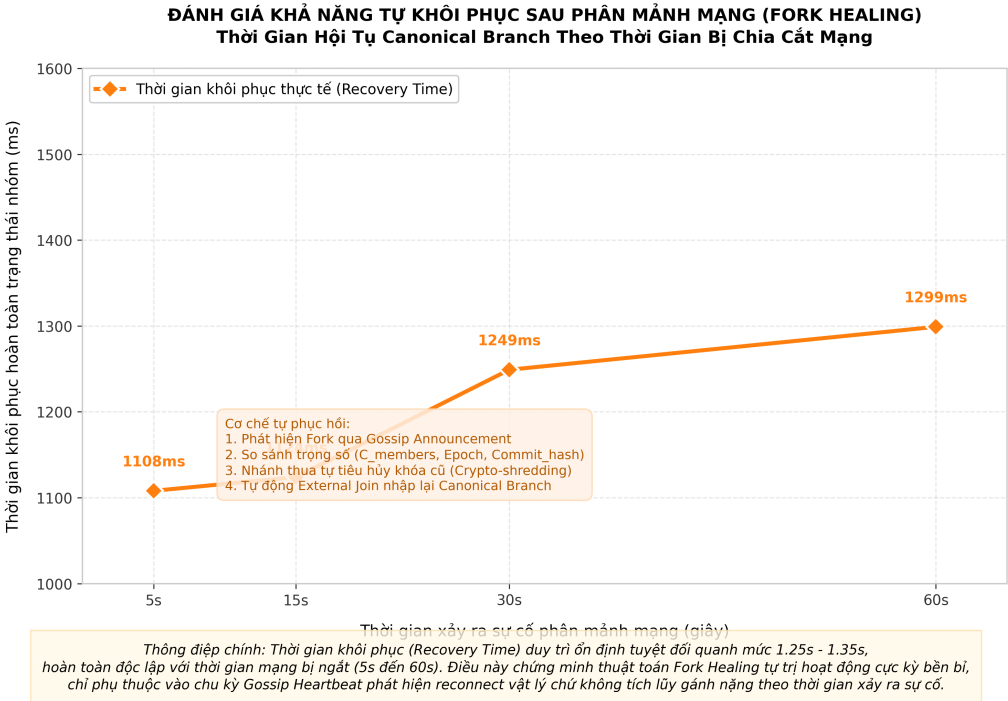
Kết quả cuối cùng cho thấy toàn bộ các nút hợp lệ hội tụ về cùng một epoch và cùng một TreeHash. Điều này xác nhận rằng cơ chế Fork Healing hoạt động đúng ở mức giao thức: khi phát hiện TreeHash khác nhau, các nút so sánh trọng số nhánh, chọn nhánh thắng, và nhánh thua thực hiện External Join để quay về trạng thái thống nhất. Quan trọng hơn, quá trình này không cần một máy chủ trung tâm đóng vai trò Delivery Service.

5.4 Đánh giá khả năng phục hồi sau phân vùng mạng

Để đánh giá riêng thời gian phục hồi, hệ thống đo thời gian từ khi hai phân vùng được nối lại cho đến khi các nút hội tụ. Dữ liệu tổng hợp (thể hiện chi tiết tại Bảng 5.2 và minh họa ở Hình 5.2) cho thấy thời gian phục hồi dao động quanh mức xấp xỉ 1,1–1,2 giây trong các kịch bản phân vùng kéo dài từ 5 giây đến 60 giây.

Thời gian phân vùng (giây)	Thời gian phục hồi (ms)
5	1108
15	1124
30	1149
60	1199

Bảng 5.2: Thời gian phục hồi sau phân vùng mạng



Hình 5.2: Thời gian phục hồi khi độ dài phân vùng thay đổi

Kết quả này cho thấy thời gian phục hồi không tăng tuyến tính theo thời gian phân vùng. Đây là đặc điểm hợp lý của thiết kế: chi phí phục hồi chủ yếu nằm ở bước phát hiện phân kỳ, trao đổi GroupInfo, thực hiện External Join và phát lại thông điệp cục bộ cần thiết; nó không phụ thuộc trực tiếp vào việc phân vùng đã kéo dài bao lâu. Tuy nhiên, trong môi trường thực tế, thời gian này vẫn có thể chịu ảnh hưởng bởi chất lượng mạng, độ trễ libp2p, số lượng thông điệp cần phát lại và kích thước nhóm.

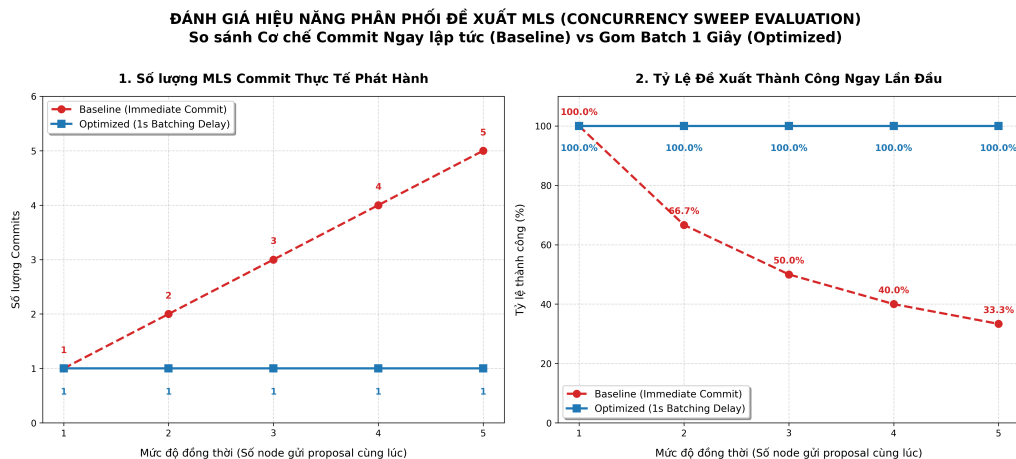
5.5 Đánh giá cơ chế Single-Writer và gom Proposal

Một rủi ro lớn khi đưa MLS lên mạng P2P là nhiều nút có thể cùng tạo Commit cho cùng một epoch. Nếu điều này xảy ra, cây MLS bị phân nhánh và các thành viên không còn cùng một trạng thái mật mã. Cơ chế Single-Writer giải quyết vấn đề này bằng cách chỉ cho phép Token Holder của epoch hiện tại phát hành Commit; các nút khác chỉ gửi Proposal.

Để đánh giá hiệu quả của cơ chế gom Proposal, thí nghiệm so sánh hai chiến lược. Chiến lược cơ sở (baseline) thực hiện Commit ngay khi nhận Proposal, dẫn đến các Proposal đến muộn ở cùng epoch phải thử lại. Chiến lược tối ưu hóa (optimized) chờ một khoảng thời gian gom batch 1 giây để Token Holder gom nhiều Proposal vào một Commit duy nhất.

Đồng thời	Baseline			Optimized		
	Proposal	Commit	Tỷ lệ thành công	Proposal	Commit	Tỷ lệ thành công
1	1	1	1,00	1	1	1,00
2	3	2	0,67	2	1	1,00
3	6	3	0,50	3	1	1,00
4	10	4	0,40	4	1	1,00
5	15	5	0,33	5	1	1,00

Bảng 5.3: So sánh baseline và cơ chế gom Proposal



Hình 5.3: Hiệu quả của cơ chế gom Proposal khi mức đồng thời tăng

Bảng 5.3 và Hình 5.3 cho thấy khi mức đồng thời tăng, chiến lược cơ sở cần nhiều Proposal hơn do phải thử lại sau khi epoch đã thay đổi. Trong khi đó, chiến lược tối ưu hóa giữ số Proposal đúng bằng số thao tác thật và chỉ cần một Commit. Kết quả này khẳng định vai trò của Single-Writer không chỉ ở tính đúng đắn, mà còn ở hiệu quả vận hành: thay vì giải quyết xung đột sau khi đã xảy ra, hệ thống ngăn xung đột Commit ngay từ đầu.

5.6 Đánh giá khả năng mở rộng của thao tác thành viên

Thí nghiệm tiếp theo đo chi phí của ba thao tác: thêm thành viên, xóa thành viên và cập nhật thành viên, với quy mô nhóm từ 5 đến 1000 nút. Dữ liệu đánh giá chi tiết được trình bày tại Bảng 5.4.

Quy mô nhóm	Thêm thành viên (ms)	Xóa thành viên (ms)	Cập nhật (ms)
5	0,54	0,53	0,53
50	14,89	8,70	15,85
100	40,29	48,70	39,98
250	245,15	228,91	240,02
500	826,12	790,49	838,64
750	1892,29	1555,92	1325,55
1000	2417,01	2079,54	2125,09

Bảng 5.4: Chi phí thao tác thành viên theo quy mô nhóm

Kết quả cho thấy chi phí tăng rõ rệt khi số lượng thành viên tăng. Điều này phù hợp với đặc điểm triển khai hiện tại: mỗi thao tác thay đổi thành viên không chỉ cập nhật logic điều phối, mà còn kéo theo việc xử lý trạng thái nhóm và phát tán Commit đến các nút liên quan. Với quy mô 1000 nút, thời gian xử lý ở mức vài giây trong môi trường mô phỏng. Vì vậy, kết quả này không nên được hiểu là giới hạn lý thuyết của MLS, mà là chi phí thực tế của pipeline triển khai hiện tại khi trạng thái nhóm được xử lý theo mô hình full-state.

5.7 Đánh giá chi phí mật mã MLS

Để đánh giá phần mật mã, hệ thống so sánh hai hướng tiếp cận: mã hóa pairwise truyền thống và đường MLS full-blob hiện tại của nguyên mẫu. Kết quả đo đạc được thể hiện tại Bảng 5.5. Mỗi điểm đo sử dụng 100 mẫu với payload 1024 byte.

Kết quả cho thấy MLS hiệu quả hơn rõ rệt so với mã hóa pairwise khi số lượng thành viên tăng. Với 4096 nút, pairwise baseline có trung vị khoảng 955 ms, trong khi full-blob MLS khoảng 79 ms. Điều này thể hiện lợi thế cơ bản của MLS: người gửi không phải mã hóa riêng cho từng người nhận.

Tuy nhiên, kết quả cũng chỉ ra điểm nghẽn của kiến trúc hiện tại. Đường full-blob MLS vẫn phải truyền và tuần tự hóa toàn bộ trạng thái nhóm giữa Go và Rust

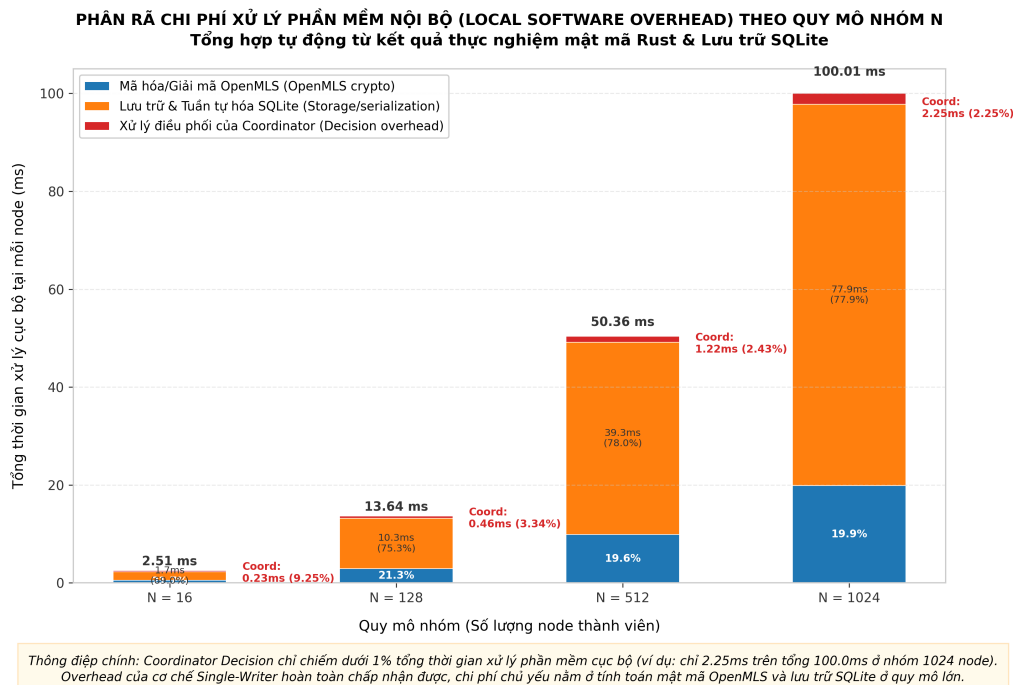
Số nút	Pairwise (ms)	Full-blob MLS (ms)
16	3,62	0,55
256	59,62	5,41
1024	238,75	19,86
4096	955,18	78,95

Bảng 5.5: So sánh độ trễ mật mã giữa các mô hình

sidecar. Vì vậy, mặc dù MLS tránh được chi phí mã hóa riêng cho từng người nhận, nguyên mẫu vẫn có overhead triển khai khi kích thước GroupState tăng. Đây là hạn chế cần được xem xét trong các hướng tối ưu hóa IPC và persistence ở các nghiên cứu tiếp theo.

5.8 Chi phí phụ trợ của lớp điều phối

Bên cạnh chi phí mật mã, lớp điều phối cũng tạo thêm chi phí cho việc bầu Token Holder, kiểm tra epoch, lưu trữ envelope, cập nhật ActiveView và phát hiện fork. Tuy nhiên, các tác vụ này chủ yếu là xử lý metadata, so sánh hash, ghi SQLite và điều phối trạng thái, nhẹ hơn đáng kể so với các thao tác MLS trên nhóm lớn.



Hình 5.4: Phân rã chi phí phụ trợ của lớp điều phối

Từ góc độ kiến trúc, dữ liệu được trình bày ở Hình 5.4 là một kết quả tích cực. Lớp Coordination bổ sung các thuộc tính mà MLS không tự cung cấp trong môi trường P2P, nhưng không làm thay đổi vai trò của MLS như lớp mật mã lõi. Go

chịu trách nhiệm điều phối, lưu trữ, kiểm tra epoch và khôi phục phân nhánh; Rust sidecar vẫn chỉ thực hiện các thao tác MLS thuần túy.

5.9 Kiểm chứng Forward Secrecy bằng Rust sidecar thật

Một kiểm chứng bảo mật quan trọng là `\texttt{TestBusinessPl_E2E_RealSidecar_ForwardSecrecy}`. Khác với các kiểm thử điều phối dùng mock, bài kiểm thử này sử dụng Rust sidecar thật. Kịch bản gồm các bước: Alice tạo nhóm, Bob tham gia nhóm, Alice loại Bob khỏi nhóm, sau đó Alice gửi một thông điệp mới ở epoch sau khi Bob đã bị loại.

Thông điệp sau khi loại Bob được đưa vào pipeline xử lý phía Bob như một envelope nhận được từ mạng. Nếu hệ thống sai, Bob có thể giải mã và lưu plaintext vào cơ sở dữ liệu cục bộ. Kết quả kiểm thử cho thấy Bob không giải mã được thông điệp này và không có plaintext nào được lưu lại. Điều này xác nhận rằng việc bọc MLS bằng lớp Coordination không phá vỡ Forward Secrecy của MLS.

Kết quả này có ý nghĩa đặc biệt quan trọng đối với mô hình zero-trust. Ngay cả khi một nút từng là thành viên hợp lệ của nhóm, sau khi bị xóa khỏi cây MLS, nút đó không thể đọc các thông điệp tương lai. Lớp điều phối chỉ quyết định thứ tự và trạng thái nhóm; quyền giải mã cuối cùng vẫn được kiểm soát bởi trạng thái mật mã MLS.

5.10 Nhận xét và giới hạn đánh giá

Các kết quả thực nghiệm cho thấy thiết kế đạt được ba mục tiêu chính. Thứ nhất, cơ chế Single-Writer loại bỏ xung đột Commit trong một epoch và giúp hệ thống hội tụ ổn định. Thứ hai, Fork Healing đưa các nhánh phân kỳ quay về cùng trạng thái mà không cần Delivery Service trung tâm. Thứ ba, kiến trúc Go–Rust sidecar bảo toàn thuộc tính bảo mật của MLS, trong đó có Forward Secrecy.

Tuy nhiên, vẫn tồn tại một số giới hạn. Các chaos test hiện tại chủ yếu dùng mạng giả lập nên chưa phản ánh đầy đủ độ trễ, mất gói và hành vi NAT của mạng thực tế. Các benchmark MLS cho thấy xu hướng chi phí rõ ràng, nhưng chưa phải đánh giá hiệu năng đầu cuối của toàn bộ ứng dụng desktop. Ngoài ra, mô hình full-blob hiện tại có chi phí serialization lớn khi nhóm rất đông thành viên; đây là điểm cần tối ưu nếu hệ thống được triển khai ở quy mô doanh nghiệp lớn.

Kết chương

Chương này đã trình bày quá trình đánh giá thực nghiệm hệ thống theo cả hai hướng: tính đúng đắn của giao thức phối hợp phi tập trung và hiệu năng của lớp mật mã MLS. Kết quả chaos test chứng minh các nút có thể hội tụ về cùng epoch và TreeHash sau phân vùng mạng. Kết quả concurrency test cho thấy cơ chế gom

Proposal giúp giảm số Commit và loại bỏ retry không cần thiết. Các benchmark mật mã cho thấy MLS có ưu thế rõ rệt so với mã hóa pairwise, đồng thời chỉ ra nút thắt hiện tại nằm ở chi phí quản lý trạng thái full-blob. Cuối cùng, kiểm thử với Rust sidecar thật xác nhận Forward Secrecy được bảo toàn sau khi loại thành viên khỏi nhóm.

Những kết quả này củng cố tính khả thi của hướng tiếp cận đã đề xuất: thay thế Delivery Service tập trung của MLS bằng một lớp điều phối phi tập trung, trong khi vẫn giữ MLS làm lõi mật mã an toàn. Chương tiếp theo sẽ tổng kết các đóng góp chính của đề án và trình bày các hướng phát triển tiếp theo.

CHƯƠNG 6. KẾT LUẬN

6.1 Kết luận

Đồ án tốt nghiệp này được thực hiện với hai mục tiêu chính: nghiên cứu một cơ chế điều phối phi tập trung cho giao thức Messaging Layer Security (MLS) trong môi trường mạng ngang hàng và xây dựng một nguyên mẫu ứng dụng để hiện thực hóa cơ chế đó. Trên cơ sở các mục tiêu đã đặt ra, đồ án đã tập trung giải quyết bài toán quan trọng là duy trì tính nhất quán trạng thái của MLS trong bối cảnh không tồn tại hệ thống Delivery Service tập trung như giả định ban đầu của tài liệu đặc tả RFC 9420 [1].

Về mặt nghiên cứu, đồ án đã đề xuất một lớp điều phối bao quanh MLS gồm bốn thành phần chính: cơ chế Single-Writer nhằm hạn chế khả năng xuất hiện các lệnh Commit đồng thời (concurrent commits) trong cùng một epoch; cơ chế kiểm tra tính nhất quán Epoch để xử lý các thông điệp cũ hoặc vượt trước trạng thái cục bộ; cơ chế Fork Healing để phát hiện và xử lý tình huống phân nhánh trạng thái sau khi mạng bị phân mảnh; và cơ chế Hybrid Logical Clock để hỗ trợ sắp xếp thứ tự hiển thị thông điệp ứng dụng một cách nhất quán giữa các nút. Đây là đóng góp cốt lõi của đồ án ở khía cạnh thiết kế giao thức, bởi cơ chế này cung cấp một hướng tiếp cận khả thi để điều chỉnh MLS tương thích với môi trường mạng P2P bất đồng bộ.

Về mặt hiện thực hệ thống, đồ án đã xây dựng thành công một nguyên mẫu ứng dụng hoạt động trên kiến trúc hai tầng gồm Go Host và Rust Sidecar. Trong đó, phía Go đảm nhiệm điều phối, truyền thông mạng ngang hàng, giao tiếp với giao diện người dùng và lưu trữ dữ liệu cục bộ; phía Rust phụ trách các thao tác mật mã MLS dựa trên thư viện OpenMLS. Trên nền tảng đó, hệ thống đã cung cấp một số chức năng quan trọng như khởi tạo định danh, cấp quyền tham gia thông qua thẻ mời (InvitationToken), tạo nhóm, trao đổi thông điệp mã hóa đầu cuối, thêm và xóa thành viên, đồng bộ lại dữ liệu khi thiết bị kết nối trở lại, sao lưu và khôi phục danh tính, cũng như truyền tải tệp tin theo cơ chế sử dụng khóa dẫn xuất từ MLS Exporter. Các kết quả này chứng minh rằng mô hình kết hợp giữa giao thức điều phối đề xuất và nền tảng MLS hoàn toàn có thể được triển khai thành một hệ thống phần mềm thực tiễn.

Về mặt đánh giá, các kết quả thực nghiệm bước đầu xác nhận giải pháp đề xuất có khả năng vận hành phù hợp với mục tiêu thiết kế. Các bài kiểm thử về tính hội tụ chỉ ra rằng hệ thống có khả năng đưa các nút trở về cùng một epoch và cùng mã băm TreeHash sau khi xảy ra phân mảnh mạng. Đánh giá đối với các Proposal đồng

thời cho thấy cơ chế gom khối (batching) làm giảm đáng kể số lượng Commit phát sinh trong tình huống cạnh tranh. Bên cạnh đó, các thử nghiệm với Rust sidecar thực tế xác nhận rằng sau khi một thành viên bị loại khỏi nhóm, thành viên đó không còn khả năng giải mã các thông điệp ở epoch tiếp theo. Mặc dù những kết quả này chưa đủ để kiểm chứng toàn diện mọi thuộc tính an toàn của hệ thống trong môi trường triển khai thực tế, chúng đóng vai trò là cơ sở thực nghiệm quan trọng để khẳng định tính hợp lý và tiềm năng phát triển của hướng tiếp cận.

Bên cạnh các kết quả đạt được, nghiên cứu này vẫn còn một số hạn chế nhất định. Thứ nhất, phần lớn các bài kiểm thử hiện nay được thực hiện trên môi trường mô phỏng (sử dụng FakeNetwork hoặc MockMLSEngine), do đó mức độ phản ánh điều kiện vận hành thực tế của mạng libp2p trên quy mô lớn vẫn còn giới hạn. Thứ hai, kiến trúc sidecar hiện tại áp dụng phương pháp trao đổi toàn bộ trạng thái nhóm (full-state) giữa Go và Rust trong nhiều thao tác, dẫn đến chi phí tuần tự hóa và giải tuần tự hóa gia tăng đáng kể khi quy mô nhóm mở rộng. Thứ ba, mặc dù cơ chế Fork Healing đã được thiết kế và kiểm chứng ở mức logic, việc đánh giá toàn diện trên các kịch bản thực tế phức tạp (nhiều tiến trình thực, nhiều phân vùng mạng, thời gian vận hành dài) vẫn cần được bổ sung. Thứ tư, nguyên mẫu hiện tại chủ yếu chứng minh tính khả thi của giải pháp về mặt kỹ thuật, chưa đạt mức độ hoàn thiện của một sản phẩm thương mại và chưa đủ cơ sở thực nghiệm để đưa ra những kết luận tuyệt đối về mô hình an toàn ở cấp độ hệ thống tổng thể.

Tóm lại, đề án đã hoàn thành mục tiêu đề xuất và hiện thực hóa cơ chế điều phối phi tập trung cho giao thức MLS trong môi trường P2P. Kết quả thu được khẳng định việc vận dụng MLS ngoài mô hình Delivery Service tập trung là hoàn toàn khả thi nếu được tích hợp một lớp điều phối phù hợp. Đồng thời, nguyên mẫu phần mềm được xây dựng là nền tảng thực tiễn để tiếp tục nghiên cứu, tối ưu hóa và mở rộng trong các công trình khoa học tiếp theo.

6.2 Hướng phát triển trong tương lai

Định hướng phát triển đầu tiên trong tương lai là tiếp tục hoàn thiện cơ sở lý thuyết của giao thức đề xuất. Cụ thể, cần xây dựng các phân tích hình thức (formal analysis) chặt chẽ đối với các bất biến của hệ thống như tính an toàn Single-Writer, tính đơn điệu của epoch, khả năng hội tụ sau Fork Healing và giới hạn an toàn của cơ chế Autonomous Replay. Việc áp dụng các phương pháp kiểm chứng hình thức sẽ giúp gia tăng tính chặt chẽ và giá trị học thuật của nghiên cứu.

Định hướng phát triển thứ hai là mở rộng phạm vi đánh giá thực nghiệm trên các môi trường mạng sát với thực tế. Các thí nghiệm cần được triển khai trên môi trường phân tán thực thụ (nhiều tiến trình, nhiều máy trạm) với các điều kiện mạng

khắc nghiệt như độ trễ cao, tỷ lệ mất gói lớn, hiện tượng thay đổi thành viên liên tục (churn) hoặc phân mảnh mạng kéo dài. Đồng thời, hệ thống cần được đánh giá qua các chỉ số đo lường hiệu năng nâng cao như thông lượng (throughput), độ trễ phân vị (p95, p99), thời gian hội tụ sau phân mảnh và chi phí lưu trữ tương ứng với quy mô nhóm, nhằm cung cấp cái nhìn toàn diện về khả năng chịu tải.

Định hướng phát triển thứ ba tập trung vào tối ưu hóa kiến trúc triển khai giữa Go Host và Rust Sidecar. Mặc dù mô hình phi trạng thái (stateless) hiện tại giúp phân tách trách nhiệm rõ ràng, nó đồng thời tạo ra chi phí bổ sung đáng kể đối với các nhóm có quy mô thành viên đông đảo. Quá trình tối ưu hóa có thể tập trung vào việc giảm chi phí IPC, nghiên cứu cơ chế lưu trạng thái dạng delta hoặc checkpoint hiệu quả hơn, đồng thời vẫn bảo đảm Go/SQLite là nguồn sự thật bền vững của hệ thống.

Định hướng phát triển thứ tư là nâng cấp nguyên mẫu ứng dụng để đạt độ ổn định và tính thực tiễn cao hơn. Các cải tiến tiềm năng bao gồm tối ưu hóa cơ chế truyền tải tệp tin, nâng cao khả năng đồng bộ dữ liệu khi thiết bị kết nối lại, hoàn thiện các công cụ giám sát và chẩn đoán lỗi hệ thống, cũng như phát triển giao diện tương tác đa nền tảng. Ngoài ra, việc bổ sung các cơ chế quản trị nhóm cấp cao, phân quyền chi tiết và lưu vết hoạt động (audit log) sẽ giúp ứng dụng đáp ứng tốt hơn các tiêu chuẩn triển khai thực tế.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

TÀI LIỆU THAM KHẢO

- [1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara **and** K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420, **july** 2023. DOI: 10.17487/RFC9420 **url:** <https://www.rfc-editor.org/rfc/rfc9420.html>
- [2] S. Turner, R. Barnes, B. Beurdouche, R. Robert **and** E. Omara, *The Messaging Layer Security (MLS) Architecture*, RFC 9750, **february** 2025. DOI: 10.17487/RFC9750 **url:** <https://www.rfc-editor.org/rfc/rfc9750.html>
- [3] T. Perrin, M. Marlinspike **and** R. Schmidt, *The Double Ratchet Algorithm*, Signal Specifications, Revision 4, 2025. **url:** <https://signal.org/docs/specifications/doubleratchet/>
- [4] WhatsApp, *WhatsApp Encryption Overview*, Whitepaper, 2026.
- [5] Matrix.org, *Megolm group ratchet*, Matrix Specification, version 1.18, 2026.
- [6] J. Alwen, M. Mularczyk **and** Y. Tselekounis, “Fork-Resilient Continuous Group Key Agreement,” *in Advances in Cryptology – CRYPTO 2023* Springer, 2023, **pages** 3–34.
- [7] K. Kohbrok, *Decentralized Messaging Layer Security*, IETF Internet-Draft, Work in progress, 2023.
- [8] S. Gilbert **and** N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, **jourvol** 33, **number** 2, **pages** 51–59, 2002. DOI: 10.1145/564585.564601
- [9] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva **and** M. Leone, “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” University at Buffalo, techreport, 2014. **url:** <https://cse.buffalo.edu/tech-reports/2014-04.pdf>
- [10] libp2p Project, *libp2p Documentation: Concepts, PubSub, Peer Discovery, NAT Traversal*, Online Documentation, 2026. **url:** <https://docs.libp2p.io>
- [11] K. Kohbrok, *Decentralized Messaging Layer Security*, Internet-Draft draft-kohbrok-mls-dmls-03, Expired Internet-Draft, work in progress, **october** 2025. **url:** <https://datatracker.ietf.org/doc/draft-kohbrok-mls-dmls/>

- [12] S. Scott-Hayward, G. O’Callaghan **and** S. Sezer, “SDN security: A survey,” **in** *2013 IEEE SDN For Future Networks and Services (SDN4FNS)* IEEE, 2013, **pages** 1–7.
- [13] K. Ashton **and** others, “That ‘internet of things’ thing,” *RFID journal*, **jourvol** 22, **number** 7, **pages** 97–114, 2009.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐATN

A.1 Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đề án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.2 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.3 Đánh dấu (bullet) và đánh số (numering)

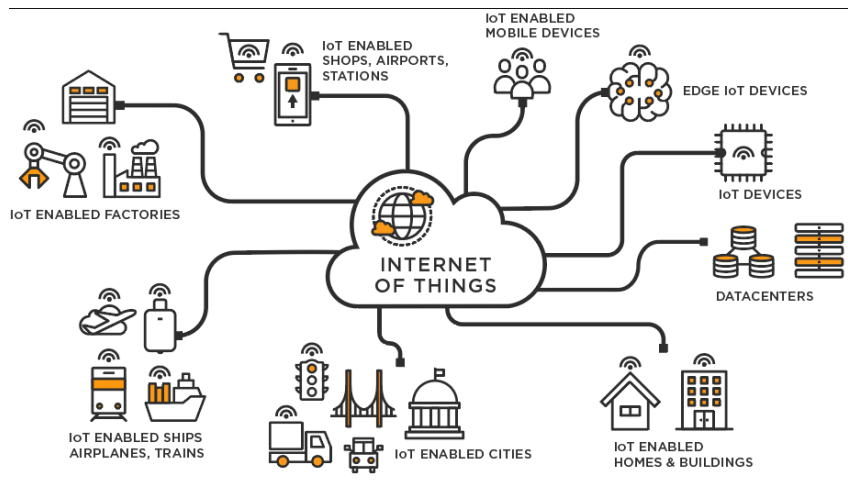
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.4 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.5 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.6 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [12]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [13]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.7 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

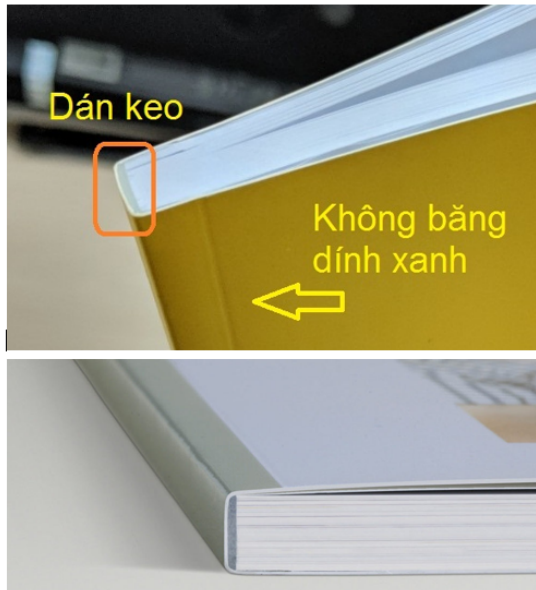
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.8 Qui cách đóng quyển

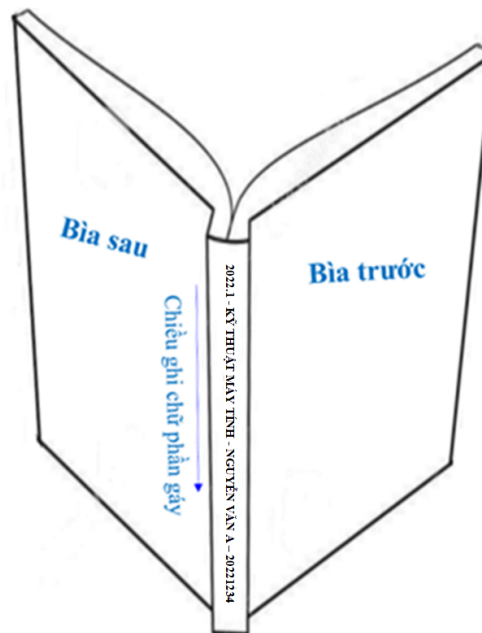
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.2 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...